
LaTeX for Research Students

A beginner's guide with examples, nuances & exercises

Syed Ali Mohsin Bukhari ^{1, 2} and Iqra Siddique ^{3, 4, 5}

¹Department of Applied Mathematics and Statistics, Institute of Space Technology,
Islamabad 44000, Pakistan

²Space and Astrophysics Research Lab (SARL), National Centre of GIS and Space
Applications (NCGSA), Islamabad 44000, Pakistan

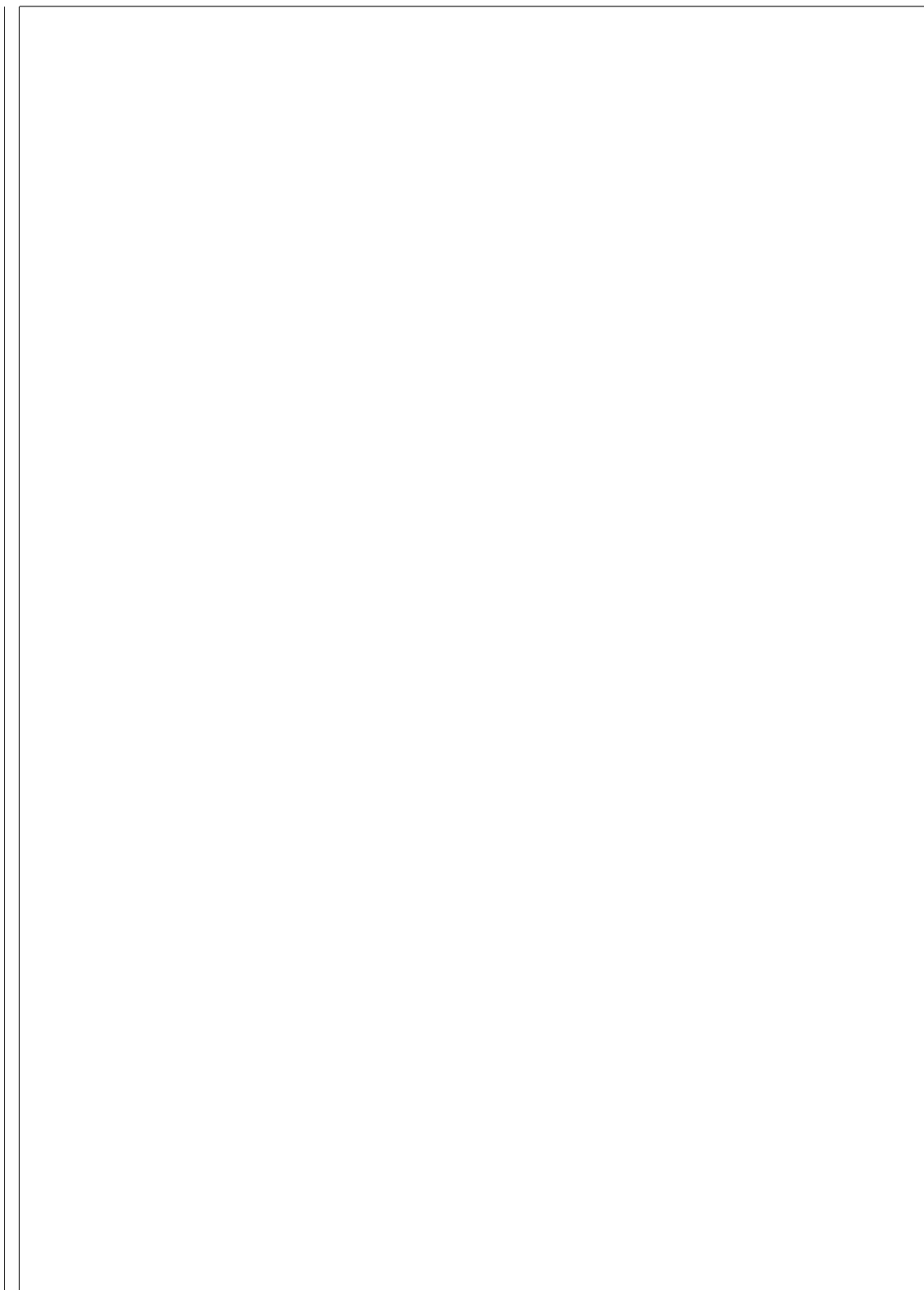
³Gran Sasso Science Institute (GSSI), Viale Crispi 7, I-67100 L'Aquila, Italy

⁴Dipartimento di Fisica, Università di Trento, Via Sommarive 14 I-38123 Trento,
Italy

⁵INFN Laboratori Nazionali del Gran Sasso, Via Acitelli 22, I-67100 Assergi,
L'Aquila, Italy

July 26, 2026

Drafting assistant: Claude (Anthropic) · Reviewer: DeepSeek (High-Flyer)



Contents

Status — Work in Progress	1
LaTeX for Research Students: What it is?	2
Part 1: Foundations & Capstone Kickoff (Chapters 1–4)	3
Chapter 1 — Why LaTeX for Research Writing	5
What LaTeX is	5
Why researchers reach for it	6
When Word might still be fine	6
Chapter 2 — Setting Up Overleaf & Your First Document	7
Getting set up	7
Choosing a Compiler	8
Stopping at the First Error	8
Chapter 3 — Anatomy of a .tex File and Survival Debugging	9
The two zones of every document	9
Document Class & Package Options	10
Character Encoding with <code>inputenc</code>	10
Survival Basics: reading the log without panicking	11
Chapter 4 — Capstone Kickoff: Choosing Your Topic & Project Setup	13
The Capstone Project	13
Step 1 — Pick a topic	13
Step 2 — Set up the project shell	14
Part 2: Core Typesetting (Chapters 5–8)	15
Chapter 5 — Text Formatting & Sectioning	17
Inline Formatting & Sectioning	17
Line Breaks, Paragraphs & Non-Breaking Spaces	18
Sentence-Ending Spacing	18
Starred Variants (*)	19
Controlling TOC Depth & Heading Numbering	20
Line Spacing with <code>setspace</code>	20

Chapter 6 — Lists & Tables	23
Lists: <code>itemize</code> & <code>enumerate</code>	23
Customizing List Labels with <code>enumitem</code>	24
Professional Tables with <code>booktabs</code>	25
Multi-Row & Multi-Column Cells	26
Wide Tables: <code>sidewaystable</code> & <code>landscape</code>	26
Aligned Decimal Columns with <code>siunitx</code>	28
Other Table Tricks	29
Chapter 7 — Figures & Floats	31
Including a Basic Figure with <code>\includegraphics</code>	31
Float Placement Specifiers	32
Controlling Where Floats Actually Land	32
Vector vs. Raster Images	33
Handling Legacy <code>.eps</code> Figures	33
Multi-Panel Figures with <code>subcaption</code>	34
Chapter 8 — Cross-Referencing, Labels & Hyperlinks	37
Automatic Numbering with <code>\ref</code> and <code>\cref</code>	37
Styling Hyperlinks with <code>\hypersetup</code>	38
The Labeling-Prefix Convention	39
Part 3: Math Mechanics (Chapters 9–10)	41
Chapter 9 — Inline & display math basics/fonts & units	43
Inline vs. display math	43
Greek letters and sub/superscripts	44
Italics vs. Roman in Math	45
Formatting Units	45
Bold Vectors & Greek Letters with <code>bm</code>	47
Putting It Together	47
Chapter 10 — Advanced Math: Matrices, Multi-line Equations, Cases	49
Matrices	49
Piecewise Functions with <code>cases</code>	50
Multi-Step Derivations	50
Controlling Fraction Size	51
Delimiter Sizing	51
One-Sided Delimiters	52
Math Spacing	52
Part 4: Math Structures & Algorithms (Chapters 11–12)	55
Chapter 11 — Theorems, Definitions, Proofs (<code>amsthm</code>)	59
Chapter 12 — Algorithms & Pseudocode	63
Reference: common <code>algorithm2e</code> commands	63

Part 5: Bibliography & Citation Tools (Chapters 13–16)	67
Chapter 13 — Managing References with BibTeX/BibLaTeX	71
Where do .bib entries actually come from?	71
The Compiler Decision Tree	75
Chapter 14 — Citation Styles for Journals & Conferences	77
Chapter 15 — Reference Managers + Overleaf	79
Chapter 16 — Glossaries, Nomenclature & Acronyms	81
Part 6: Diagrams & Data Visualization (Chapters 17–18)	85
Chapter 17 — Diagrams: Vector Graphics Workflow & TikZ (Elective)	89
The primary path: draw elsewhere, export as vector PDF	89
Chapter 18 — Plotting Data with PGFPlots	93
Part 7: Macros & Templates (Chapters 19–20)	97
Chapter 19 — Custom Commands & Preamble Management	101
Chapter 20 — Thesis/Paper Templates, Multi-file Projects & Appendices	105
Part 8: Collaboration & Diagnostics (Chapters 21–22)	111
Chapter 21 — Version Control, Collaboration & Journal Submission	115
Chapter 22 — Troubleshooting Common Errors (Full Reference)	119
Part 9: Capstone Assembly (Chapter 23)	123
Chapter 23 — Capstone Assembly	127
Appendix	131
A.1 — Symbol & Command Cheat Sheet	135
A.2 — Table Formatting Cheat Sheet	137
minipage — and how it differs from subfigure	138
A.3 — Page Layout & Spacing Reference	141
A.4 — Common Package Reference Table	143
A.5 — Compiler Comparison	147

A.6 — <code>tikz-cd</code> and <code>chemfig</code> Starter Snippets	149
A.7 — Quick Troubleshooting Checklist	151
A.8 — Starter <code>.bib</code> Files	153
A.9 — <code>hypersetup</code> Snippet	155
Supplementary Materials	157
Ch. 5 — Dense Table for Testing <code>setspace</code>	161
Ch. 6 — Full Working Example: Table “Dark Arts” Combined	163
Ch. 7 — Full Working Example: A Stuck Float, Before and After <code>\clearpage</code>	165
Ch. 9 — Reverse-Engineering Sample Targets	167
Ch. 11 — Simple Theorem Bank for Beginners	169
Ch. 11 — Exercise 2, Solved (Full Working Example)	171
Ch. 12 — Full Working Examples (Covering Most Reference-Table Commands)	173
Ch. 13 — Starter <code>.bib</code> Files for the 4 Capstone Topics	175
Ch. 16 — Glossary Printing Command Reference	181
Ch. 17 — Full Working Example: Neural Network Architecture Diagram	183
Ch. 18 — Sample Dataset (<code>ch18_sample_data.csv</code>)	187
Ch. 18 — Full Working Example (Complete Document)	189
Ch. 20 — Full Working Example: Two-Column Layout with a Spanning Figure	191
Ch. 20 — Full Working Example: Front/Back-Matter Skeleton (book Class)	193
Ch. 23 — Full Assembled Skeleton (Every Chapter, One Document)	195

Status — Work in Progress

Only Chapters 1–9 (Parts 1–3) are fully furnished — polished, complete, and ready to read.

Chapter 10 is currently being reworked — design decisions, headings, and section-level content are actively being revised.

Chapters 11–23 (Parts 4–9), the Appendix, and the Supplementary Materials are drafted but not finished — the instructional content is there, but it still needs fleshing out and structural/visual polish before it matches the standard of Chapters 1–9.

LaTeX for Research Students: What it is?

This guide takes you from your first compiled document to a complete, submission-ready research paper. It's written for students in STEM and other research fields who are new to LaTeX, and it assumes no prior experience — just a willingness to work through the examples and exercises as you go. A running capstone project threads through the chapters: starting in Chapter 4 you build one paper piece by piece, so that by the end you have both the knowledge and a finished document to show for it.

The material is organized into nine Parts (Chapters 1–23), followed by an Appendix of reference material and a Supplementary Materials file holding larger code samples and datasets referenced from the chapters.

Part 1: Foundations & Capstone Kickoff (Chapters 1–4)

--

Chapter 1 — Why LaTeX for Research Writing

Every research document eventually raises the same question: why fight with markup and a compiler instead of just clicking bold in a word processor? It doesn't pay off for everything — but for the kind of writing this guide is aimed at, it pays off often enough that most STEM researchers end up using it sooner or later.

This chapter covers what LaTeX actually is, the concrete reasons researchers reach for it over Word, and — just as importantly — the cases where it isn't the right tool, so the choice to use it is deliberate rather than automatic.

Objective 1

Understand what LaTeX actually is, why research papers and theses are so often written in it, and when it's the right tool for the job.

What LaTeX is

LaTeX is a *typesetting system*, not a word processor. Instead of clicking buttons to make text bold or center a heading, you write plain text with markup commands, and a compiler turns that markup into a formatted PDF. A Word document is WYSIWYG (“what you see is what you get”); a LaTeX document is closer to “what you *describe* is what you get.”

Worked Example 1.1

```
\documentclass{article}

\begin{document}
  Hello, research world.
\end{document}
```

Nuance 1.1

Compiling this produces a properly typeset PDF page — margins, fonts, and spacing all handled by LaTeX's rules, not by you nudging things around manually.

Why researchers reach for it

A handful of concrete advantages come up again and again once you’ve used it on a real project:

- **Math typesets beautifully and consistently.** An equation with nested fractions, summations, or matrices is a few lines of markup instead of a fight with an equation editor.
- **Long documents stay consistent.** Change the font size of every section heading in a 200-page thesis with one line, not 40 manual edits.
- **Bibliographies are managed, not typed.** Citations and reference lists are generated from a database file — add a source once, cite it anywhere (Part IV covers this in depth).
- **It’s the default for most STEM journals.** Many journals provide official LaTeX templates (`.cls` files) and *prefer* or *require* LaTeX submissions.
- **Plain text plays well with version control.** A `.tex` file is just text, so tools like Git track meaningful changes (Ch. 21 picks this up).

When Word might still be fine

LaTeX has a learning curve. For a one-page memo, a quick letter, or a document with no math and no references, Word (or Google Docs) will get you there faster. LaTeX pays off as documents get longer, more mathematical, and more collaborative — which describes most research writing, but not all writing.

(Context chapter — no formal exercise. If you want a first taste, try compiling the snippet above once you reach Ch. 2.)

Chapter 2 — Setting Up Overleaf & Your First Document

Chapter 1 made the case for LaTeX in the abstract; this one gets an actual compiled PDF in front of you, which matters more than any argument for why the tool is worth learning.

This chapter covers creating an Overleaf account and project, compiling your first document, and two settings — compiler choice and “stop on first error” — that are easy to skip past but save real time once documents get more complex.

Objective 2

Create an Overleaf account, start a project, understand the compile button, and produce your first compiled PDF.

Getting set up

1. Go to overleaf.com and create a free account (a university email often unlocks extra features).
2. Click **New Project** → **Blank Project**. This opens an editor pane (your `.tex` code) next to a PDF preview pane.
3. Click the green **Recompile** button. Overleaf runs the LaTeX compiler and renders the PDF on the right.

No local software install is required — this is why Overleaf is the recommended starting point for this guide. Replace the default project content with:

Worked Example 2.1

```
\documentclass{article}

\title{My First LaTeX Document}
\author{Your Name}
\date{\today}

\begin{document}

  \maketitle
```

```
\section{Introduction}
This is my first properly structured LaTeX document.
It has a title, an author, a date, and a numbered section.

\end{document}
```

Recompile, and you should see a title block (title, author, today’s date) followed by a numbered section. The `\maketitle` command is what actually renders the title/author/date you declared above it — forgetting this call is a common first mistake (nothing shows up even though you set a title).

Choosing a Compiler

Overleaf’s compiler choice quietly decides which packages even work, which is worth knowing before a font package mysteriously fails to compile. It lets you choose the compiler under the project’s **Menu** → **Compiler**. This matters more than it looks:

Important 2.1

If you use the `fontspec` package to change fonts, you *must* compile with XeLaTeX or LuaLaTeX — pdfLaTeX can’t process it. If you don’t need custom fonts, stick with pdfLaTeX; most journals require it for submission, and most packages assume it by default.

Stopping at the First Error

Left on Overleaf’s default “Always compile,” a single missing `$` can cascade into fifty confusing follow-on errors that all trace back to one broken line.

Important 2.2

Open the **Recompile** dropdown (small arrow next to the button) and check “**Stop on first error.**” Stopping at the first error tells you exactly which line broke — turn this on now, you’ll thank yourself in Ch. 3. The error LaTeX reports will likely point to the line *after* the missing brace (e.g., your `\author{...}` line), not the `\title{...}` line that’s actually broken. This is normal — LaTeX doesn’t realize a brace is unclosed until it hits the *next* command. When you see a strange error, always check the line above first.

Exercise 2.1

Create and compile a title page for a fictional paper:

1. Set a `\title{}`, two `\author{}` names separated by `\and`, and `\date{}`.
2. Add one section with two sentences of placeholder text.
3. Confirm “Stop on first error” is enabled, then deliberately delete the closing `}` on your title, and recompile — read the error message before fixing it.

Chapter 3 — Anatomy of a .tex File and Survival Debugging

Every .tex file you'll ever open — yours or someone else's — is built from the same two zones and the same handful of moving parts, so recognizing them on sight is what turns an unfamiliar file from intimidating into just another document.

This chapter covers that anatomy (preamble vs. body, document class, packages, options), plus the minimum debugging habits — reading the compile log, and compiling often — needed to survive your first real errors without panicking.

Objective 3

Understand the parts of a .tex file (preamble vs. body, document class, packages) well enough to read *any* LaTeX file, and learn the minimum debugging skills to survive your first errors.

The two zones of every document

Worked Example 3.1

```
\documentclass[12pt]{article}    % <- PREAMBLE starts here
\usepackage{amsmath}           %   packages, settings, custom commands
\usepackage{graphicx}          %   (nothing here is "printed")

\begin{document}               % <- BODY starts here
This text appears in the PDF.   %   everything you write here is rendered
\end{document}                 % <- document ends
```

- **Preamble** (everything before `\begin{document}`): declares the document class, loads packages, and configures global settings. Nothing here appears in the output.
- **Body** (between `\begin{document}` and `\end{document}`): the actual content that gets typeset.
- **Document class** (`\documentclass{article}`): sets the overall document type (`article`, `report`, `book`, or a university/journal-specific class). It determines what section levels exist and how they're numbered.

- **Packages** (`\usepackage{...}`): each one adds a capability — `amsmath` for advanced math, `graphicx` for images, and so on. You’ll accumulate a standard set of these as you go through this guide.

Document Class & Package Options

Both `\documentclass` and `\usepackage` accept a bracketed list of options that tweak behavior without changing the class or package name itself — worth recognizing as one consistent syntax rather than memorizing each occurrence separately.

Nuance 3.1

The `[12pt]` font size in `\documentclass[12pt]{article}` above, and the `[utf8]` option in `\usepackage[utf8]{inputenc}` later in this chapter, are both **options** — a comma-separated list in square brackets that configures how the class or package behaves, separate from the class/package name itself.

General 3.1 — ‘documentclass’ options

A few common `\documentclass` options worth knowing by name, even before you need them:

- **Font size**
 - `10pt` (the default if you omit this), `11pt`, `12pt` — sets the base text size for the whole document.
- **twoside/oneside**
 - Whether the document is meant for double-sided printing, which affects margins (alternating sides for binding) and page-parity behavior (Ch. 7’s `\cleardoublepage`).
 - `article` and `report` default to `oneside`; only `book` defaults to `twoside`.
- **twocolumn**
 - Starts the entire document in two-column layout from the first page.
 - Ch. 20 covers switching mid-document with `\onecolumn`/`\twocolumn` instead, but if the *whole* document needs two columns, as most two-column journal templates do, setting it here as a class option is simpler than switching manually.
- **draft**
 - Speeds up compiling by replacing images with outlined boxes and flagging overfull lines with a black bar in the margin.
 - Useful while iterating on a long document; remove it before your final compile.

Multiple options are comma-separated: `\documentclass[12pt,twoside,draft]{report}`.

Character Encoding with inputenc

Typing an accented letter or a pasted “smart quote” directly into a pdfLaTeX source file needs one extra declaration before the compiler interprets it correctly instead of choking on it.

`\usepackage[utf8]{inputenc}` declares that your source `.tex` file is written in UTF-8 — this is what lets pdfLaTeX correctly interpret non-ASCII characters typed directly into the file (accented letters, “smart quotes” pasted from elsewhere, and so on). It’s specific to pdfLaTeX: XeLaTeX

and LuaLaTeX (Ch. 2’s compiler nuance) are UTF-8-native by default and don’t need it — loading `inputenc` alongside them is harmless, just redundant.

Survival Basics: reading the log without panicking

When a document compile fails, Overleaf highlights the offending line and shows a message in the log panel below the editor. The two most common first errors:

Message	Usual cause
Undefined control sequence	You typo’d a command name, or forgot to load the package that defines it
Missing \$ inserted	You used a math-only character (like <code>_</code> or <code>^</code>) outside math mode, or left a <code>\$</code> unclosed

The habit that saves the most time: compile *often* — after every few lines, not after writing a whole page. If something breaks, you know it’s in the last few lines you added, not buried somewhere in 200 lines of text. Combined with “Stop on first error” from Ch. 2, this turns debugging from a hunt into a quick glance.

(This is deliberately the *minimum* survival kit. Ch. 22 returns to debugging in full depth once you’ve hit enough real errors to appreciate it.)

Worked Example 3.2

```
\documentclass{article}
\usepackage[utf8]{inputenc}
\usepackage{graphicx}

\title{Anatomy of a LaTeX File}
\author{Your Name}

\begin{document}
  \maketitle

  \section{Preamble vs Body}
  Everything above this document's begin-document tag is configuration.
  Everything below it is content.

\end{document}
```

This snippet compiles cleanly as-is. The exercise below will have you introduce a deliberate error yourself, so you get practice diagnosing one rather than hunting for a phantom bug.

Exercise 3.1

1. Build the document skeleton above from scratch (don’t copy-paste) so the structure sticks.
2. Deliberately break it: remove the `\usepackage{graphicx}` line, then add

`\includegraphics{}` anywhere in the body. Recompile and read the resulting **Undefined control sequence** error — this is exactly what it looks like when you use a command without loading the package that defines it.

3. Fix it (restore the `\usepackage{graphicx}` line), recompile, and confirm the PDF renders cleanly.

Chapter 4 — Capstone Kickoff: Choosing Your Topic & Project Setup

The first three chapters covered LaTeX in the abstract — what it is, how to compile it, what a file is made of. This chapter is where that knowledge turns into an actual, evolving document: you pick a topic and stand up a minimal project shell, and every chapter from here through Ch. 23 adds one more piece to it.

Objective 4

Choose a capstone topic and set up the minimal, compiling project shell that every later chapter's Capstone Update will build onto.

The Capstone Project

Starting now, every remaining chapter (5 through 22) ends with a short **Capstone Update** — you apply that chapter's lesson directly to one evolving paper. By Ch. 23, you'll have assembled a complete, submission-formatted document built piece by piece.

Step 1 — Pick a topic

Choose whichever of these four fits your field (or is closest to your own research — feel free to substitute your real topic once you're comfortable):

1. **Physics:** Thermal conductivity of graphene composites.
2. **CS/Math:** Convergence analysis of a gradient descent algorithm.
3. **Engineering:** Stress-strain analysis of a cantilever beam.
4. **Biology/Chem:** Reaction rate of enzyme kinetics.

Starter `.bib` files (5 dummy references each) for all four topics are provided in the guide's supplementary materials, so you'll have real citations to work with from Part IV onward regardless of which you pick.

Step 2 — Set up the project shell

Create a fresh Overleaf project and start it with a minimal, honest skeleton — no content yet, just the scaffolding every later chapter will build onto:

Worked Example 4.1

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}

\title{[Working Title - will be refined in Ch. 5]}
\author{Your Name}
\date{\today}

\begin{document}
  \maketitle

  \begin{abstract}
    [Placeholder: one-sentence summary of your chosen topic.
    Will be expanded as later chapters add content.]
  \end{abstract}

  \section{Introduction}
  % Content added starting in Ch. 5

\end{document}
```

Compile it once now to confirm the shell is error-free before building on it.

Don't worry if the abstract wording isn't perfect yet — it's a placeholder. You'll rewrite it properly once later chapters give you actual content (results, methods) to summarize.

Capstone Update 4

Create your project shell:

- Pick one of the four topics (or your own, if you're confident enough to skip the training wheels).
- Set the title, author, date, and a one-line abstract placeholder as shown above.
- Compile successfully — this file is what every following chapter's Capstone Update will extend.

Part 2: Core Typesetting (Chapters 5–8)

--

Chapter 5 — Text Formatting & Sectioning

Every document beyond a single page needs two things sorted out early: basic inline emphasis (bold, italic — for terms, warnings, foreign phrases) and a heading hierarchy that actually holds a long document together. Getting the second one wrong is what causes the most confusion later — a thesis with inconsistent section numbering, or a table of contents that silently drops a heading level, is almost always a symptom of never having set `tocdepth`/`secnumdepth` explicitly, or of nesting sections one level deeper than the document class expects.

This chapter covers both: `\section`/`\subsection`/`\subsubsection` for structured formatting, `\textbf`/`\textit` and friends for inline formatting, and `\tableofcontents` to generate a TOC from that structure automatically. It also covers two habits — starred (unnumbered) variants, and `\setspace` for draft line-spacing — that show up constantly from here on, so it's worth having them settled before the rest of the guide starts assuming you know them.

Objective 5

Format text (bold, italic), structure a document with sections/subsections, and generate a working table of contents.

Inline Formatting & Sectioning

Every document needs the same two building blocks early on: a way to emphasize individual words inline, and a heading hierarchy — `\section` down to `\subsubsection` — that `\tableofcontents` can turn into a working table of contents.

Worked Example 5.1

```
\documentclass{article}
\usepackage[utf8]{inputenc}

\title{Formatting \& Sectioning Basics}
\author{Your Name}

\begin{document}
  \maketitle
```

```

\tableofcontents

\section{Introduction}
This is \textbf{bold} text and this is \textit{italic} text.
You can also combine them: \textbf{\textit{bold italic}}.

\subsection{Background}
A subsection nested under the introduction.

\subsubsection{Fine Detail}
A subsubsection nested under the subsection.

\end{document}

```

Nuance 5.1

Recompile twice if the table of contents looks empty or stale on the first try — LaTeX writes the TOC to an auxiliary file during compilation and reads it back in on the *next* compile. This is normal, not a bug.

Line Breaks, Paragraphs & Non-Breaking Spaces

LaTeX treats a line break, a new paragraph, and a plain space as three separate things. Mixing them up is what produces a paragraph with no indent, or a line that breaks in an ugly spot.

Nuance 5.2

Three small habits worth getting right early, since you'll use all three constantly from here on:

- `\` forces a line break without starting a new paragraph (no first-line indent) — you'll see it inside `align`/`gather` environments in later chapters.
- `\newline` behaves the same way in most contexts, but isn't universally interchangeable with `\` — inside a `tabular` row, for instance, only `\` works.
- A genuine new paragraph comes from a **blank line** in your source (or, equivalently, an explicit `\par`) — that's what actually gives you the indentation and spacing that "looks like" a new paragraph. `\` alone does not do this; it only breaks the line.
- `~` inserts a non-breaking space — one LaTeX will never break a line on. Use it to keep short, related pieces of text glued together, like `Figure~5` or `Dr.~Smith`, so a line break can't land awkwardly between them. You'll use this constantly starting in Ch. 8, once `\cref`/`\ref` are introduced — `Figure~\cref{fig:...}` is the standard pairing.

Sentence-Ending Spacing

LaTeX tries to guess whether a period ends a sentence purely from the letter case right before it, and that heuristic breaks down around abbreviations and initials — worth knowing so the fix

doesn't feel like a mystery the first time you need it.

Nuance 5.3

LaTeX inserts extra space after what it assumes is the end of a sentence, and it guesses based on the character right before the period:

- A period after a **lowercase** letter is assumed to end a sentence, and gets the wider spacing by default — correct most of the time, but wrong for a lowercase abbreviation used mid-sentence, like “e.g.” or “i.e.”. Fix it with a backslash-space or ~ right after the period: `e.g.\ apples` or `e.g.~apples`.
- A period after an **uppercase** letter is assumed to be an abbreviation (an initial, or something like “NASA”), and gets the narrower spacing by default — wrong when that period genuinely ends the sentence. Fix it with `\@` right before the period: `NASA\@. We went...`

If you'd rather not think about this distinction at all, `\frenchspacing` in your preamble disables it entirely and uses the same spacing everywhere — a reasonable default some style guides prefer.

Starred Variants (*)

A pattern shows up here for the first time that recurs constantly for the rest of the guide — appending `*` to a command or environment name to turn off its automatic numbering — so it's worth learning as a general reflex rather than a one-off trick for `\section`.

Nuance 5.4

Many LaTeX commands and environments have a **starred variant** — the same name with a `*` appended — which almost always means “the same thing, but unnumbered.” The most common first encounter is an unnumbered section, useful for things like an “Acknowledgments” heading that shouldn't get a number:

```
\section{Introduction}           % numbered: "1 Introduction"
\section*{Acknowledgments}      % unnumbered: just "Acknowledgments"
```

A rule of thumb is, whenever you see a trailing `*` on a LaTeX command or environment, assume “same behavior, minus the automatic numbering” and check the specific case for any extra detail.

Important 5.1

`\section*{}` also doesn't appear in the table of contents by default (since the TOC is built from numbered headings). If you want it listed anyway, add this line right after the `\section*{}` call:

```
\addcontentsline{toc}{section}{Acknowledgments}
```

Cross-Check 5.1 — Cross-Check: The star-pattern cheat sheet

This same star pattern reappears throughout the guide, so it’s worth recognizing on sight rather than re-learning each time:

- `equation*` (Ch. 9) — a display equation with no number.
- `align*` (Ch. 10) — multiple aligned equations, none numbered.
- `figure*/table*` — in two-column document classes (common in some journal templates), a float that spans both columns instead of sitting in just one.

Controlling TOC Depth & Heading Numbering

The document class quietly caps how deep the table of contents and heading numbers go, so a `\subsubsection` can look “wrong” for reasons that have nothing to do with how it was typed — two counters in the preamble are what actually control that depth.

Nuance 5.5

By default, `\subsubsection` and `\paragraph` headings can silently disappear from the TOC, or appear unnumbered, which causes panic the first time it happens. Control both behaviors explicitly in your preamble:

```
\setcounter{tocdepth}{3} % include subsubsections in the TOC
\setcounter{secnumdepth}{3} % number them too
```

Important 5.2

Heading depth in article:

1. `\section = 1,`
2. `\subsection = 2,`
3. `\subsubsection = 3,`
4. `\paragraph = 4.`

Many universities leave `\paragraph` unnumbered by default — these two counters are how you take control of that instead of guessing why a heading looks “wrong.”

Line Spacing with `setspace`

Draft and thesis formatting requirements frequently specify double or 1.5 line spacing, and hand-rolling that with `\vspace` or `\linespread` throws off table and math spacing elsewhere in the document — a dedicated package avoids the side effects.

Nuance 5.6

Thesis offices often require double or 1.5 line spacing for draft submissions; journals usually want single spacing. Don’t reach for `\vspace` or `\linespread{2}` by hand — it distorts table and math spacing. Use the `setspace` package instead:

```
\documentclass{article}

\usepackage{setspace}

\begin{document}
  \doublespacing
  % ... rest of the document is now double-spaced ...

  \begin{singlespace}
    % a dense table or the abstract,
    % kept readable even in an otherwise double-spaced document
  \end{singlespace}

\end{document}
```

Nuance 5.7

`\onehalfspacing` and `\singlespacing` work the same way if you need those instead.

Cross-Check 5.2 — Cross-Check

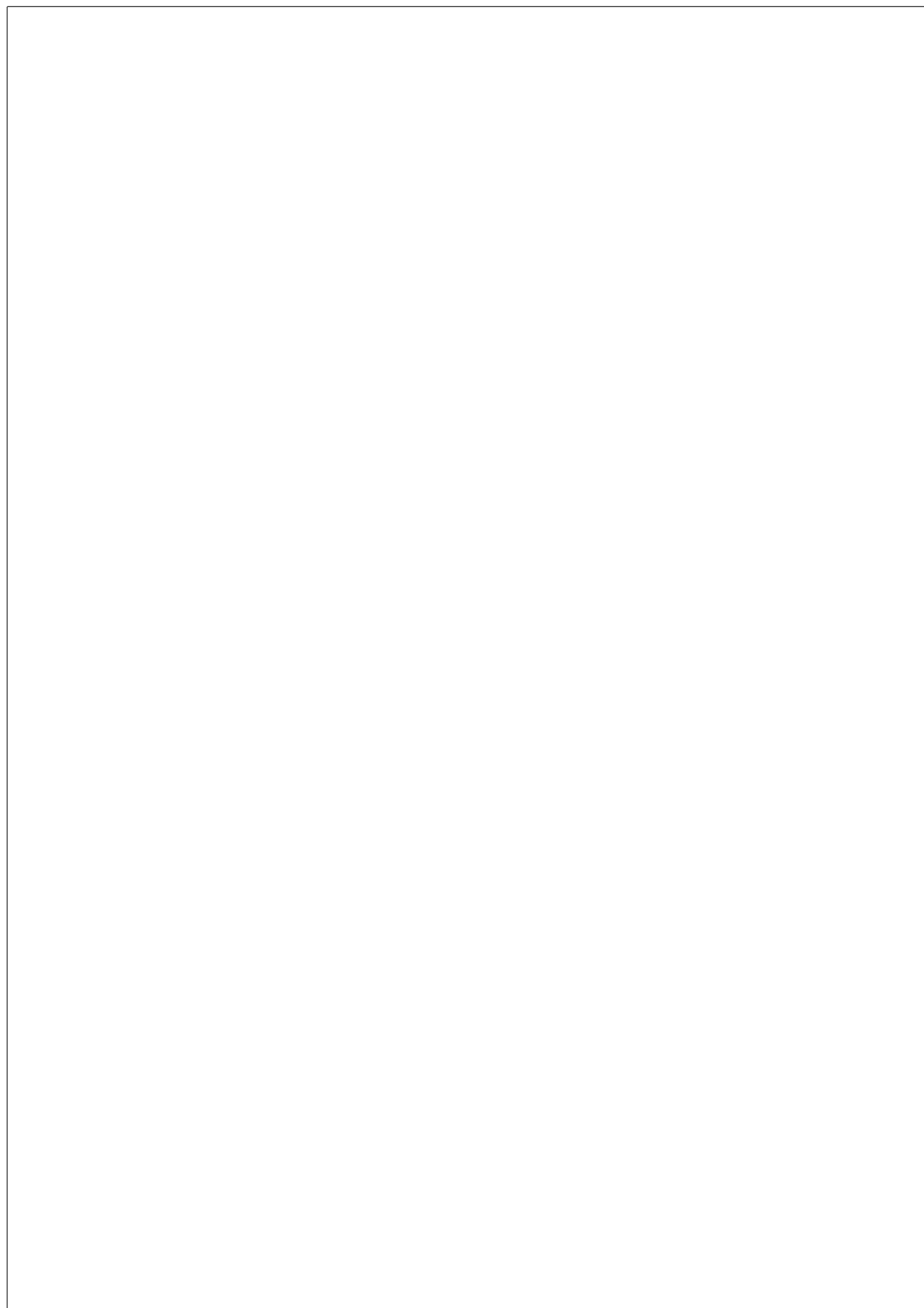
Want to see the effects for yourself, with a working example, rather than just read about it? `supplementary_materials.md` has a ready-made dense table you can paste in and toggle `singlespace` on/off around — see “Ch. 5 — Dense Table for Testing `setspace`.”

Exercise 5.1

1. Take your Ch. 4 capstone shell and add two more section levels (`\subsection`, `\subsubsection`) somewhere in the introduction.
2. Add `\tableofcontents` after `\maketitle` and recompile twice to see it populate.
3. Set `tocdepth/secnumdepth` to 3 and confirm your subsubsection now appears, numbered, in the TOC.

Capstone Update 5

- Draft real section headings for your paper (e.g. Introduction, Methods, Results, Discussion, Conclusion) instead of the single placeholder `\section{Introduction}` from Ch. 4.
- Decide whether your target format needs double-spacing (thesis draft) or single (journal submission), and set it with `setspace`.
- Confirm the TOC shows the depth you want.



Chapter 6 — Lists & Tables

Research writing constantly needs to present information as discrete points or structured data rather than flowing prose — a list of assumptions, a sequence of experimental steps, a table of measurements. LaTeX handles both natively, but its *default* output for each is noticeably plain: bullets/numbers with no styling control, and tables ringed with the boxy vertical/horizontal rules typical of a spreadsheet rather than a printed page. Neither is wrong, exactly, but neither is what you'll see in a published paper or a well-typeset thesis either.

This chapter starts from those defaults and builds up to what journals and thesis offices actually expect: customizable list labels (lettered, Roman numeral, tightly spaced), and tables built with `booktabs`'s minimal open rules and `siunitx`'s decimal-aligned numeric columns — plus fixes for the two problems wide tables run into (columns overflowing the page, and numbers that don't line up).

Objective 6

Format lists and build clean, professional-looking tables — including wide tables and tables with aligned numeric data.

Lists: `itemize` & `enumerate`

LaTeX's two built-in list environments cover the two everyday cases out of the box: `itemize` for unordered/bulleted points, `enumerate` for numbered steps.

Worked Example 6.1

```
\begin{itemize}
  \item First point
  \item Second point
  \item Third point
\end{itemize}

\begin{enumerate}
  \item Step one
  \item Step two
  \item Step three
\end{enumerate}
```

Nuance 6.1

The `itemize` environment gives bullets; `enumerate` environment gives numbers. Both can be nested inside each other.

Customizing List Labels with `enumitem`

Default bullets and numbers cover most cases, but journal or thesis styles sometimes want something different — lettered sub-items, Roman numerals, or a tighter list in a dense layout.

Nuance 6.2

The `enumitem` package handles all of it with one consistent syntax:

```
\documentclass{article}
\usepackage{enumitem}

\begin{document}

  \begin{enumerate}[label=(\alph*)]
    \item First
    \item Second
  \end{enumerate}
  % renders as (a), (b), ...

  \begin{enumerate}[label=\Roman*.]
    \item First
    \item Second
  \end{enumerate}
  % renders as I., II., ...

  \begin{itemize}[nosep, leftmargin=*]
    \item Tight, flush-left bullet
    \item No vertical space between items
  \end{itemize}

\end{document}
```

General 6.1 — Usage

- `label=`
 - Controls what marks each item — `\alph*`/`\Alph*` for lower/uppercase letters, `\roman*`/`\Roman*` for lower/uppercase Roman numerals, alongside the default `\arabic*`.
- `nosep`
 - Strips the vertical spacing between items, useful for a dense list inside a table cell or a tight layout.
- `leftmargin=*`
 - Flushes the list to the left margin instead of the default indent — common in some thesis styles that want lists aligned with body text.

Professional Tables with booktabs

A plain `tabular` with `\hline` and vertical bars (`|c|c|`) technically works, but reads as dated compared to the open, rule-light tables standard in published papers — `booktabs` is the package that gets you there:

Worked Example 6.2

```
\documentclass{article}
\usepackage{booktabs}

\begin{document}

  \begin{table}[htbp]
    \centering
    \caption{Sample thermal conductivity measurements}
    \label{tab:sample}
    \begin{tabular}{lcc}
      \toprule
      Material    & Conductivity (W/mK) & Temperature (K) \\
      \midrule
      Graphene A & 5000                & 300 \\
      Graphene B & 4200                & 350 \\
      \bottomrule
    \end{tabular}
  \end{table}

\end{document}
```

Nuance 6.3

`booktabs` gives you `\toprule`, `\midrule`, and `\bottomrule` instead of `\hline` — no vertical rules at all, which is the field standard for professional tables. A cheat sheet comparing the two styles is in the appendix.

Multi-Row & Multi-Column Cells

A table cell sometimes needs to span more than one row or column instead of holding a single value — a group header that sits above two sub-columns (“Measurement” spanning “Trial 1” and “Trial 2”), or a sample name that applies to several data rows underneath it and shouldn’t be repeated on every line. Plain `tabular` has no way to do either; two commands handle it, one column-wise and one row-wise:

Worked Example 6.3

```
\documentclass{article}
\usepackage{multirow}

\begin{document}
  \begin{tabular}{lcc}
    \toprule
    \multirow{2}{*}{Sample} & \multicolumn{2}{c}{Measurement} & \\
    & Trial 1 & Trial 2 & \\
    \midrule
    A & 10 & 12 & \\
    \bottomrule
  \end{tabular}
\end{document}
```

Nuance 6.4

- `\multicolumn{cols}{align}{text}` is built into LaTeX already.
- `\multirow{rows}{width}{text}` (from the `multirow` package) is the one that needs an explicit `\usepackage`.

Wide Tables: `sidewaystable` & `landscape`

A table with enough columns will overflow the page margins regardless of how well it’s styled, and two packages fix that by rotating things 90 degrees — they differ only in *how much* they rotate.

sidewaystable**Worked Example 6.4**

```

\documentclass{article}

\usepackage{rotating}

\begin{document}
  \begin{sidewaystable}
    \centering
    \caption{A wide table of supplementary data}
    \begin{tabular}{cccccccc}
      \toprule
        ... 8 columns of data ...
      \bottomrule
    \end{tabular}
  \end{sidewaystable}
\end{document}

```

Nuance 6.5

The one-line fix is the `rotating` package's `sidewaystable` environment — it rotates the entire table (and its caption) 90 degrees. Great for supplementary-materials tables that just don't fit portrait orientation.

landscape**Worked Example 6.5**

```

\documentclass{article}
\usepackage{pdflscape}

\begin{document}
  \begin{landscape}
    \begin{table}[htbp]
      \centering
      \caption{An especially wide table, needing a full landscape page}
      \begin{tabular}{cccccccccc}
        \toprule
          ... 10 columns of data ...
        \bottomrule
      \end{tabular}
    \end{table}
  \end{landscape}
\end{document}

```

Nuance 6.6

`sidewaystable` above rotates a single float on an otherwise-portrait page. Sometimes that's still not enough — a table wide enough that even a rotated float feels cramped, or you want a full landscape page with its own header and margins. That's a job for the `landscape` environment instead, from `lscape` (or `pdfscape` — the pdfLaTeX-safe version; use this one on Overleaf's default compiler, since it also sets the PDF's page-rotation metadata so viewers display it right-side-up).

Important 6.1

1. `sidewaystable` rotates a float within an otherwise-portrait page; `landscape` rotates the physical page itself.
2. Reach for `sidewaystable` first — it's simpler and keeps everything else on the page portrait — and only switch to `landscape` when a single rotated float genuinely isn't wide enough.

Aligned Decimal Columns with `siunitx`

Numeric results (regression coefficients, experimental measurements) when laid out in a plain `c` column are not eye friendly, or eye catching, since the decimal points don't align down the column. It also becomes hard to scan and a bit unprofessional-looking.

Worked Example 6.6

```
\documentclass{article}
\usepackage{siunitx}

\begin{document}
  \begin{tabular}{l S[table-format=3.2]}
    \toprule
    {Sample} & {Conductivity (W/mK)} \\
    \midrule
    A & 123.45 \\
    B & 98.60 \\
    \bottomrule
  \end{tabular}
\end{document}
```

Nuance 6.7

The `siunitx` package's `S` column type fixes this — declare the decimal precision once, and every row's decimal point lines up automatically.

Important 6.2

`S[table-format=3.2]` reserves 3 digits before and 2 after the decimal point, and aligns every row's decimal point vertically. Note the curly braces around non-numeric header text (`{Sample}`, `{Conductivity...}`) — the `S` column expects numbers by default, so text needs to be explicitly wrapped.

Other Table Tricks**General 6.2 — Table cheat sheet**

The tools below solve real, specific table problems, but aren't everyday defaults — reach for them only when you actually hit the edge case each one is for:

- **`@{} column specifier`** — removes the automatic padding LaTeX inserts between columns, for columns that should visually touch: `\begin{tabular}{l @{} c}`.
- **`\resizebox{\textwidth}{!}{...}`** (from `graphicx`, Ch. 7) — scales an entire table to fit the page width. Use sparingly: it shrinks the font along with everything else, which can make a table technically fit but practically unreadable.
- **`tablefootnote`** — a regular `\footnote` breaks inside a `table` float (it either silently disappears or lands on the wrong page); `tablefootnote` fixes that.
- **`threeparttable`** — for a table that needs *several* notes tied to the table's own width rather than the page's — a more formal notes block underneath, versus `tablefootnote`'s one quick note at a time.
- **`minipage`** — places a table next to text, or next to another small table, side by side rather than stacked.
- Row height can also be adjusted, globally or per-table, with `\arraystretch` — covered in full in Ch. 19, once `\renewcommand` has been introduced.

Cross-Check 6.1 — Cross-Check

See `supplementary_materials.md` for a single table that puts several of these to work together — more useful to see combined than as isolated one-liners.

Exercise 6.1

1. Recreate this data as a table using `booktabs`: `Sample, Value / A, 12.5 / B, 7.3 / C, 150.25`
2. Do it twice: once with a plain `c` column, once with an `S[table-format=3.2]` column — compare how the decimals line up.
3. Bonus: wrap the same table in `sidewaystable` and observe the orientation change.
4. Reformat the `enumerate` list from the very first worked example using `enumitem`: switch it to lettered labels (a), (b), (c), and tighten it with `nosep`.

Nuance 6.8

When using the **S** column, wrap your column headers in curly braces, e.g. `{Value}`, so `siunitx` doesn't try to parse them as numbers and throw an "Invalid token" error.

Capstone Update 6

Add a result table to your paper using the topic's own data (real or plausible placeholder numbers), with `booktabs` for the rules and **S** columns for any numeric measurements. If your table ends up especially wide or needs a footnoted value, this is also a good place to try `sidewaystable/landscape` or `tablefootnote` for real instead of just in the exercise.

Chapter 7 — Figures & Floats

Almost every research document needs to include images — plots, diagrams, photographs of apparatus — and LaTeX treats them as *floats*: elements the layout engine is free to place wherever it fits best on the page, rather than pinning exactly where you wrote the code. That’s a deliberate design choice (it avoids ugly page breaks splitting a figure across two pages), but it’s also the single most common source of “why did my figure end up three pages later than I put it” confusion for anyone coming from a word processor, where images stay exactly where you drop them.

This chapter covers `\includegraphics` and the `figure` environment for the basic case, placement specifiers and `\clearpage` for controlling *where* floats actually land, vector-vs-raster format choice, and `subcaption` for multi-panel figures — the building blocks for the labeled, cross-referenceable figures Chapter 8 will start pointing back to by number.

Objective 7

Include images, caption and label them correctly, control where floats land on the page, and lay out multi-panel figures.

Including a Basic Figure with `\includegraphics`

Getting an image onto the page at all is the first hurdle — LaTeX has no native concept of an image file, so `\includegraphics` (from the `graphicx` package) is the command that bridges the gap, and wrapping it in a `figure` environment is what turns a bare image into a captioned, labeled, referenceable float.

Worked Example 7.1

```
\documentclass{article}

\usepackage{graphicx}

\begin{document}

  \begin{figure}[htbp]
    \centering
    \includegraphics[width=0.6\textwidth]{example-image}
    \caption{A sample figure.}
```

```

\label{fig:sample}
\end{figure}

\end{document}

```

Nuance 7.1

`example-image` is a built-in Overleaf/LaTeX placeholder (from the `mwe` package family) that renders without needing to upload a real file — useful while drafting before you have your actual figures ready.

Float Placement Specifiers

That `[htbp]` in the figure’s opening brackets looks like a fixed instruction, but LaTeX treats it as a ranked list of acceptable spots rather than a command to obey exactly — which is exactly why a figure can end up somewhere you didn’t ask for.

Important 7.1

The `[htbp]` argument is a *suggestion*, not a command:

- `h` (here),
- `t` (top of page),
- `b` (bottom),
- `p` (its own page).

LaTeX will override your preference if the figure doesn’t fit — this is intentional, not a bug. Adding `!` (e.g. `![htbp]`) tells LaTeX to relax some of its usual placement restrictions, which helps but doesn’t guarantee “exactly here.”

Controlling Where Floats Actually Land

Sometimes tweaking the placement letters isn’t enough and a figure still queues up, drifting further from where you wrote it as body text accumulates — for that there are commands that force the issue outright instead of just suggesting. Placement specifiers only *suggest* where a float should land — sometimes LaTeX still queues one up and drops it several pages later, once enough body text has accumulated to make room.

Nuance 7.2

Three related commands, each doing a different amount of work:

- `\newpage`
 - Starts a new page. Doesn’t touch any floats still waiting to be placed.
- `\clearpage`
 - Starts a new page *and* forces every pending float to flush and place before continuing. This is the actual fix for “my figure floated away from where I put it” — reach for it right before a section that shouldn’t inherit the previous section’s leftover floats.

- `\cleardoublepage`
 - The same as `\clearpage`, but also guarantees the next page is odd-numbered, inserting a blank page if needed. This only matters in two-sided (`twoside`) documents — see Ch. 20's `\frontmatter/\mainmatter/\backmatter`, which call this internally at every transition.

General 7.1 —

```
\section{Methods}
... several figures queued up, none placed yet ...

\clearpage
\section{Results}
% figures from the Methods section are forced to place here,
% before Results starts, instead of drifting further into the document
```

Cross-Check 7.1 — Cross-Check

See `supplementary_materials.md` for a full before/after example — a document with a genuinely stuck float, and the one-line fix.

Vector vs. Raster Images

Once a figure lands where you want it, the next question is what kind of image file to feed `\includegraphics` in the first place — the format you choose decides whether it still looks crisp at print size or starts looking blurry the moment it's scaled up.

Nuance 7.3

- **Vector** (PDF, EPS): scales to any size with no quality loss — ideal for line plots, diagrams, schematics.
- **Raster** (PNG, JPG): fixed resolution — fine for photos or screenshots, but can look blurry if scaled up.

For anything you generate yourself, export as PDF whenever the source tool allows it.

Handling Legacy .eps Figures

Vector is the right call in principle, but not every figure you inherit was exported in a format pdfLaTeX can actually read directly, and `.eps` files are the format most likely to turn up from older plotting tools.

Nuance 7.4

Older figures — especially ones exported from MATLAB or `gnuplot` sometimes only come as `.eps` files. It is a vector format pdfLaTeX can't include directly (only PDF/PNG/JPG).

If you're stuck with one and can't re-export it, `\usepackage{epstopdf}` converts it to PDF automatically at compile time:

General 7.2 —

```
\usepackage{epstopdf}
...
\includegraphics[width=0.6\textwidth]{old_plot.eps}
```

Important 7.2

This needs “shell escape” enabled for your compiler — if the conversion silently fails, that's usually why. The better fix, when possible, is re-exporting the figure as PDF from its source tool in the first place; treat `epstopdf` as a fallback for a figure you can't regenerate, not a habit.

Multi-Panel Figures with subcaption

A single `\includegraphics` call handles one image, but plenty of results are best shown as several related panels side by side under one shared caption — that's a layout plain `figure` can't do alone, and `subcaption` is the package built for it.

Worked Example 7.2

```
\documentclass{article}
\usepackage{subcaption}

\begin{document}
  \begin{figure}[htbp]
    \centering
    \begin{subfigure}{0.45\textwidth}
      \centering
      \includegraphics[width=\linewidth]{example-image-a}
      \caption{First panel}
      \label{fig:sub1}
    \end{subfigure}
    \hfill
    \begin{subfigure}{0.45\textwidth}
      \centering
      \includegraphics[width=\linewidth]{example-image-b}
      \caption{Second panel}
      \label{fig:sub2}
    \end{subfigure}
    \caption{Two panels side by side.}
    \label{fig:combined}
  \end{figure}
```



```
\end{document}
```

Nuance 7.5

Each `subfigure` gets its own caption/label (`fig:sub1`, `fig:sub2`); the outer figure gets an overall caption/label (`fig:combined`) for referring to the pair as a whole.

Nuance 7.6

The `caption` package (load it alongside `subcaption` above, which already depends on it) lets you adjust how every caption in the document looks — font size, and the separator between “Figure 1” and the caption text:

General 7.3 —

```
\usepackage{caption}
...
\captionsetup{font=small, labelsep=colon}
% "Figure 1: A sample figure." instead of the default
% "Figure 1 A sample figure."
```

Set this once in the preamble to apply it document-wide, or scope it locally by wrapping it and the figure together in an extra pair of `{ }` — the same scoping trick Ch. 19 uses for `\renewcommand{\arraystretch}`.

Important 7.3

The `subfigure` is deprecated. `subcaption`, which this chapter uses throughout, is its modern replacement; if you see `\usepackage{subfigure}` in an old template or forum answer, swap it for `subcaption` instead.

Exercise 7.1

1. Build a 2×2 subfigure grid: extend the example above to four subfigures, arranged two per row. To force the third and fourth subfigures onto a new row, insert `\par` (or `\\`) right after the second `\end{subfigure}`, before starting the third `\begin{subfigure}` — then repeat the two-subfigure pattern for the bottom row. Just pressing Enter/leaving a blank line is **not** enough: LaTeX treats a newline as an ordinary space, not a line break, so without `\par`/`\\` your third subfigure will just sit awkwardly next to the second.
2. Deliberately create a stuck float:
 - Add three or four figures in a row with no body text between them, followed by a new `\section`. Recompile and note where they actually land — then add `\clearpage` right before the new section and recompile again to see the difference.

Capstone Update 7

Insert a placeholder figure into your paper (using `example-image` is fine for now) with a proper caption and label — this is what Ch. 8 will cross-reference. If you’ve got more than one figure queued up by the time you assemble the full paper, use `\clearpage` to make sure they land where you intend rather than drifting into the next section.

Chapter 8 — Cross-Referencing, Labels & Hyperlinks

Every figure, table, and section you’ve labeled so far (`fig:sample`, `tab:sample`, and so on, from Chapters 6–7) has been sitting unused — a `\label` on its own does nothing until something else points back to it with `\ref`. Manually typing “see Figure 3” is exactly the kind of thing that goes stale the moment you insert a new figure earlier in the document and every number after it shifts by one; automatic cross-referencing exists specifically so that renumbering is the compiler’s problem, not yours.

This chapter covers `\ref`/`\cref` for that automatic numbering (including why `cleveref`’s load order relative to `hyperref` matters), the labeling-prefix convention (`fig:`, `tab:`, `sec:`, ...) used consistently throughout the rest of the guide, and `\hypersetup` for turning `hyperref`’s default loud link-boxes into the quiet colored-text links expected in a submitted PDF.

Objective 8

Reference figures, tables, and sections by number automatically (so they never go stale), and set up hyperlinks that look professional instead of like a default web browser’s blue-boxed mess.

Automatic Numbering with `\ref` and `\cref`

A `\label` is inert on its own. It only becomes useful once something points back to it by number, and that’s what the reference commands are for. `cleveref`’s `\cref`/`\Cref` build on plain `\ref` by also supplying the right noun (“Figure”, “Table”, “Section”) automatically, so renumbering — or retyping the wrong word — is never something you do by hand.

Worked Example 8.1

```
\documentclass{article}
\usepackage{hyperref}
\usepackage{cleveref}

\begin{document}
  As shown in \cref{fig:sample} and \cref{tab:sample}.
  The results support the hypothesis.
```

See `\Cref{tab:sample}` at the start of a sentence (capitalized).
`\end{document}`

Important 8.1

- `\ref{fig:sample}` gives you just the number,
- `\cref{fig:sample}` (from `cleveref`) gives you the number *and* automatically prepends “Figure”/“Table”/“Section” as appropriate — one less thing to hand-type and get out of sync.
- `\Cref` capitalizes it for sentence starts.

Important 8.2

Load `hyperref` near the very end of your preamble, and load `cleveref` immediately *after* `hyperref`. Getting this backwards (loading `cleveref` before `hyperref`) is a classic source of broken or missing links — package load order matters more in LaTeX than in most other languages you may have used.

Styling Hyperlinks with `\hypersetup`

`hyperref` makes every cross-reference, citation, and URL clickable, but its out-of-the-box styling — thick colored boxes around every single link — reads like a rough draft rather than a submission-ready PDF. A single `\hypersetup` call swaps that for quiet, professional-looking links instead.

Nuance 8.1

By default, `hyperref` draws loud colored boxes around every link and citation — fine on screen, unprofessional in a printed or submitted PDF. Set this up properly instead:

Important 8.3

```
\hypersetup{
  colorlinks=true,
  linkcolor=blue,
  citecolor=blue,
  filecolor=blue,
  urlcolor=blue,
}
```

`colorlinks=true` colors the link *text* rather than boxing it. For a fully print-ready PDF with no visible link styling at all, use `hidelinks` instead.

General 8.1 — Forward Note

`\eqref` for equation cross-references is coming in Ch. 9 once equations are introduced — the same `\label/\ref` mechanics apply.

The Labeling-Prefix Convention

Naming every `\label` consistently can look like a cosmetic habit, but in a document with hundreds of labels spread across a dozen chapters, it’s what keeps a reference like `\label{intro}` from meaning three different things and colliding with itself.

Nuance 8.2

Every `\label` in this guide so far has used a prefix — `fig:`, `tab:`, and so on — without ever naming the convention directly. Worth making explicit now that cross-referencing is the whole point of this chapter:

Important 8.4

Prefix	For
<code>fig:</code>	Figures
<code>tab:</code>	Tables
<code>eq:</code>	Equations (Ch. 9)
<code>sec:/ch:</code>	Sections/chapters
<code>thm:</code>	Theorems (Ch. 11)
<code>alg:</code>	Algorithms (Ch. 12)
<code>def:</code>	Definitions (Ch. 11)

General 8.2 — What the prefix actually does/doesn’t do

`cleveref` does *not* read the prefix text to decide whether to print “Figure” or “Table.” It determines that from the *counter* that was stepped when `\label` was called. A `\caption` inside a `figure` steps the `figure` counter, a `\section` steps the `section` counter, and so on, via `\refstepcounter` working behind the scenes.

Important 8.5

The prefix is a discipline **for you, the author**. It prevents name collisions (`\label{intro}` could be a section, a figure, or a theorem in a 23-chapter document) and makes every `\ref`/`\cref` call in your source instantly scannable, without needing to jump to the label to know what it points to.

Exercise 8.1

1. Take a document with a labeled figure and table where one `\label` and its matching `\ref`/`\cref` have been deliberately mistyped (e.g. `\label{fig:sampl}` vs `\cref{fig:sample}`). Recompile, note the ?? that appears in place of the number, and fix the mismatch.
2. Add the `\hypersetup` block above and recompile. Compare the look of your links before and after.

3. Bonus: audit the labels in your capstone paper so far. Confirm every one uses a consistent prefix (`fig:`, `tab:`, `sec:`, etc.), and fix any that don't.

Capstone Update 8

Cross-reference the result table (Ch. 6) and figure (Ch. 7) from your paper's body text using `\cref`, and apply the `colorlinks` setup so your capstone PDF looks presentation-ready rather than default-browser-blue.

Part 3: Math Mechanics (Chapters 9–10)

--

Chapter 9 — Inline & display math basics/fonts & units

Math is where LaTeX earns its reputation — an equation that would be a fight with an equation editor elsewhere is a few lines of markup here, and it renders consistently every time instead of drifting out of alignment across a long document.

This chapter covers the basics everything else in Part 3 builds on: inline vs. display math, Greek letters and sub/superscripts, and the font/unit conventions (roman vs. italic, `siunitx`, `bm`) that separate a professionally typeset paper from an obviously beginner one.

Objective 9

Typeset inline and display equations correctly, use Greek letters and sub/superscripts, and get the font and unit conventions right from the start — the habits that separate a professionally typeset paper from an obviously beginner one.

Inline vs. display math

The same equation can sit inline within a sentence, or be set apart on its own numbered line — LaTeX uses different syntax for each, and mixing them up is an easy first mistake.

Worked Example 9.1

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
  Einstein's mass-energy equivalence, written inline, is  $E = mc^2$ .
  The same relationship, set apart and automatically numbered:

  \begin{equation}
    E = mc^2
    \label{eq:mass-energy}
  \end{equation}
```

Two related equations, aligned at the equals sign and each numbered:

```
\begin{align}
  F &= ma & \label{eq:newton2} \\
  W &= Fd & \label{eq:work}
\end{align}
\end{document}
```

Important 9.1

- `$...$` is inline math — it flows within a sentence.
- `equation` is display math with automatic numbering. It can be references with `\eqref{eq:mass-energy}` or `\cref{eq:mass-energy}`. The same `\label`/reference mechanics from Ch. 8, just applied to an equation.
- `\eqref` wraps the number in parentheses ((1)), which is the standard convention for equation references; `\cref` instead produces “Equation (1)” with the word spelled out. Either is correct, just pick one convention and stay consistent.
- Need an equation with *no* number at all? `equation*` is the starred, unnumbered variant. See Ch. 5’s “star convention” nuance if that’s new to you.
- `align` (from `amsmath`, loaded above) numbers *every* line and aligns them at the `&` marker. The `&` sign is typically placed right before the `=` sign.

Greek letters and sub/superscripts

Variable names and exponents in physics and math notation lean heavily on Greek letters and sub/superscripts, both typed with plain-text shortcuts rather than any special input method.

Worked Example 9.2

```
\documentclass{article}

\begin{document}
  The decay follows  $N(t) = N_0 e^{-\lambda t}$ , where  $\lambda$  is the
  decay constant.
  Summing over  $i = 1, \dots, n$ :
  \[
    S = \sum_{i=1}^n x_i^2
  \]
\end{document}
```

Important 9.2

- Greek letters are typed by name:
 - `\alpha`, `\beta`, `\gamma`, `\theta`, `\pi`, `\sigma`, `\omega` for lowercase,
 - `\Gamma`, `\Delta`, `\Theta`, `\Pi`, `\Sigma`, `\Omega` (capitalized command) for the uppercase forms that differ from Latin letters.

- Subscripts use `_`, superscripts use `^`; wrap multi-character sub/superscripts in braces (`x_{i,j}^2`), since `x_i,j^2` would only attach `i` to the subscript.

Italics vs. Roman in Math

Math mode italicizes every letter by default, which is correct for a variable but wrong for a handful of other things that just happen to also be letters.

Nuance 9.1

Letters in math mode render in italic by default, which is correct for single-letter variables (x , y , α), but wrong for a few other things:

- **Vectors/matrices** should be bold, not italic; see the `\bm` nuance below.
- **Units** should be upright/roman, not italic; see `\siunitx` below.
- **Plain English words** inside math mode (like “if” in a `cases` block, covered in Ch. 10) need `\text{}`, or LaTeX renders them as multiplied variables.

Worked Example 9.3

```
\documentclass{article}

\begin{document}
  % correct: x is italic automatically, no extra command needed
  $x^2 + y^2 = r^2$

  % correct: \mathrm{} for upright text/units inside an equation
  $F = 5\ \mathrm{N}$

  % WRONG: don't reach for \textit inside math mode
  $\textit{x}^2$
\end{document}
```

Important 9.3

Never use `\textit{}` inside math mode to italicize a variable. `\textit{}` is a *text-mode* command; using it in math mode produces inconsistent spacing and kerning compared to LaTeX’s own math italics. Just write $\mathbf{x}$ — it’s already italic automatically.

Formatting Units

A number with a unit can be typed by hand correctly, but staying consistent about it across a 40-page document with hundreds of units is the part that actually goes wrong.

Nuance 9.2

You *can* write units manually, and it's not wrong:

Worked Example 9.4

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
  % manual approach - technically correct
  $10\,\,\text{keV}$
  % or, equivalently
  $10\,\,\mathrm{keV}$
\end{document}
```

Both use tools you already have: `\,` for the thin space between number and unit, `\text{}`/`\mathrm{}` to keep the unit upright instead of italic. Nothing here is broken.

siunitx: The solution

The catch is consistency, not correctness. Across a 40-page thesis with hundreds of units, it's easy to forget the `\,` on line 200 when you remembered it on line 20, mix `\text{}` and `\mathrm{}` inconsistently, or format `\times 10^{-3}` one way in one equation and another way elsewhere. None of these individually breaks compilation, they just make the document look inconsistent, which readers (and reviewers) notice even if they can't say why.

Nuance 9.3

The `siunitx` package's `\SI{value}{unit}` command removes that risk entirely by enforcing the same spacing and formatting every time, automatically:

Worked Example 9.5

```
\documentclass{article}
\usepackage{siunitx}

\begin{document}
  The measured velocity was \SI{5}{\meter\per\second}.
  The sample mass was \SI{10.2}{\kilogram}.
  A unit can also be shown on its own, without a value, using
  \si{\meter\per\second}.
\end{document}
```

`\SI{}{}` inserts the correct thin space between number and unit and renders the unit upright. Manual `\,\,\text{}` is fine for a one-off; `siunitx` is what you want the moment a document has more than a handful of units to stay consistent across.

Bold Vectors & Greek Letters with `bm`

LaTeX's standard bold command quietly fails on Greek letters, which is a surprising first encounter for anyone typesetting a bold coefficient vector because the default LaTeX font doesn't cover Greek symbols.

Nuance 9.4

For bold vectors or bold Greek symbols (common in statistics/ML notation — a coefficient vector β , for instance), use the `bm` package:

Worked Example 9.6

```
\documentclass{article}
\usepackage{bm}

\begin{document}
  The regression coefficients  $\bm{\beta}$  minimize the loss function.
  A bold Latin vector works the same way:  $\bm{v}$ .
\end{document}
```

`\boldsymbol{}` (from `amsmath`) is a lighter alternative that covers most Greek letters too, but `bm` handles more symbol combinations, so it's the one used throughout this guide.

Putting It Together

A single short derivation that combines display equations, `\bm{}`, and `\SI{...}{...}` in one place is worth more than seeing each of them in isolation.

Worked Example 9.7

```
\documentclass{article}
\usepackage{amsmath}
\usepackage{siunitx}
\usepackage{bm}

\begin{document}
  Newton's second law relates force and acceleration:
  \begin{equation}
    \mathbf{F} = m\bm{a}
    \label{eq:newton2}
  \end{equation}
  where  $m$  is mass in \si{kilogram} and  $\bm{a}$  is acceleration in
  \si{meter\per\second\squared}.

  A measured force of \SI{12.5}{\newton} acting on a \SI{2}{kilogram}
  mass gives an acceleration of \SI{6.25}{\meter\per\second\squared}.
\end{document}
```

Exercise 9.1

- Transcribe these into LaTeX (use `align` for anything spanning more than one line):
 - $a^2 + b^2 = c^2$
 - $\int_0^\infty e^{-x} dx = 1$ (*we'll fix this integral's spacing properly in Ch. 10*)
 - $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$
 - A vector dot product: $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$
 - A bold Greek coefficient: $\hat{y} = \boldsymbol{\beta}^T \mathbf{x}$
- Find a professionally typeset equation from a textbook or paper in your field (a screenshot is fine), and write the LaTeX that would reproduce it. Check your version against the rules above: is anything that should be bold left plain? Any unit set in italics that shouldn't be?

Cross-Check 9.1 — Cross-Check

Sample target equations are collected in `supplementary_materials.md` — see “Ch. 9 — Reverse-Engineering Sample Targets.”

Capstone Update 9

Typeset the single most important equation in your capstone paper — the one that defines your key result or model — using correct roman/bold conventions and `\SI{}{}{}` for any units involved.

Chapter 10 — Advanced Math: Matrices, Multi-line Equations, Cases

Chapter 9 covered single equations; real derivations are rarely just one line — they’re matrices, branching definitions, and multi-step algebra that each need their own environment to typeset cleanly.

This chapter covers `pmatrix`/`bmatrix`/`vmatrix` for matrices, `cases` for piecewise functions, `align`/`split` for multi-step derivations, and the delimiter-sizing and spacing details that make a finished derivation look deliberately typeset rather than dumped from a text editor.

Objective 10

Typeset matrices, piecewise functions, and multi-step derivations, and control delimiter sizing and spacing so equations look deliberate rather than cramped or oversized.

Matrices

A matrix is just a grid inside brackets, and the bracket style is a one-word swap once you know the environment name to change.

Worked Example 10.1

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}
  \[
    A = \begin{pmatrix}
      1 & 2 \\
      3 & 4
    \end{pmatrix}
  \]
\end{document}
```

Nuance 10.1

`pmatrix` gives parentheses; swap the environment name for a different bracket style

- `bmatrix` for square brackets,
- `vmatrix` for a determinant (single vertical bars),
- `Vmatrix` for double vertical bars.

The `&` (column separator)/`\\` (row separator) syntax stays the same across all of them.

Piecewise Functions with cases

A function defined differently across a few conditions needs its branches aligned under one shared brace, which is exactly what the `cases` environment is for.

General 10.1 — Usecase of ‘cases’

```
\[
  f(x) =
  \begin{cases}
    x^2 & \text{if } x \geq 0 \\
    -x^2 & \text{if } x < 0
  \end{cases}
\]
```

Nuance 10.2

Note the `\text{}` around “if”. Without it, “if” renders in italic math font as though it were two multiplied variables (*i* times *f*) and it’ll look wrong and read worse. Any plain-English word inside math mode needs `\text{}` around it.

Multi-Step Derivations

A derivation that unfolds over several lines needs every line aligned at the same operator, and a choice about whether each intermediate step gets its own equation number.

Worked Example 10.2

```
\documentclass{article}

\begin{document}
  \begin{align}
    (a+b)^2 &= (a+b)(a+b) && \\
            &= a^2 + ab + ba + b^2 && \\
            &= a^2 + 2ab + b^2
  \end{align}
\end{document}
```


Important 10.1

`align` numbers every line by default. If you only want the *final* line numbered, wrap a `split` block inside a single `equation` instead:

Worked Example 10.3

```
\begin{equation}
  \begin{split}
    (a+b)^2 &= (a+b)(a+b) \\
    &= a^2 + ab + ba + b^2 \\
    &= a^2 + 2ab + b^2
  \end{split}
\end{equation}
```

Use `align*` (with an asterisk) if you want *no* line numbered at all — the same starred-variant pattern as `\section*{}` from Ch. 5, applied here to equations.

Controlling Fraction Size

A fraction’s default size depends on whether it’s inline or in a display equation, and three commands exist for overriding that default in either direction.

Nuance 10.3**`\tfrac`, `\dfrac`, and `\displaystyle`**

A fraction’s size normally depends on context: `\frac{1}{2}` written inline renders small and compact to fit the line; the same `\frac{1}{2}` inside a display equation (`\[...\]`, `equation`, `align`) renders larger, in what’s called “display style.” Three commands (all from `amsmath`) let you override that default in either direction:

- `\displaystyle` forces display-style sizing even inline — `\displaystyle\frac{1}{2}` renders full-size in the middle of a sentence. Useful occasionally, but can disrupt line spacing if overused.
- `\dfrac{}{}` is shorthand for exactly that, applied just to one fraction: `\dfrac{1}{2}` always renders at display size, wherever it’s used.
- `\tfrac{}{}` is the opposite shorthand: always renders at the smaller, inline (“text style”) size, even inside a display equation — handy for a small fraction that would otherwise look oversized as part of a larger expression.

Delimiter Sizing

`\left/\right` auto-sizing is convenient but not always the right call — a simple fraction wrapped in it often ends up more oversized than the expression actually needs.

Nuance 10.4**Size Matters: delimiter sizing**

`\left(` (and `\right)`) auto-size to match their contents — convenient, but easy to overdo. Wrapping a simple fraction in `\left( \right)` often produces oversized, over-spaced parentheses:

% often too large / spaced-out for something this simple

```
$\left( \frac{1}{2} \right)$
```

% better: a manual, smaller, tighter size

```
$\bigl( \tfrac{1}{2} \bigr)$
```

The manual sizing commands, from smallest to largest, are `\bigr/\bigr`, `\Bigl/\Bigr`, `\biggl/\biggr`, `\Biggl/\Biggr` (each pair is the left/right delimiter respectively). Pick whichever size actually matches your content instead of defaulting to `\left/\right` every time. Note the deliberate use of `\tfrac` rather than `\frac` here too — per the nuance above, it keeps the fraction at its natural compact size instead of forcing display-style sizing into an already-tight expression.

One-Sided Delimiters

Showing a function evaluated between two bounds needs a tall bar on the right and deliberately nothing on the left to match it.

Nuance 10.5**one-sided delimiters: `\left. ... \right|`**

Evaluating a function or antiderivative between two bounds needs a vertical bar sized to match the expression’s height, but nothing on the left side to match it:

```
\[
  \int_0^1 x \, dx = \left. \frac{x^2}{2} \right|_0^1 = \frac{1}{2}
\]
```

`\left.` (a left delimiter followed immediately by a period) tells LaTeX “size the right delimiter to match, but draw nothing on the left.” `\right|` then draws a properly sized vertical bar, with the bounds as its subscript/superscript (`_0^1`). This is the standard way to show a function evaluated at bounds — you’ll use it constantly whenever a derivation is followed by “evaluated from a to b .”

Math Spacing

LaTeX doesn’t insert any breathing room around math operators or differentials on its own — everything you see spaced out in a polished equation was spaced out on purpose.

Nuance 10.6**Math spacing: `\,` and `\!`**

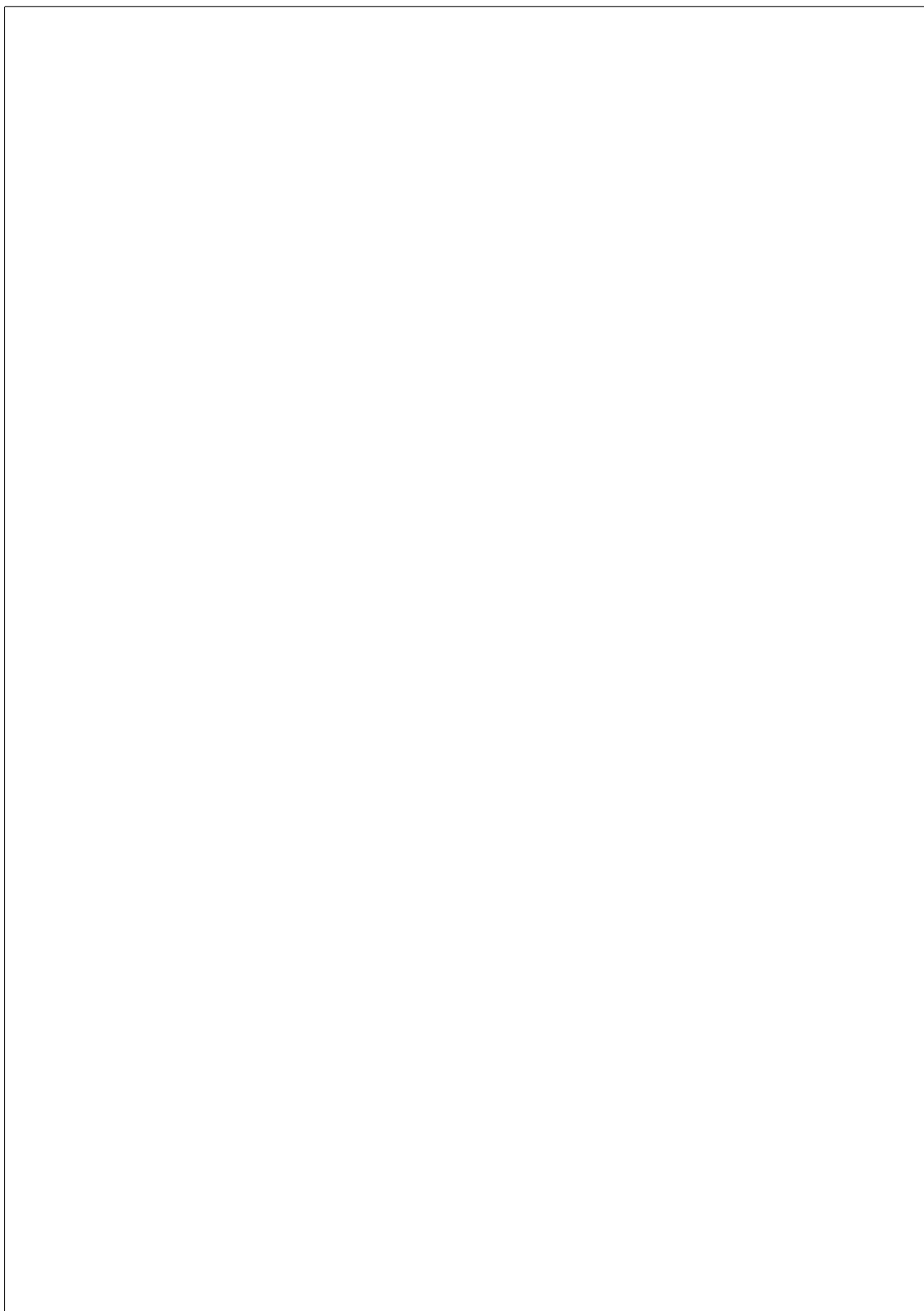
Differentials in integrals look cramped without a small gap before them:

`\,` inserts a thin space; `\!` inserts a *negative* thin space (pulls two elements closer together) — useful for tightening nested exponents or an overly loose double integral. These are small, cosmetic details, but exactly the kind that make a derivation look deliberately typeset rather than dumped from a text editor.

1. Typeset the coefficient matrix and right-hand-side vector of a 2-equation linear system as a single matrix equation, using `\pmatrix`.
2. Typeset a 3-branch piecewise function with `\cases`, remembering `\text{}` around any English words.
3. Take one `\left( \right)` expression (your own, or from Ch. 9's exercises) and rewrite it using `\bigl/ \bigr` instead — compare the rendered size.
4. Revisit the integral from Ch. 9's exercise 1 ($\int_0^\infty e^{-x} dx = 1$) and add the thin space: $\int_0^\infty e^{-x} dx = 1$.
5. Typeset $\int_0^2 x^2 dx = \frac{x^3}{3} \Big|_0^2 = \frac{8}{3}$ using `\left. / \right|`.

Add a real derivation relevant to your capstone topic (2–4 steps is plenty) using `align` or `split`, and tighten the spacing around any integrals or products with `\,`.

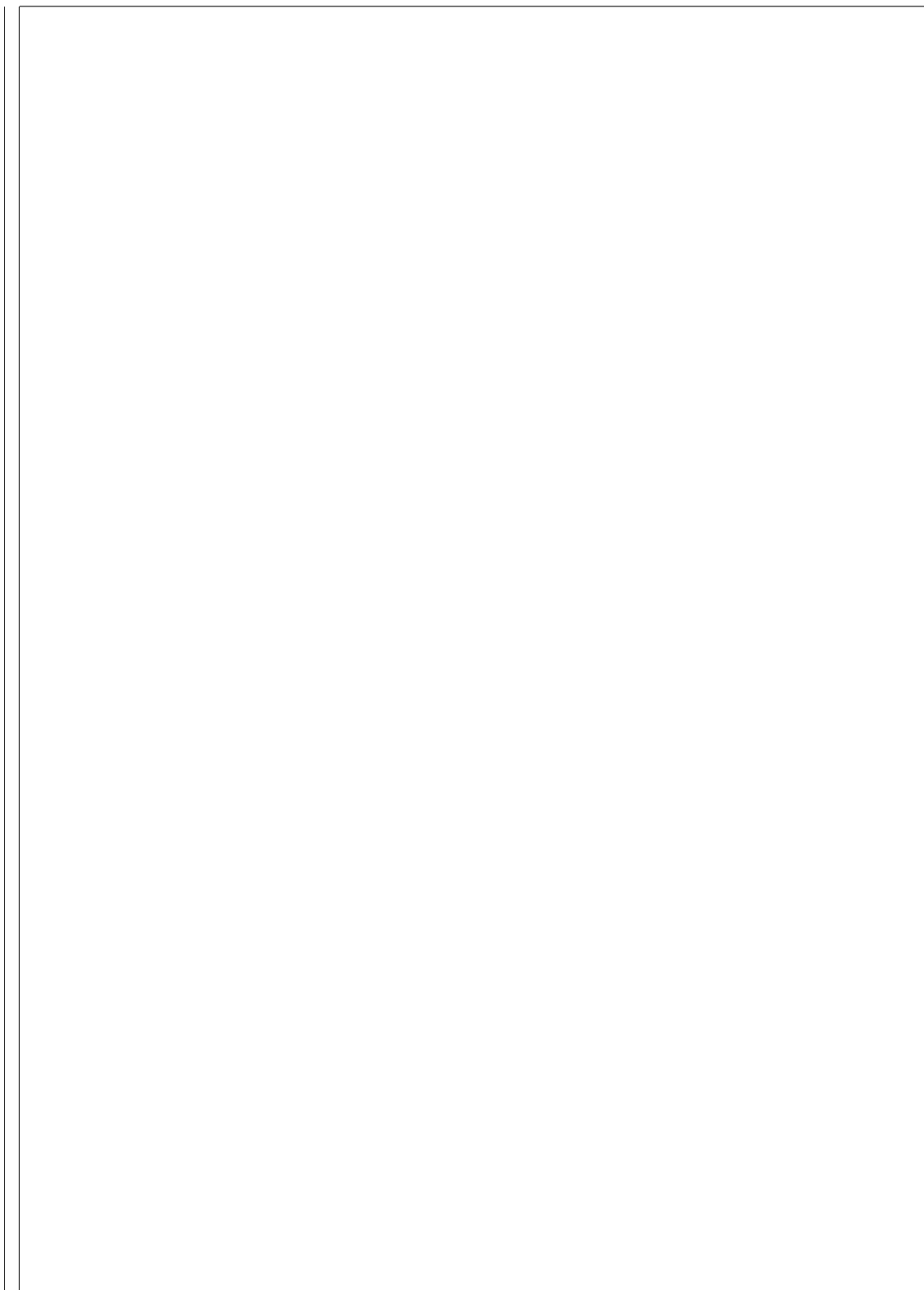
End of Part 3 (Ch. 9–10). Next: Part 4 — Math Structures & Algorithms (Ch. 11–12).



Part 4: Math Structures & Algorithms (Chapters 11–12)

--

Where Part 3 was about everyday math notation, this part is about *structured blocks*: formally stated theorems/proofs, and algorithm pseudocode.



Chapter 11 — Theorems, Definitions, Proofs (amsthm)

Objective 11

Define reusable, numbered environments for theorems, lemmas, and definitions, and typeset proofs with the standard end-of-proof marker.

Worked Example 11.1

defining and using a theorem

```
\documentclass{article}
\usepackage{amsthm}
\usepackage{amsmath}

\newtheorem{theorem}{Theorem}

\begin{document}

\begin{theorem}
\label{thm:pythagoras}
For a right triangle with legs  $a$ ,  $b$  and hypotenuse  $c$ ,
 $a^2 + b^2 = c^2$ .
\end{theorem}

\begin{proof}
Consider a square of side  $a+b$  built from four copies of the
triangle arranged around a central square of side  $c$ . Comparing
areas gives  $(a+b)^2 = c^2 + 4 \left(\frac{1}{2}ab\right)$ , which
simplifies to  $a^2 + b^2 = c^2$ .
\end{proof}

\end{document}
```

`\newtheorem{theorem}{Theorem}` creates the `theorem` environment and tells LaTeX to label instances “Theorem 1,” “Theorem 2,” and so on. The `proof` environment comes built into `amsthm` — no `\newtheorem` needed for it — and automatically appends the

end-of-proof symbol (■) when you close it.

Nuance 11.1

two brackets, two very different meanings

`\newtheorem` takes an optional bracketed argument, but its meaning depends entirely on *where* the bracket goes — a genuinely easy pair to mix up. Here are both patterns as complete, copy-adaptable preambles:

% Option A: shared counter (one running number across all theorem types)

```
\newtheorem{theorem}{Theorem}
```

```
\newtheorem{lemma}[theorem]{Lemma}
```

```
\newtheorem{corollary}[theorem]{Corollary}
```

% Result: Theorem 1, Lemma 2, Corollary 3, Theorem 4, ...

% Option B: per-section numbering (resets every time a new section starts)

```
\newtheorem{theorem}{Theorem}[section]
```

```
\newtheorem{lemma}[theorem]{Lemma}
```

% still shares theorem's counter

```
\newtheorem{definition}{Definition}[section] % its own counter, also per-section
```

% Result (inside Section 1): Theorem 1.1, Lemma 1.2, Definition 1.1

% Result (inside Section 2): Theorem 2.1, ...

`[theorem]` placed right after `\newtheorem{lemma}` means “share *this* counter with the theorem environment already defined.” `[section]` placed at the very end means “reset my own counter every time a new section starts, and prefix it with the section number.”

Same syntax shape, opposite purpose — read carefully before copying either pattern. Note in Option B that `lemma` can share `theorem`’s per-section counter *and* inherit its section-prefixed numbering automatically — you don’t need to add `[section]` to `lemma` itself.

Nuance 11.2

theorem styles (`\theoremstyle`)

`amsthm` ships three built-in visual styles, declared with `\theoremstyle{...}` *before* the `\newtheorem` calls it should apply to:

```
\usepackage{amsthm}
```

% plain: bold heading, italic body - the default, for theorems/lemmas

```
\theoremstyle{plain}
```

```
\newtheorem{theorem}{Theorem}
```

```
\newtheorem{lemma}[theorem]{Lemma}
```

% definition: bold heading, upright body - for definitions/examples

```
\theoremstyle{definition}
```

```
\newtheorem{definition}{Definition}
```

```
\newtheorem{example}{Example}
```

% remark: italic heading, upright body - for remarks/notes

```
\theoremstyle{remark}
```

```
\newtheorem{remark}{Remark}
```

The style you pick is a visual signal to the reader: `plain` says “this is a formal claim that gets proved,” `definition` says “this is stating what a term means,” `remark` says “this is context, not something requiring proof.” Everything after a `\theoremstyle{...}` call uses that style until you declare a different one — that’s why the three blocks above are ordered the way they are.

Worked Example 11.2

definitions

```
\newtheorem{definition}{Definition}
```

```
...
```

```
\begin{definition}
```

```
\label{def:convergence}
```

A sequence (x_n) **converges** to L if for every

$\varepsilon > 0$ there exists N such that $|x_n - L| < \varepsilon$ for all $n > N$.

```
\end{definition}
```

`definition` environments work exactly like `theorem` — they’re just conventionally styled differently, per the `\theoremstyle` nuance above, since they’re stating a term rather than a provable claim.

Nuance 11.3

\qedhere for proofs ending in a displayed equation

The `proof` environment’s automatic ■ symbol normally lands at the end of the last line. But if a proof ends with a displayed equation (`equation`, `align`, `\[...\]`), the ■ can get pushed down to its own line by itself, floating awkwardly. Fix it by adding `\qedhere` right inside that final display:

```
\begin{proof}
```

Expanding and simplifying,

```
\[
  (a+b)^2 = a^2 + 2ab + b^2. \qedhere
\]
```

```
\end{proof}
```

`\qedhere` tells `amsthm` to place the ■ symbol on this line instead of pushing a new one below.

Exercise 11.1

1. Define a `lemma` environment sharing the `theorem` counter, using the `[theorem]` syntax above.
2. State and prove a small lemma of your choosing (even a simple one, e.g. “the sum of two even integers is even”) using your new environment and the built-in `proof` environment.
3. Cross-reference your lemma from a sentence elsewhere in the document using `\cref` (from Ch. 8) — confirm the number matches.

Don't have a lemma in mind? `supplementary_materials.md` has a short bank of simple, classic theorems (“Ch. 11 — Simple Theorem Bank for Beginners”) to pick from instead of inventing one from scratch. If you'd like to check your work against a solved reference afterward (not before — try it yourself first), exercise 2's exact suggested example is fully solved there too (“Ch. 11 — Exercise 2, Solved”).

Capstone Update 11

State one assumption or definition from your capstone topic formally, using a `definition` environment you define with `\newtheorem`.

Chapter 12 — Algorithms & Pseudocode

Objective 12

Typeset a method or procedure as numbered, captioned pseudocode that can be referenced from your text like any other float.

Worked Example 12.1

```
algorithm2e
\usepackage[ruled,vlined]{algorithm2e}

\begin{algorithm}[htbp]
\caption{Gradient Descent}
\label{alg:gd}
\KwIn{learning rate  $\eta$ , initial point  $x_0$ , tolerance  $\epsilon$ }
\KwOut{approximate minimizer  $x^*$ }
 $x \leftarrow x_0$ ;
\While{ $\|\nabla f(x)\| > \epsilon$ }{
   $x \leftarrow x - \eta \nabla f(x)$ ;
}
\Return{ $x$ }
\end{algorithm}
```

$\backslash\text{KwIn}\{\}/\backslash\text{KwOut}\{\}$ label inputs and outputs, $\backslash\text{While}\{\text{condition}\}\{\text{body}\}$ handles loops, and $\backslash;$ marks the end of a pseudocode line (like a semicolon ending a line of real code). The outer `algorithm` environment behaves like the floats from Ch. 7 — `[htbp]` placement rules and captions work exactly the same way.

Reference: common `algorithm2e` commands

Command	Purpose
$\backslash\text{KwIn}\{\dots\}$	Input declaration
$\backslash\text{KwOut}\{\dots\}$	Output declaration
$\backslash\text{While}\{\text{cond}\}\{\text{body}\}$	While loop

Command	Purpose
<code>\For{cond}{body}</code>	For loop
<code>\If{cond}{body}</code>	Conditional branch
<code>\Else{body}</code>	Else branch
<code>\ElseIf{cond}{body}</code>	Else-if branch
<code>\Return{value}</code>	Return statement
<code>\;</code>	End of pseudocode line (see nuance below)

The worked example above only exercises three of these (`\KwIn`, `\KwOut`, `\While`). Rather than manufacture an example for every remaining row, `supplementary_materials.md` (“Ch. 12 — Full Working Examples”) has two short algorithms that between them put almost everything else in this table to use — `\For`, `\If`/`\ElseIf`/`\Else`, `\Return`. Anything left over is worth looking up directly in the package docs at this point.

Nuance 12.1

the `\;` line terminator

Every line of pseudocode in `algorithm2e` should end with `\;`. This is the most common `algorithm2e` stumble: forget it, and LaTeX doesn’t error out — it just runs the next statement onto the same visual line, so your pseudocode reads like a run-on sentence instead of a numbered procedure. If your algorithm compiles cleanly but looks visually wrong (lines mashed together that should be separate), a missing `\;` is the first thing to check.

Nuance 12.2

what `ruled` and `vlined` actually do

The worked example above loads `algorithm2e` as `\usepackage[ruled,vlined]{algorithm2e}` — two commonly used, optional formatting flags:

- **`ruled`** draws a horizontal rule at the top and bottom of the algorithm block, similar to how a table caption sits above `\toprule`/`\bottomrule` in Ch. 6.
- **`vlined`** draws vertical lines connecting the start and end of each loop/conditional block, making nested control flow easier to scan visually.

Both are optional — plain `\usepackage{algorithm2e}` works fine — but most published pseudocode uses at least `ruled`.

Nuance 12.3

don’t load both algorithm packages at once

Two competing package families exist for this: `algorithm` + `algorithmic` (older, minimal) and `algorithm2e` (newer, more batteries-included, different command syntax). They define overlapping commands, so loading both in the same preamble produces confusing “already defined” errors. Pick one per document. This guide uses `algorithm2e` throughout, since its `\KwIn`/`\KwOut`/`\While`/`\For` syntax reads closer to plain English for a beginner — but you may encounter journal templates that specifically expect the other pair.

Worked Example 12.2

the algorithm + algorithmic alternative (for reference)

If a target journal's template specifically requires the older combination instead:

```
\usepackage{algorithm}
\usepackage{algorithmic}

\begin{algorithm}
\caption{Gradient Descent}
\label{alg:gd-alt}
\begin{algorithmic}
\STATE  $x \leftarrow x_0$ 
\WHILE{ $|\nabla f(x)| > \epsilon$ }
  \STATE  $x \leftarrow x - \eta \nabla f(x)$ 
\ENDWHILE
\RETURN  $x$ 
\end{algorithmic}
\end{algorithm}
```

Same result, different syntax (`\STATE`, `\WHILE`/`\ENDWHILE` instead of `algorithm2e`'s more concise brace blocks). Whichever pair you use, don't mix the two.

Nuance 12.4

algorithm2e and hyperref can clash — follow the Golden Load Order

`algorithm2e` has a documented history of caption/numbering quirks when combined with `hyperref` (loaded back in Ch. 8). If your algorithm's caption number looks wrong or its cross-reference doesn't match, double check you're following Ch. 8's Golden Load Order: load `algorithm2e` (like most content packages) *before* `hyperref`, with `hyperref` and then `cleveref` loaded last. This is the same rule already established in Ch. 8, just worth re-confirming here since it's the specific pairing that surfaces the problem most often.

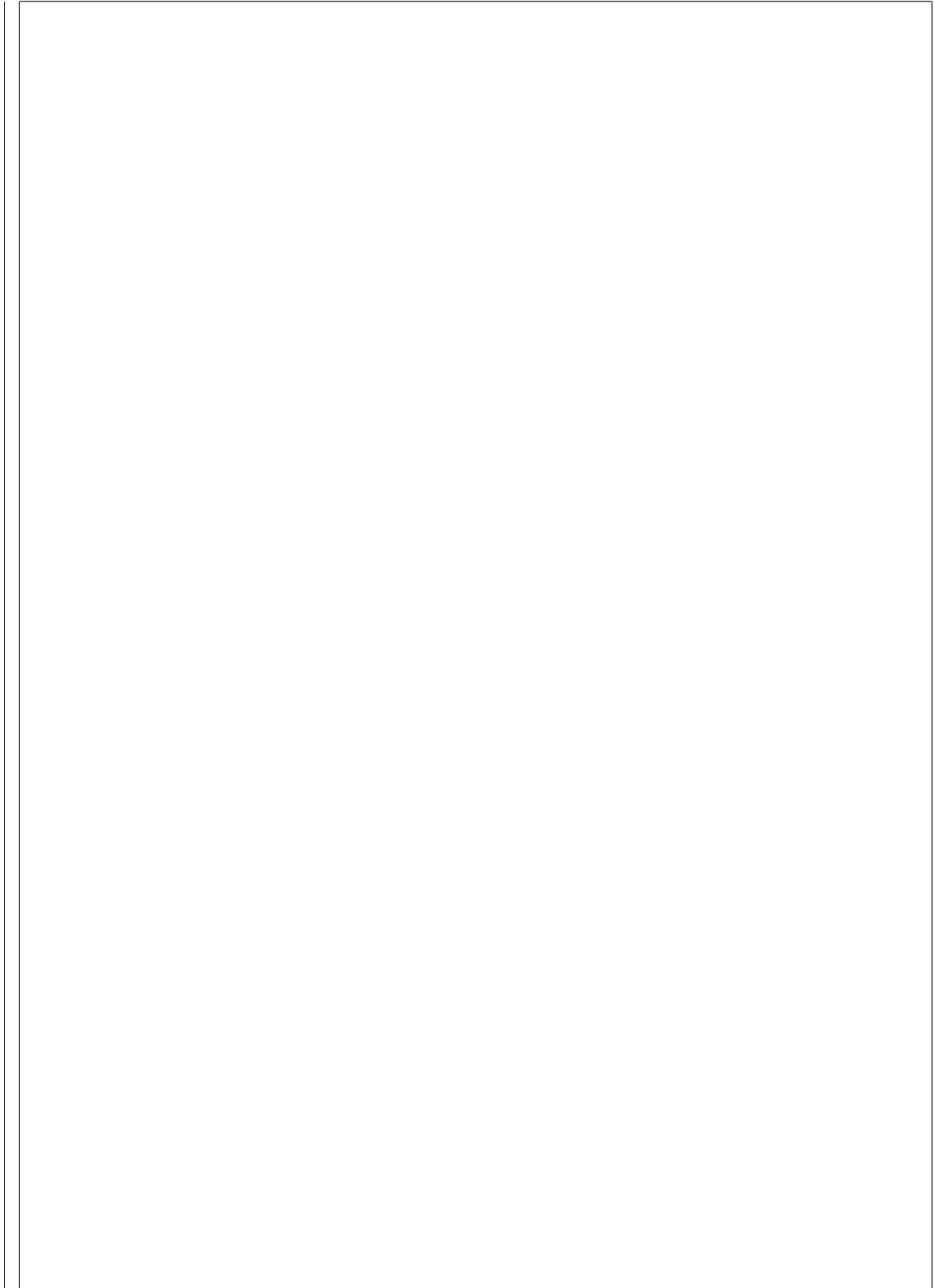
Exercise 12.1

Typeset pseudocode for a method relevant to your own field (a simplified version is fine) using `algorithm2e`. Include at least one input/output declaration, one loop (`\While` or `\For`), and one conditional (`\If`/`\Else`).

Capstone Update 12

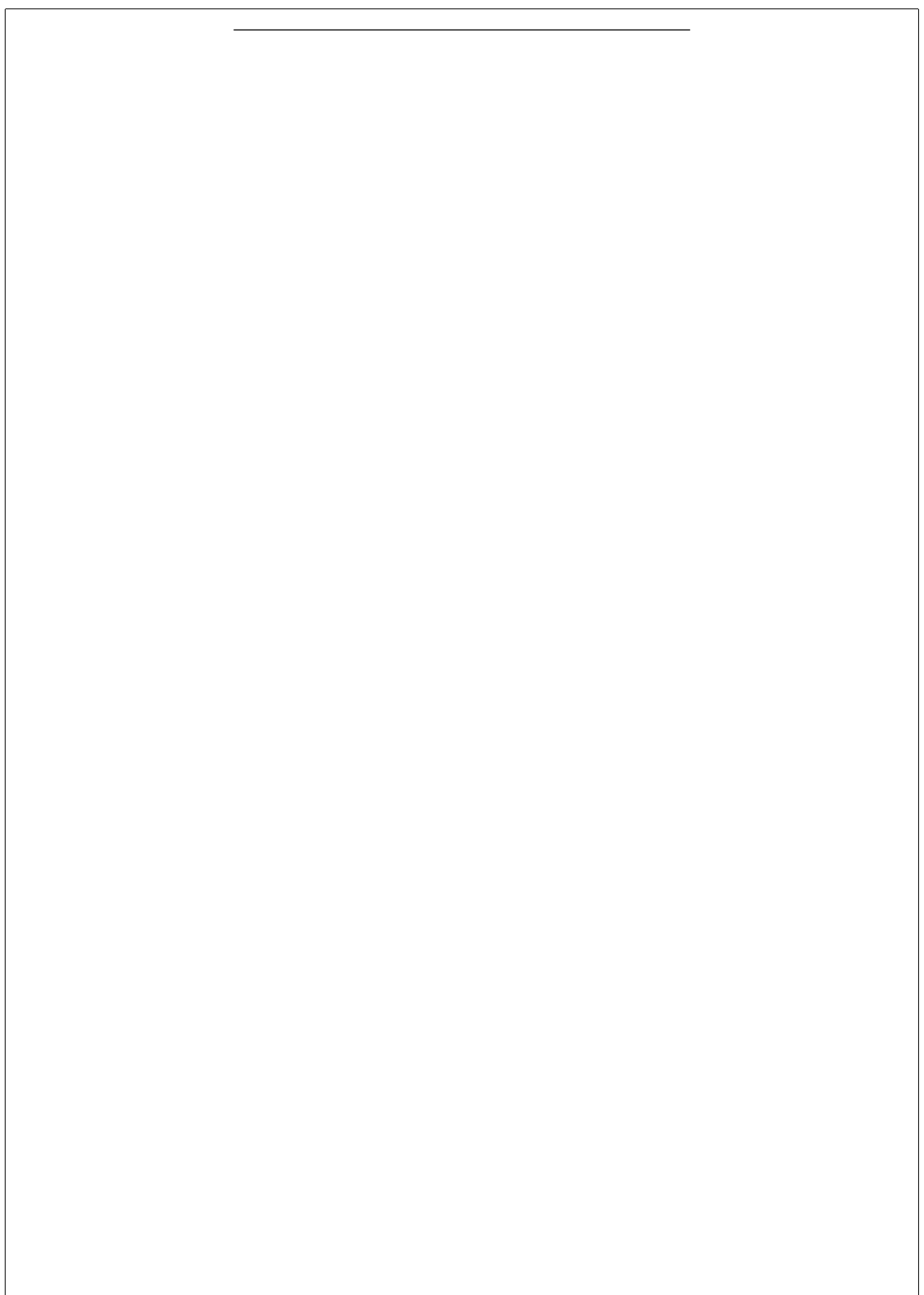
Add a methods algorithm block to your capstone paper describing the core procedure behind your topic (an experimental protocol, a numerical method, a computational pipeline — whatever fits), captioned and labeled so it can be cross-referenced from your text.

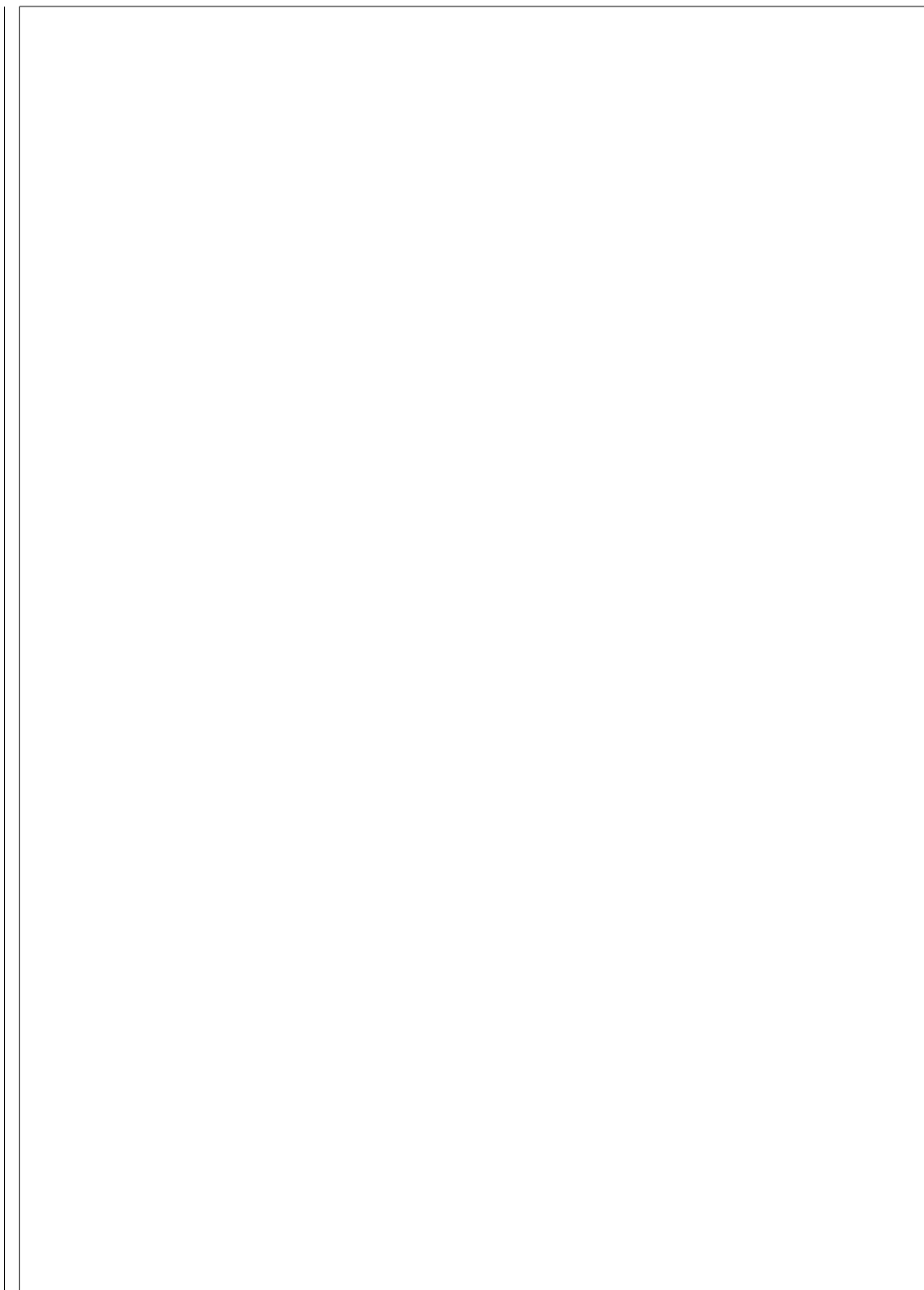
End of Part 4 (Ch. 11–12). Next: Part 5 — Bibliography & Citation Tools (Ch. 13–16).



Part 5: Bibliography & Citation Tools (Chapters 13–16)

--





Chapter 13 — Managing References with BibTeX/BibLaTeX

Objective 13

Store citations in a `.bib` database file instead of typing them by hand, and understand the two competing citation systems (BibTeX and BibLaTeX) well enough to avoid the single most common cause of “my citations aren’t showing up” panic.

Where do `.bib` entries actually come from?

Before diving into syntax: you’re not expected to hand-type every field of every reference. Most literature databases will generate a ready-made BibTeX entry for you — you just copy-paste it into your `.bib` file. A few of the major ones:

- **arXiv** (arxiv.org) — the standard preprint server across physics, astronomy, math, and CS. On any paper’s abstract page, look for an “Export citation” or “Export Bibtex Citation” link.
- **NASA ADS** (Astrophysics Data System, ui.adsabs.harvard.edu) — the standard literature search engine specifically for astronomy and astrophysics. Every record has an “Export Citation” button with a BibTeX option, and you can export an entire library at once.
- **Google Scholar** (scholar.google.com) — cross-field, not always as complete or accurate as a dedicated database, but broadly available. Click “Cite” under any result, then “BibTeX.”
- **Field-specific alternatives:** INSPIRE-HEP for particle physics, PubMed/PubMed Central for biology and medicine, IEEE Xplore or the ACM Digital Library for CS/engineering.
- **Journal publisher websites** (ACS, IEEE, Springer, Elsevier, etc.) — the published (not preprint) version usually has a “Cite this article” or “Export citation” link with a BibTeX option directly on the article page.
- **Crossref** (via DOI) — if you only have a paper’s DOI, `https://api.crossref.org/works/{DOI}` can return citation metadata. More technical than the sources above; only worth it if the other options don’t have the paper.
- **JabRef** (jabref.org) — a free desktop app that manages a `.bib` file directly, rather than exporting from somewhere else. Paste in a DOI and JabRef fetches the full entry for you automatically, cleans up formatting, and lets you edit fields in a proper interface instead of

hand-editing raw BibTeX text. Particularly handy once your `.bib` file has grown past a handful of entries and needs organizing.

- **Your own Zotero/Mendeley library** — covered in full in Ch. 15, but worth knowing now: once you’ve built a library there, exporting it to `.bib` is a single click.

Not every field or journal indexes through ADS specifically — it’s an astronomy/astrophysics-specific tool. If your field isn’t astronomy, arXiv (if your subfield uses it) or your field’s own major database is the more likely first stop; Google Scholar works as a fallback across almost any field.

A word of caution: exported BibTeX entries aren’t always clean. Fields get mangled by an odd export bug, DOIs are sometimes missing, author name formatting can be inconsistent between sources. Treat an exported entry as a solid starting point, not a guarantee — skim it before you trust it.

Worked Example 13.1

BibTeX (the classic system)

```
% mybib.bib
@article{einstein1905,
  author = {Einstein, A.},
  title  = {Zur Elektrodynamik bewegter K{"o}rper},
  journal = {Annalen der Physik},
  year   = {1905},
  volume = {322},
  number = {10},
  pages  = {891--921}
}

% main.tex
\documentclass{article}
\usepackage{natbib}

\begin{document}

Einstein's original formulation \citep{einstein1905} remains
foundational.

\bibliographystyle{plain}
\bibliography{mybib}

\end{document}
\cite{einstein1905} (built into LaTeX) or \citep{einstein1905}/\citet{einstein1905}
(from natbib, giving parenthetical vs. textual citation forms) pull the entry from
mybib.bib. \bibliographystyle{plain} sets the formatting; \bibliography{mybib}
(no .bib extension) points to the database file.
```

Worked Example 13.2

BibLaTeX (the modern alternative)

```
\documentclass{article}
```

```
\usepackage[backend=biber,style=numeric]{biblatex}
\addbibresource{mybib.bib}
```

```
\begin{document}
```

Einstein's original formulation \cite{einstein1905} remains foundational.

```
\printbibliography
```

```
\end{document}
```

Same .bib file, different loading mechanism: \usepackage{biblatex} plus \addbibresource{} instead of \bibliography{}, and \printbibliography instead of \bibliographystyle+\bibliography. biblatex is more actively maintained and flexible (easier style-switching, better Unicode support); BibTeX+natbib is still extremely common, especially in older journal templates.

Rule of thumb: if a journal template already uses natbib, stick with it — don't swap systems mid-template. Starting a document from scratch with no imposed style? biblatex is the more modern default choice.

Nuance 13.1

the citation commands you'll actually use

\cite{} isn't the only citation command, and picking the right one changes how the citation reads in a sentence. Here are the ones you'll reach for most:

What you want	natbib	biblatex	Renders roughly as
Parenthetical citation	\citep{key}	\parencite{key}	"...is well established (Smith, 2020)."
Textual/narrative citation	\citet{key}	\textcite{key}	"Smith (2020) showed that..."
Just the author name(s)	\citeauthor{key}	\citeauthor{key}	"Smith"
Just the year	\citeyear{key}	\citeyear{key}	"2020"
Plain \cite	\cite{key}	\cite{key}	Behavior depends on your chosen style — don't assume it looks the same as \citep/\citet

In practice, \citep/\parencite and \citet/\textcite cover the large majority of citations you'll ever write — reach for those two first, and only look up the rest as a specific sentence needs them.

Nuance 13.2

adding a page number or note — prenotes and postnotes

Both systems accept up to **two** optional arguments in square brackets before the citation key:

```
\citep[p.~5]{einstein1905}
% -> (Einstein, 1905, p. 5)
```

```
\citep[see][p.~5]{einstein1905}
% -> (see Einstein, 1905, p. 5)
```

With one optional argument, it's treated as the **postnote** (typically a page or chapter reference) appended after the citation. With two, the first is the **prenote** (a word like “see” or “cf.” placed before the citation) and the second is the postnote. This is the second optional argument Ali was trying to recall — `\cite[prenote][postnote]{key}`.

A common mistake, using your own example as the case study: `\cite[Good paper here]{Kaneko et al. 2006}` has two problems, both worth catching early: 1. The argument inside `{}` must be the `.bib` file's citation **key** (e.g. `kaneko2006`), not the human-readable author/year text. LaTeX has no way to look up “Kaneko et al. 2006” — it needs the exact key you defined in your `.bib` entry. 2. The bracketed text becomes a literal postnote printed after the citation — `\cite[Good paper here]{kaneko2006}` would render as something like “(Kaneko et al., 2006, Good paper here),” which is valid syntax but probably not the intent. Postnotes are conventionally short (a page number, “p. 12”, “chap. 3”), not descriptive commentary about the paper.

The corrected version, assuming a page reference was actually intended:

```
\citep[p.~12]{kaneko2006}
% -> (Kaneko et al., 2006, p. 12)
```

Nuance 13.3

useful natbib package options

A few `\usepackage[...]{natbib}` options control the most common formatting decisions: `\usepackage[numbers,sort]{natbib}`

- **numbers** — switches citations to numeric style ([1], [2]) instead of the default author-year style.
- **sort** — sorts the keys inside a single multi-citation call (e.g. `\citep{a,b,c}`) into the order they appear in your bibliography, rather than the order you happened to type them.
- **sort&compress** — does what **sort** does, *and* collapses consecutive numeric citations into a range ([1,2,3] becomes [1-3]). Only meaningful when combined with **numbers**. Note the exact spelling: it's **sort&compress** (one combined option with an ampersand) — there's no separate standalone **compress** or **compact** option in `natbib`.

The Compiler Decision Tree

Nuance 13.4

the compiler decision tree

This single setting causes more “citations show up as [?]” panic than anything else in this guide:

- Using `\bibliography{...}` (the classic system)? Set Overleaf’s compiler to run **BibTeX**.
- Using `\usepackage{biblatex} + \addbibresource{...}`? Set Overleaf’s compiler to run **Biber**.
- Seeing “No citation found” or [?] in your PDF? It’s almost always this exact mismatch — check Overleaf’s compiler dropdown before anything else.

Nuance 13.5

citing a webpage or software

Citing a tool, dataset, or website doesn’t fit neatly into “article” or “book,” but comes up constantly in research. With `biblatex`, use the `@online` entry type:

```
@online{comsol2023,
  author = {{COMSOL AB}},
  title  = {Heat Transfer Module User's Guide},
  year   = {2023},
  url    = {https://www.comsol.com/heat-transfer-module},
  note   = {Accessed: 2026-07-10}
}
```

With plain BibTeX, `@online` isn’t a standard type — use `@misc` instead, moving the URL into `howpublished`:

```
@misc{comsol2023,
  author      = {{COMSOL AB}},
  title       = {Heat Transfer Module User's Guide},
  year        = {2023},
  howpublished = {\url{https://www.comsol.com/heat-transfer-module}},
  note        = {Accessed: 2026-07-10}
}
```

Either way, include an access-date `note` — web content changes or disappears, and reviewers may ask when you retrieved it. Make sure `\usepackage{url}` or `hyperref` (already loaded from Ch. 8) is present so `\url{}` renders instead of erroring.

Nuance 13.6

not everyone uses a .bib file at all

BibTeX/BibLaTeX are the right tool once you have more than a handful of references, but they’re not the only option. Many mathematicians (and anyone writing a short paper with only a few citations) skip `.bib` files entirely and type the bibliography directly with `thebibliography` and `\bibitem`:

```
\begin{thebibliography}{9}
```

```

\bbitem{einstein1905}
A.~Einstein,
\emph{Zur Elektrodynamik bewegter K\orper},
Annalen der Physik \textbf{322} (1905), no.~10, 891--921.

\bbitem{riemann1859}
B.~Riemann,
\emph{"Uber die Anzahl der Primzahlen unter einer gegebenen Gr\osse},
Monatsberichte der Berliner Akademie (1859).

```

```
\end{thebibliography}
```

You still cite these exactly the same way — `\cite{einstein1905}` works whether that key comes from a `.bib` file or a `\bbitem`. The `{9}` argument in `\begin{thebibliography}{9}` isn't a count of your references — it's a placeholder LaTeX uses to figure out how wide the numbering column should be (use `{9}` for up to 9 references, `{99}` for up to 99, and so on).

The trade-off: `thebibliography` is entirely manual — no database, no automatic sorting, no one-line style switching like Ch. 14 covers. You format and order every entry by hand, and changing citation style later means retyping the list yourself. That's fine for a short paper with a handful of references; it stops scaling once you're managing dozens or hundreds of citations across a thesis, which is exactly where BibTeX/BibLaTeX earn their keep.

Exercise 13.1

1. Build a `.bib` file with 5 references relevant to your field, pulled from arXiv/ADS/Google Scholar's export feature rather than typed by hand — or use the ready-made starter file for your capstone topic in `supplementary_materials.md` ("Ch. 13 — Starter `.bib` Files for the 4 Capstone Topics").
2. Make sure at least one entry is an `@online` (biblatex) or `@misc` (BibTeX) web/software citation.
3. Cite 3 of your 5 references in a short paragraph, and confirm your Overleaf compiler (BibTeX or Biber, matching your `\bibliography/biblatex` choice) is set correctly.
4. Optional: rewrite 2 of your 5 references as `\bbitem` entries in a `thebibliography` block instead, and cite them the same way — compare the manual effort against the `.bib` approach.

Capstone Update 13

Add your topic's starter `.bib` file (or your own 5 references) to your capstone project, and cite 3 sources in your paper's introduction — including the online/software reference.

Chapter 14 — Citation Styles for Journals & Conferences

Objective 14

Switch between citation styles (numeric vs. author-year, IEEE vs. APA) without manually reformatting a single citation by hand.

Worked Example 14.1

switching styles with BibTeX

```
\bibliographystyle{plain}    % numeric, alphabetical by first author
\bibliographystyle{ieetr}    % numeric, in order of first citation (IEEE)
\bibliographystyle{apalike}  % author-year, APA-like
```

Change one word — the `\bibliographystyle{}` argument — and every citation and the entire reference list reformats automatically. This is the whole point of managing references through a `.bib` database instead of typing them by hand: formatting is separated from data.

Worked Example 14.2

switching styles with BibLaTeX

```
\usepackage[backend=biber,style=numeric]{biblatex} % [1], [2], ...
\usepackage[backend=biber,style=authoryear]{biblatex} % (Smith, 2020)
\usepackage[backend=biber,style=ieee]{biblatex}      % IEEE numeric
```

Same idea: the `style=` option controls formatting, independent of your citation data. (`style=ieee` requires the separate `biblatex-ieee` package, freely available on Overleaf/CTAN.)

Nuance 14.1

journal-specific .cls and .bst files

Most journals provide an official LaTeX template — a `.cls` (document class) and often a `.bst` (bibliography style) file specific to their house style. When you use one, it typically already sets `\bibliographystyle{}` (or the `biblatex style=`) for you — don't override it, even if the citations look unfamiliar. Journal submission systems check for exact

compliance with their own style file.

Exercise 14.1

Take the 3 citations from Ch. 13's exercise and reformat them under two different styles — one numeric (`plain` or `ieeetr`), one author-year (`apalike`, or `authoryear` if using `biblatex`) — and compare how both the in-text citations and the reference list change.

Capstone Update 14

Pick the citation style appropriate for a real or hypothetical target journal in your field, switch to it, and confirm your paper still compiles cleanly with all citations resolving correctly.

Chapter 15 — Reference Managers + Overleaf

Objective 15

Manage a growing reference library in Zotero or Mendeley, and keep it in sync with your Overleaf project's `.bib` file instead of retyping entries by hand.

Worked Example 15.1

Zotero → Overleaf

1. In Zotero, select the references you want, right-click → **Export Items** → format **BibTeX** → save as `references.bib`.
2. In Overleaf, upload `references.bib` into your project (or use Overleaf's built-in reference-sync integration, which can pull an entire Zotero/Mendeley library in automatically).
3. Reference the file exactly as in Ch. 13: `\bibliography{references}` (BibTeX) or `\addbibresource{references.bib}` (biblatex).

Mendeley works the same way in principle: **File** → **Export** → BibTeX format.

Nuance 15.1

JabRef as a lighter, `.bib`-native alternative

Zotero and Mendeley are general-purpose reference managers that happen to export to `.bib`. If you'd rather work with your `.bib` file directly instead of managing a separate library and exporting from it, **JabRef** (mentioned in Ch. 13) is built specifically around `.bib` files — there's no export step at all, since the `.bib` file *is* the library. Paste in a DOI, JabRef fetches and adds the entry; save, and your `.bib` file is already up to date. Upload that file to Overleaf the same way as any other `.bib` file from this chapter.

Nuance 15.2

keep your citation keys stable

When you re-export from Zotero/Mendeley after adding new references, existing keys (the `einstein1905`-style identifiers) can sometimes shift depending on export settings. If citations that used to work suddenly show `[?]` after a re-export, check whether the key

you're citing in your `.tex` file still matches the key in the freshly exported `.bib` file — this is a sync issue, distinct from Ch. 13's compiler-mismatch nuance.

Exercise 15.1

Set up a Zotero (or Mendeley) library with at least 5 references, export it as a `.bib` file, and cite 3 of them in a scratch Overleaf project.

Capstone Update 15

Import a real reference library relevant to your capstone topic (even a small starter one) via Zotero/Mendeley export, replacing or supplementing the starter `.bib` file from Ch. 13.

Chapter 16 — Glossaries, Nomenclature & Acronyms

Objective 16

Define acronyms and symbols once, reference them everywhere with a single command, and generate an automatic glossary/nomenclature list.

Worked Example 16.1

```
the glossaries package
\usepackage[acronym]{glossaries}
\makeglossaries

\newacronym{cnn}{CNN}{Convolutional Neural Network}
\newacronym{rnn}{RNN}{Recurrent Neural Network}

\begin{document}

A \gls{cnn} was used for image classification, while a \gls{rnn}
handled the sequential data. Later uses of the same terms - another
\gls{cnn} here - automatically stay abbreviated.

\printglossary[type=\acronymtype]

\end{document}
```

\gls{cnn} automatically expands to “Convolutional Neural Network (CNN)” the *first* time it’s used, then just “CNN” every time after — no manual tracking of “have I defined this yet?” required. \printglossary[type=\acronymtype] generates the acronym list.

Nuance 16.1

define once, reference with \gls

The entire point of this package is that each acronym lives in one place. Decide halfway through writing that a term should be worded differently, or need to rename it? Fix the single \newacronym{} line — every \gls{} call updates automatically. Hand-typing

“Convolutional Neural Network (CNN)” every time you first mention it in a new section is exactly the find-and-replace nightmare this package exists to prevent.

Worked Example 16.2

a nomenclature/symbols list

A symbols list is a *separate* glossary from your acronyms, so it needs its own declared type before you can define entries into it or print it on its own:

```
\usepackage[acronym]{glossaries}
\newglossary[slg]{symbols}{sym}{sbl}{List of Symbols}
\makeglossaries

\newglossaryentry{lambda}{
  type=symbols,
  name={\lambda},
  description={decay constant},
  sort={lambda}
}
```

Referenced in text: `\gls{lambda}` controls the rate of decay.

The same `glossaries` machinery handles a symbols list, not just acronyms — useful for a thesis’s “List of Symbols” front matter. The `type=symbols` key is what routes this entry into the symbols glossary instead of the default one.

Nuance 16.2

printing acronyms and symbols separately

`\printglossary[type=\acronymtype]` (shown above) only prints the acronym list. If you’ve also defined a `symbols` type as above, print it with its own call:

```
\printglossary[type=\acronymtype, title={List of Acronyms}]
\printglossary[type=symbols, title={List of Symbols}]
```

Calling plain `\printglossary` with no `type` prints only the *default* glossary (neither acronyms nor your custom `symbols` type) — a common source of “why is my list empty?” confusion. Always match the `type=` in `\printglossary` to whatever `type=` you used when defining the entries.

Nuance 16.3

compiling glossaries needs an extra step

Unlike most packages, `glossaries` requires an additional compilation step beyond pdfLaTeX/XeLaTeX — a helper program (`makeglossaries`) that builds the sorted glossary file. Overleaf handles this automatically in most templates; if your glossary or acronym list shows up empty, that extra step not running is the first thing to check.

Exercise 16.1

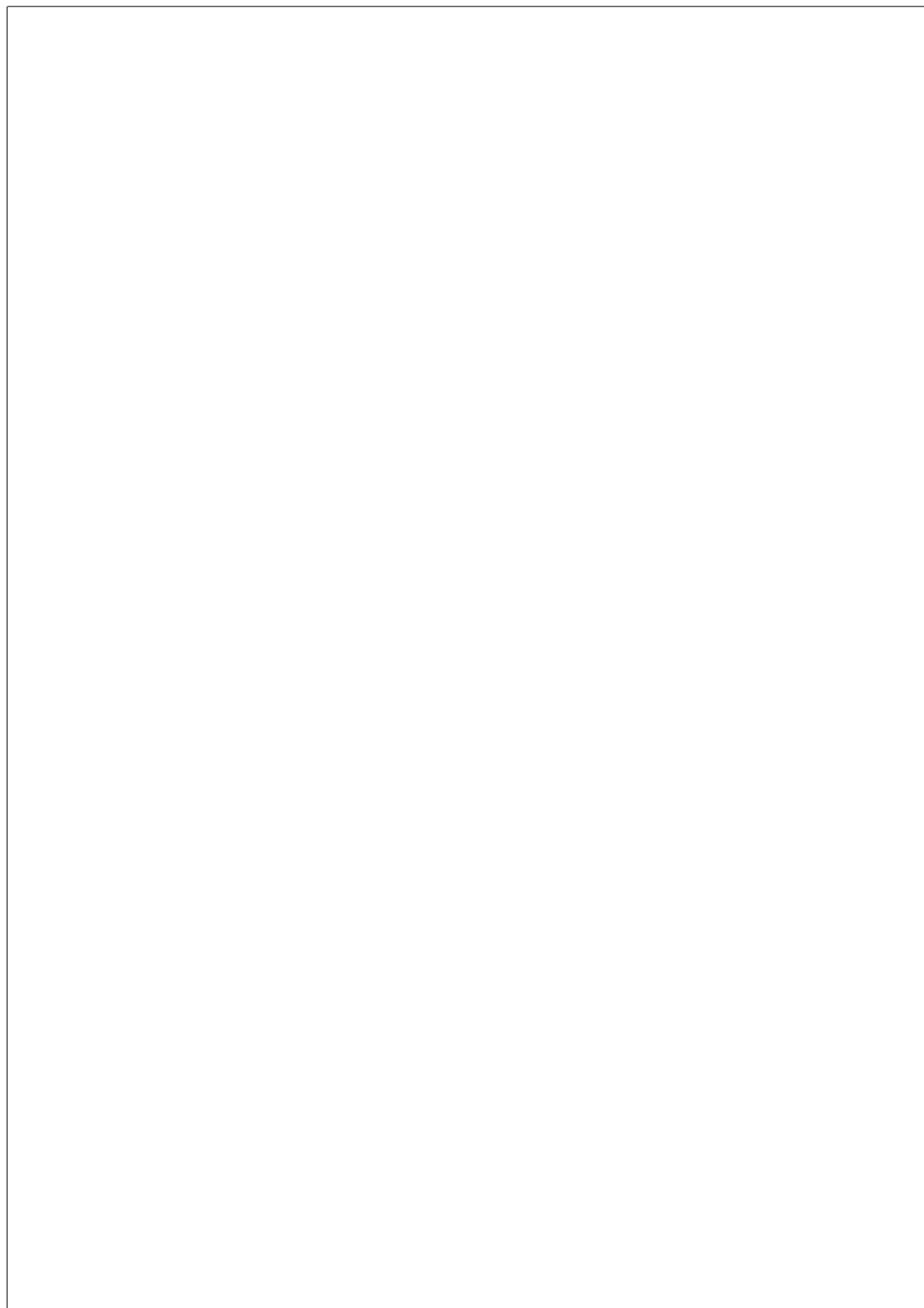
Build a glossary of 8 acronyms or symbols relevant to your field using `\newacronym`/`\newglossaryentry`, use at least 3 of them with `\gls{}` in a short para-

graph (including one repeated use, to see the auto-abbreviation behavior), and print the list. (A quick-reference table of `\printglossary` scenarios is in `supplementary_materials.md` — “Ch. 16 — Glossary Printing Command Reference,” if you want it side by side while working.)

Capstone Update 16

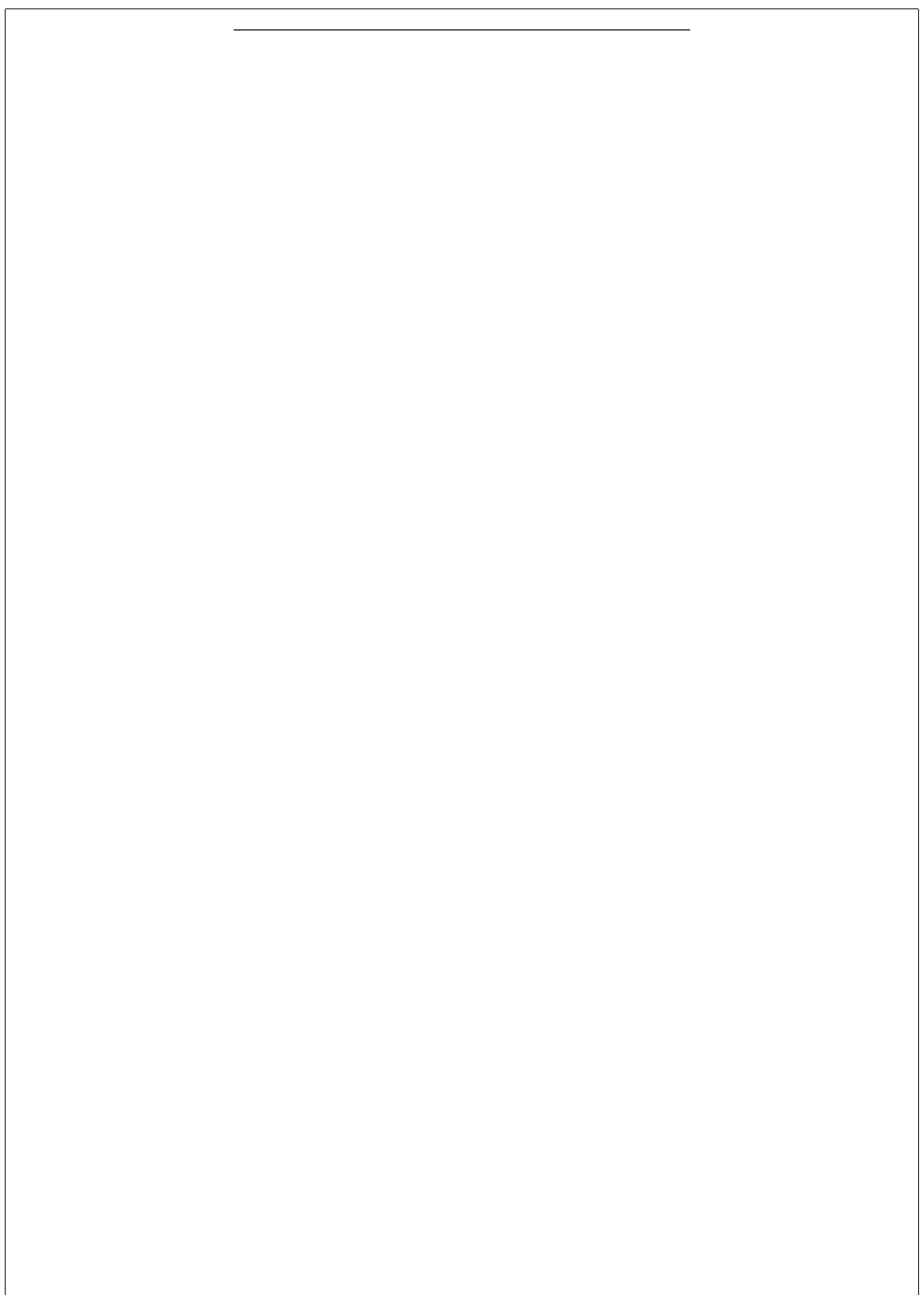
Add an acronym list and/or a nomenclature table to your capstone paper for any recurring abbreviations or symbols in your topic, and print it as front matter.

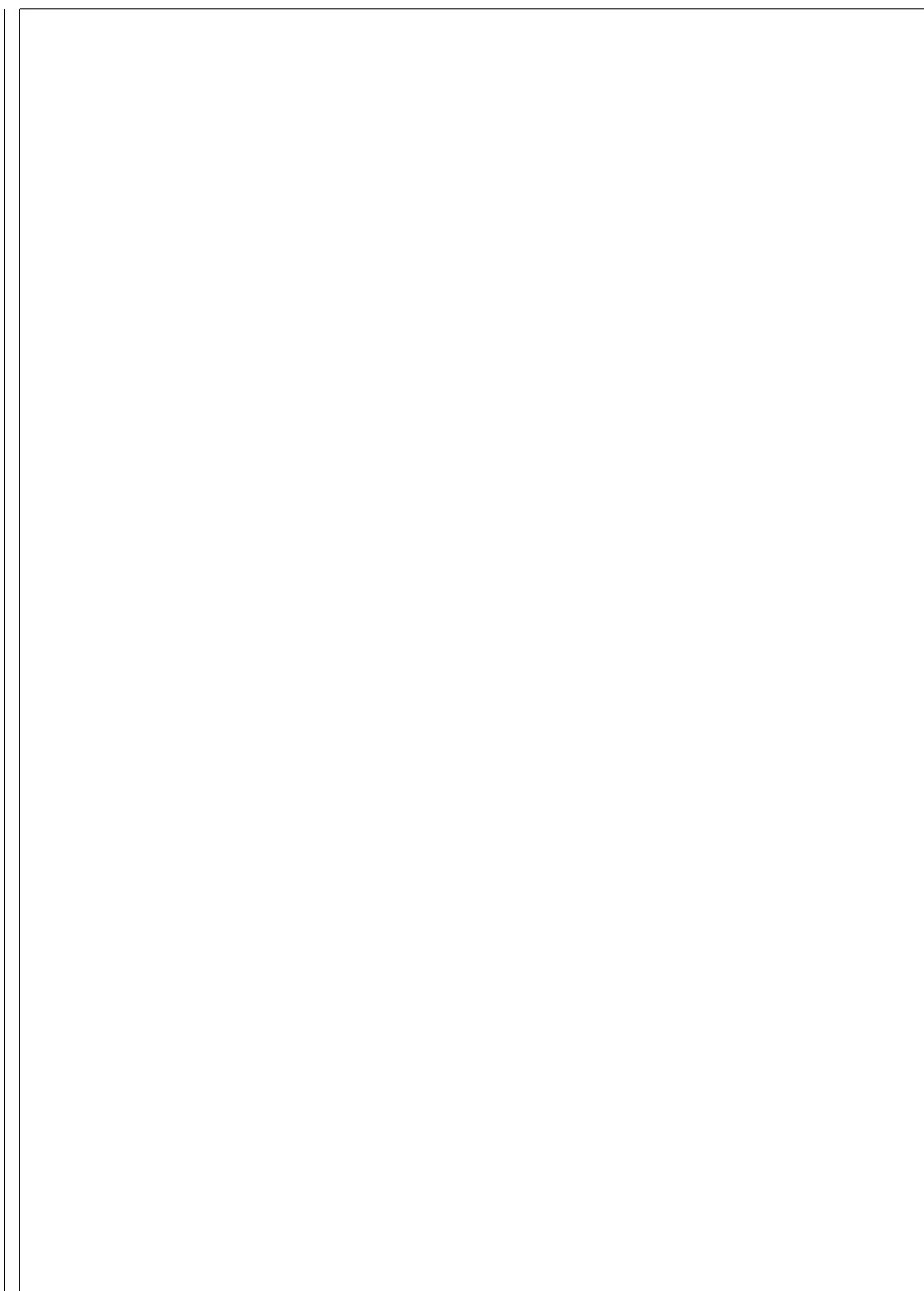
End of Part 5 (Ch. 13–16). Next: Part 6 — Diagrams & Data Visualization (Ch. 17–18).



Part 6: Diagrams & Data Visualization (Chapters 17–18)

--





Chapter 17 — Diagrams: Vector Graphics Workflow & TikZ (Elective)

Objective 17

Get a diagram into your paper the way most researchers actually do it, and know when — if ever — TikZ is worth reaching for instead.

The primary path: draw elsewhere, export as vector PDF

For most research diagrams — flowcharts, block diagrams, schematics, conceptual figures — the fastest and most reliable path is: 1. Draw the diagram in PowerPoint, Google Slides, Inkscape, or any tool you're already comfortable with. 2. Export/save it as a **PDF** (vector format) — not PNG or JPG. PowerPoint: File → Export → PDF. Inkscape: File → Export As → PDF. 3. Include it exactly like any other figure from Ch. 7:

Worked Example 17.1

```
including the exported PDF like any other figure
\usepackage{graphicx}
...
\begin{figure}[htbp]
\centering
\includegraphics[width=0.7\textwidth]{my-diagram.pdf}
\caption{Overview of the experimental setup.}
\label{fig:setup}
\end{figure}
```

This is what roughly 90% of researchers actually do, and it's a completely legitimate, professional-quality workflow — a vector PDF from PowerPoint or Inkscape scales cleanly and looks identical to a hand-coded diagram once it's sitting in your paper.

Nuance 17.1**PowerPoint’s PDF export can leave a white margin**

PowerPoint’s PDF export often includes extra white space around your diagram — the exported PDF is slide-shaped, not diagram-shaped, so `\includegraphics` renders a mostly-empty page instead of a tight crop. Two fixes:

- **In PowerPoint:** File → Export → Create PDF/XPS, then check the “Minimum size” option before saving. Inkscape’s PDF export crops to content by default and usually doesn’t have this problem.
- **In LaTeX, after the fact:** crop it yourself with `graphicx`’s `trim/clip`:
`\includegraphics[trim=1cm 1cm 1cm 1cm, clip, width=0.7\textwidth]{my-diagram.pdf}`
 The four trim values are left/bottom/right/top margins to cut away, in that order.

Nuance 17.2**TikZ is powerful, but it’s elective**

TikZ (via the `tikz` package) draws diagrams directly in LaTeX code — precise, version-controlled, and automatically matched to your document’s fonts. It’s also notoriously steep to learn, and can slow compilation noticeably on complex diagrams. **This guide flags it as elective:** nothing later depends on it, and choosing PowerPoint/Inkscape instead is not a lesser approach. Only invest the time in TikZ if you specifically want diagrams generated from code — for reproducibility, or a diagram that needs to update automatically alongside changing data.

Worked Example 17.2**a minimal TikZ diagram**

```
\usepackage{tikz}
...
\begin{figure}[htbp]
\centering
\begin{tikzpicture}
  \node[draw, rectangle] (a) at (0,0) {Input};
  \node[draw, rectangle] (b) at (3,0) {Process};
  \node[draw, rectangle] (c) at (6,0) {Output};
  \draw[->] (a) -- (b);
  \draw[->] (b) -- (c);
\end{tikzpicture}
\caption{A minimal three-stage pipeline.}
\label{fig:pipeline}
\end{figure}
```

`\node[draw, rectangle] (a) at (x,y) {text}` places a labeled box at coordinates (x,y) and names it **a** for later reference. `\draw[->] (a) -- (b)` draws an arrow between two named nodes. Coordinates are in the document’s default unit (roughly centimeters) unless you specify otherwise.

Nuance 17.3**(0,0) isn't fixed at the center of the picture**

(0,0) is just your coordinate origin — positive x goes right, positive y goes up — not a fixed “center” of the final image. TikZ automatically sizes the picture's bounding box to fit whatever you draw, so where (0,0) ends up visually depends entirely on the other coordinates you use. If all your nodes happen to sit at positive coordinates, (0,0) will appear near a corner, not the middle — nothing to fix, just don't expect centering that isn't actually happening.

Nuance 17.4**field-specific TikZ libraries**

If TikZ is worth learning for your field specifically, two extensions handle domain-specific diagrams far more concisely than raw `tikzpicture`:

- **tikz-cd** — for commutative diagrams (common in pure math, category theory):

```
\usepackage{tikz-cd}
...
\begin{tikzcd}
A \arrow[r] \arrow[d] & B \arrow[d] \\
C \arrow[r] & D
\end{tikzcd}
```

- **chemfig** — for chemical structure diagrams:

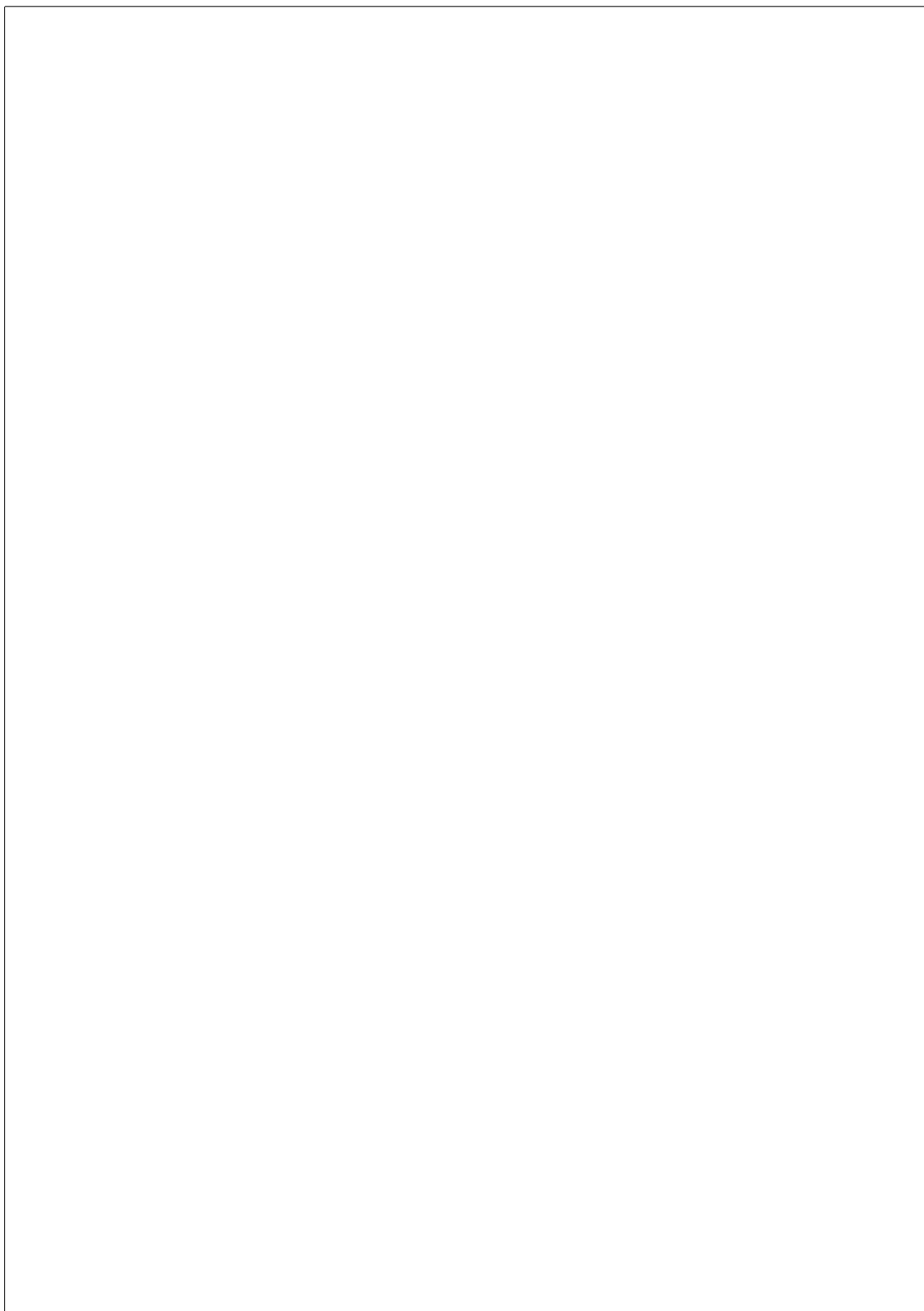
```
\usepackage{chemfig}
...
\chemfig{H-C(-[2]H)(-[6]H)-H}
```

Exercise 17.1

Pick one simple diagram (3-5 boxes/arrows is plenty) and reproduce it two ways: once as an exported vector PDF from PowerPoint/Inkscape, once as a minimal TikZ picture. Compare how long each approach actually took — that comparison is the whole point of the “elective” framing above. (*If you want to see how far TikZ can scale before you decide whether it's worth it for you, [supplementary_materials.md](#) has a fully-commented, more complex example — “Ch. 17 — Full Working Example: Neural Network Architecture Diagram.”*)

Capstone Update 17

Add one diagram to your capstone paper — a system overview, experimental setup, or conceptual figure — using whichever method (imported PDF or TikZ) fits your comfort level and time budget.



Chapter 18 — Plotting Data with PGFPlots

Objective 18

Plot real data — line plots, scatter plots, error bars — directly from LaTeX, so your figures automatically share fonts and styling with the rest of your paper.

Worked Example 18.1

a simple line plot

```
\usepackage{pgfplots}
\pgfplotsset{compat=1.18}
...
\begin{figure}[htbp]
\centering
\begin{tikzpicture}
\begin{axis}[
  xlabel={Time (s)},
  ylabel={Temperature (K)},
]
\addplot coordinates {
  (0, 300) (1, 310) (2, 325) (3, 335) (4, 340)
};
\end{axis}
\end{tikzpicture}
\caption{Temperature rise over time.}
\label{fig:temp-plot}
\end{figure}
```

`pgfplots` is built on `TikZ`, so it shares the same `tikzpicture` wrapper. `\pgfplotsset{compat=1.18}` locks in a specific version of `pgfplots`' default behavior — always set this, since defaults have shifted across versions and an unset `compat` can make your plot look subtly different depending on which TeX Live version compiles it. The `axis` environment auto-generates axes, tick marks, and labels; `\addplot coordinates {...}` supplies the data points directly.

Nuance 18.1

importing from an external .csv instead of typing data by hand

Typing data points inline doesn't scale past a handful of values. For a real dataset, import it directly:

```
\usepackage{pgfplotstable}
...
\begin{axis}[xlabel={Time (s)}, ylabel={Temperature (K)}]
\addplot table[x=time, y=temp, col sep=comma] {data.csv};
\end{axis}
```

where `data.csv` has a header row (`time,temp`) followed by your rows of numbers. `col sep=comma` tells pgfplots the file is comma-separated — use `col sep=tab` instead for tab-separated data. *(A ready-made copy of exactly this dataset, as an actual .csv file, is in the guide's supplementary materials — see `ch18_sample_data.csv` — so you can test this without building your own file first.)*

Nuance 18.2

two ways this silently goes wrong

- **Wrong file path.** The `.csv` path is relative to your main `.tex` file's location, not to where you happen to be looking in Overleaf's file browser. If your data file is in a subfolder (e.g. `data/experiment.csv`), you must include that folder in the path: `{data/experiment.csv}`. Get the path wrong and the compiler either errors on a missing file or — depending on your LaTeX distribution — just renders an empty axis with no visible warning.
- **Column name mismatch.** `x=time`, `y=temp` must match your CSV's header row *exactly*, including capitalization and spacing. `x=Time` when your header says `time` (lowercase) won't silently fall back to the right column — it fails to find that column, and the plot comes out empty or pgfplots reports it can't find a column by that name. If your plot looks blank, check the header spelling before assuming the data itself is wrong.

Worked Example 18.2

scatter plot with a legend

```
\begin{axis}[
  xlabel={Concentration (mM)},
  ylabel={Reaction Rate},
  legend pos=north west,
]
\addplot[only marks, mark=*] coordinates {(1,2.1) (2,3.8) (3,5.2) (4,6.9)};
\addlegendentry{Trial 1}
\addplot[only marks, mark=square*] coordinates {(1,1.9) (2,3.5) (3,5.0) (4,6.5)};
\addlegendentry{Trial 2}
\end{axis}
```

`only marks` (instead of the default connected line) produces a scatter plot; `mark=*/mark=square*` distinguish the two series visually; `\addlegendentry{}` labels each series in the legend, matched to the `\addplot` calls in the same order they appear.

Worked Example 18.3

```

error bars
\addplot+[
    error bars/.cd,
    y dir=both,
    y explicit,
]
coordinates {
    (1, 2.1) +- (0, 0.2)
    (2, 3.8) +- (0, 0.3)
    (3, 5.2) +- (0, 0.25)
};
error bars/.cd, y dir=both, y explicit turns on vertical error bars in both direc-
tions; each coordinate's +- (x-error, y-error) supplies the bar size for that specific
point.

```

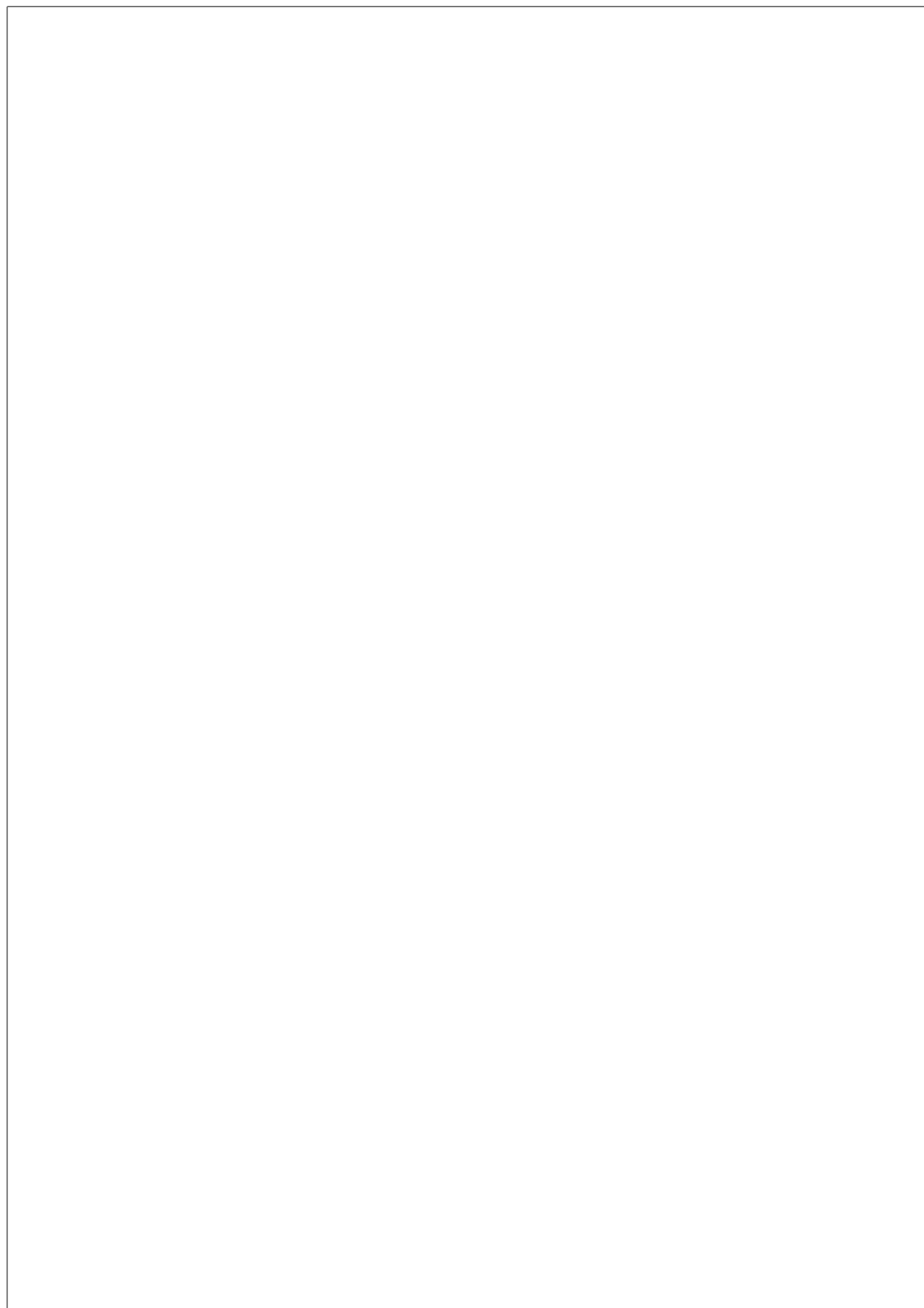
Exercise 18.1

Plot a small dataset (5-8 points) of your own choosing with `pgfplots`, including error bars on at least 3 points and a legend if you have more than one data series. (*No dataset handy? `ch18_sample_data.csv` in the supplementary materials has 8 points with an uncertainty column ready to plot — see “Ch. 18 — Full Working Example” for a complete document using it.*)

Capstone Update 18

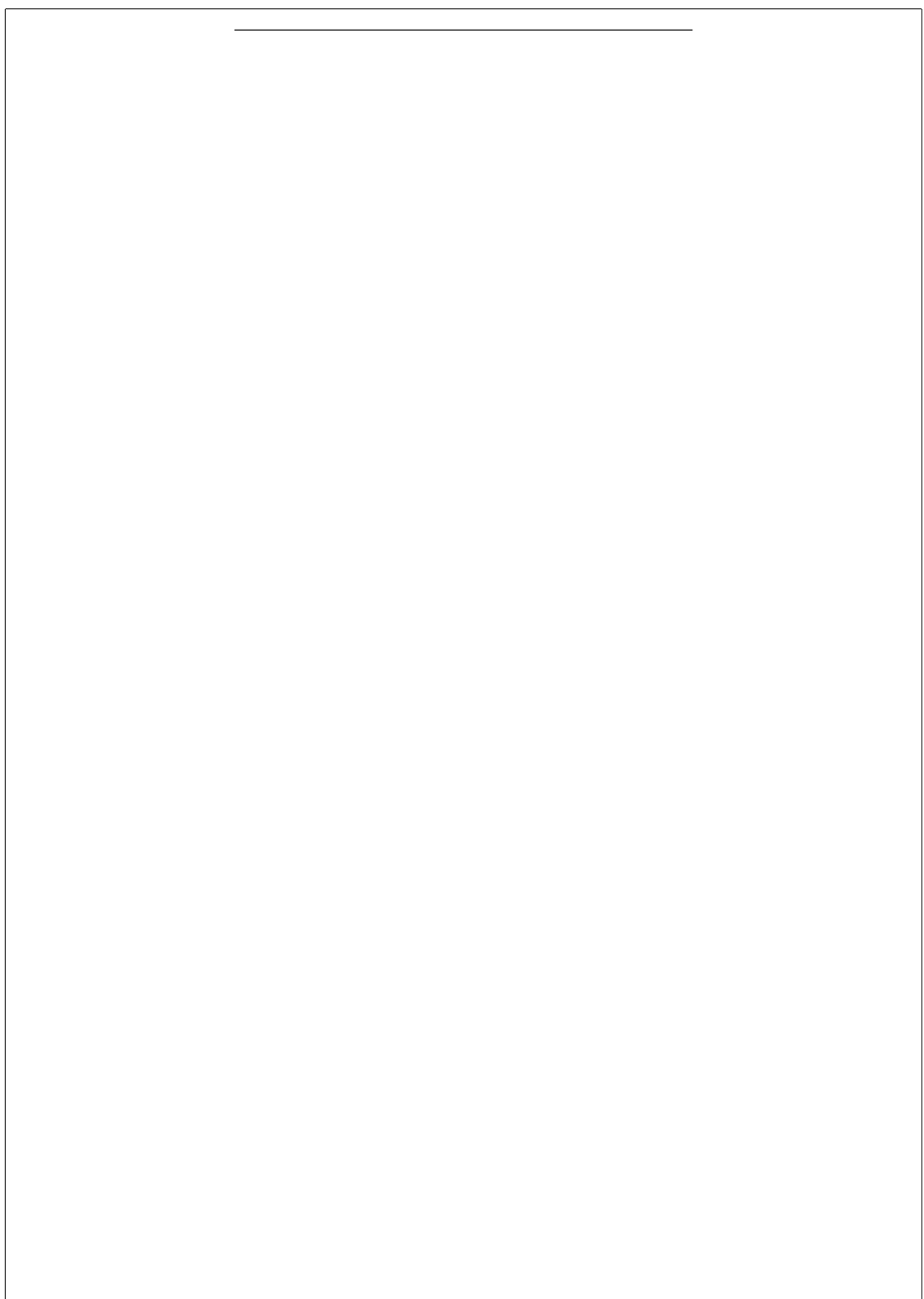
Add a data plot to your capstone paper’s results section — real data if you have it, plausible placeholder numbers otherwise — with axis labels, a legend if applicable, and error bars if your data has associated uncertainty.

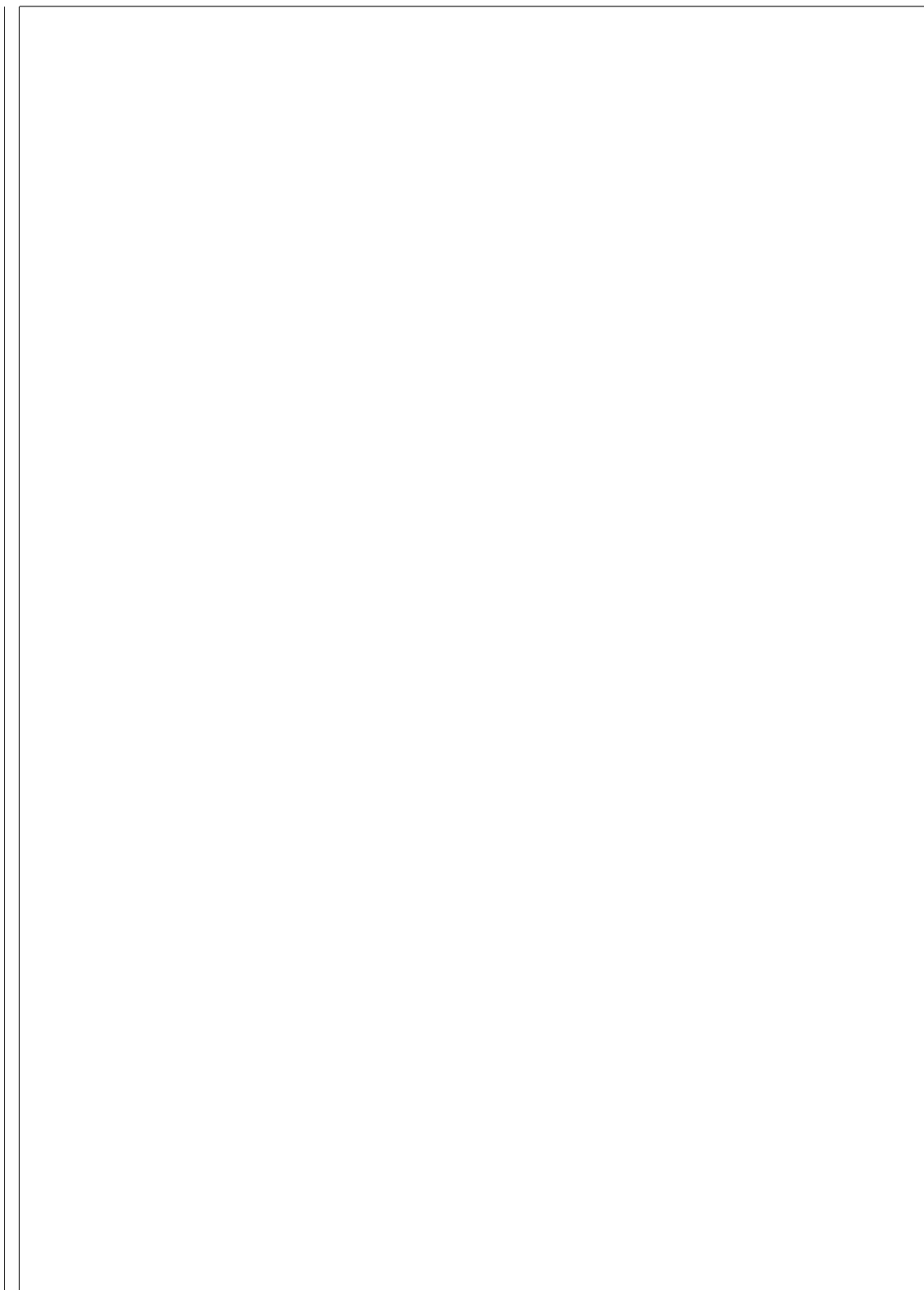
End of Part 6 (Ch. 17–18). Next: Part 7 — Macros & Templates (Ch. 19–20).



Part 7: Macros & Templates (Chapters 19–20)

--





Chapter 19 — Custom Commands & Preamble Management

Objective 19

Define reusable macros for notation you type repeatedly, and avoid the most common way custom commands silently break or collide.

Worked Example 19.1

```
\newcommand basics
\usepackage{amssymb} % for \mathbb - separate from amsmath, load both
...
\newcommand{\R}{\mathbb{R}}
\newcommand{\norm}[1]{\lVert #1 \rVert}

...
```

For any $x \in \mathbb{R}$, $\|x\| \geq 0$.

`\newcommand{name}{replacement}` defines a zero-argument macro.
`\newcommand{name}[1]{replacement using #1}` defines one that takes an argument — `#1` stands in for whatever you pass in braces when you call it (`\norm{x}` expands to `\lVert x \rVert`). `\lVert`/`\rVert` come from `amsmath` (already loaded from Ch. 9); `\mathbb{}` specifically needs `amssymb`, a separate package worth loading alongside it.

Worked Example 19.2

```
multiple arguments
\newcommand{\deriv}[2]{\frac{d#1}{d#2}}
...
 $\frac{dy}{dx}$  % renders as a dy/dx fraction

\newcommand{\vect}[1]{\bm{#1}} % wraps the bm package from Ch. 9
...
 $\mathbf{v}$  % bold v, without retyping \bm{} every time
Macros can take multiple arguments (#1, #2, ... up to #9), and can wrap commands
```

you’ve already learned so you don’t have to remember the underlying syntax every time you need it.

Nuance 19.1

an optional argument with a default value

`\newcommand` can also define a macro with **one optional argument** (with a default value you specify) followed by any number of required arguments:

```
\newcommand{\deriv}[3][x]{\frac{d#2}{d#1^{#3}}}
```

```
\deriv{y}{2}      % no optional argument supplied -> defaults to x -> dy/dx^2
```

```
\deriv[t]{y}{3}   % optional argument supplied as t -> dy/dt^3
```

The syntax is `\newcommand{\name}[n][default]{...}`, where `n` is the *total* number of arguments including the optional one, and `default` is what `#1` becomes if you don’t supply it in square brackets at call time. `#1` always refers to the optional argument; `#2`, `#3`, ... are the required ones. This is less commonly needed than the plain multi-argument form above, but useful once you’re building a small library of your own macros — for something like a derivative command where the variable is almost always `x`, but occasionally needs to be something else.

Nuance 19.2

macros can reuse other macros you’ve already defined

A `\newcommand` definition can reference another macro you’ve already defined, letting you build more specific notation on top of general notation instead of repeating yourself:

```
\newcommand{\R}{\mathbb{R}}
```

```
\newcommand{\Rn}{\R^n}    % reuses \R from above
```

```
...
```

A vector in $\mathbb{R}^n$ has n real-valued components.

`\Rn` expands to `\mathbb{R}^n` by way of `\R` — if you ever redefine `\R` itself, `\Rn` updates automatically along with it, the same “fix it in one place” benefit Ch. 16 highlighted for `\gls`.

Nuance 19.3

name clashes with package-defined commands

If you `\newcommand{\R}{...}` and later load a package that already defines `\R` (some do), you’ll get a `Command \R already defined` error. Two ways to handle it:

- Rename your macro to something more specific (`\Real` instead of `\R`).
- Use `\renewcommand{\R}{...}` instead — this overrides an existing definition rather than erroring on the clash, but only do this deliberately, since it also silently changes that command’s behavior everywhere else in your document, including anywhere a package was relying on its original meaning.

Nuance 19.4

`\renewcommand` for adjusting built-in package settings — `\arraystretch`

`\renewcommand` isn't only for overriding your own macros on a name clash — it's also the standard way to adjust settings that packages expose as commands rather than options. The most common example, useful directly with the tables from Ch. 6: LaTeX's default table rows sit close together, which can look cramped with tall content (multi-line cells, larger fonts). Loosen it with:

```
\renewcommand{\arraystretch}{1.5}
\begin{tabular}{lcc}
\toprule
Sample & Trial 1 & Trial 2 \\
\midrule
A & 10 & 12 \\
\bottomrule
\end{tabular}
```

Wrapping the `\renewcommand` and the table together in an extra pair of `{ }` scopes the change to just that table, instead of stretching every table row for the rest of the document. 1.5 is a common value (1.5× the default row height) — adjust to taste.

Nuance 19.5

`\DeclareMathOperator` for custom math operators

Typing “argmin” or “rank” directly in math mode renders it in italics, as if it were several multiplied variables (*a* times *r* times *g*...) instead of a single operator name — the same issue Ch. 10 covered for the word “if” inside cases. Fix it once, for good, with `\DeclareMathOperator` (from `amsmath`):

```
\DeclareMathOperator{\argmin}{arg\,min}
\DeclareMathOperator{\rank}{rank}
...
 $\hat{\theta} = \argmin_{\theta} \, \mathcal{L}(\theta)$ 
 $\text{rank}(A) = 3$ 
```

Once declared, `\argmin` and `\rank` behave exactly like built-in operators such as `\sin` or `\max` — upright text, correct spacing, and subscripts/limits positioned properly underneath. If you need both `\argmin` and `\argmax`, declare both separately — neither is built into LaTeX, but once declared, both behave identically to the built-in operators.

Exercise 19.1

Define 3 macros for notation you use repeatedly in your own field: at least one plain text-replacement macro (like `\R` for a set), one that takes an argument (like `\norm{}` or `\deriv{}{}`), and one custom operator via `\DeclareMathOperator`. Optional: try an argument with a default value, or a macro that reuses one you've already defined.

Capstone Update 19

Find at least one expression you've typed more than twice in your capstone paper and replace it with a macro. If your field uses a named operator not built into LaTeX (argmin, rank, softmax, trace, etc.), declare it properly with `\DeclareMathOperator` instead of leaving it in italics.

Chapter 20 — Thesis/Paper Templates, Multi-file Projects & Appendices

Objective 20

Split a long document into manageable per-chapter files, adopt a proper thesis or journal template instead of assembling one from scratch, assemble correct front/back matter, add a correctly-numbered appendix, and switch to two-column layout when a template requires it.

Worked Example 20.1

splitting a document with `\input`

```
% main.tex
```

```
\documentclass{report}
```

```
\begin{document}
```

```
\input{chapters/introduction}
```

```
\input{chapters/methods}
```

```
\input{chapters/results}
```

```
\end{document}
```

```
% chapters/introduction.tex
```

```
\chapter{Introduction}
```

This chapter introduces the problem...

`\input{filename}` (no `.tex` extension needed) pulls that file's content in at exactly that point, as if you'd pasted it there directly — nothing special happens around it.

Nuance 20.1

a separate preamble file is a good convention, not a requirement

`\input` isn't only for splitting up the body — the same mechanism works for the preamble. Once you've accumulated a dozen `\usepackage` lines and a handful of

`\newcommand/\DeclareMathOperator` definitions from Ch. 19, it's a common (optional) convention to move all of that into its own file:

```
% main.tex
\documentclass{article}
\input{preamble}

\begin{document}
\input{chapters/introduction}
...
\end{document}
% preamble.tex
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{bm}
...
\newcommand{\R}{\mathbb{R}}
\DeclareMathOperator{\argmin}{arg\,min}
...
```

Nothing about this is required — a single-file preamble works fine for a short paper. It earns its keep once your document (or your set of macros) grows large enough that scrolling past all of it just to get to the actual content becomes annoying, or when you're reusing the same preamble across multiple documents (a thesis and its related papers, for instance).

Nuance 20.2

`\input` vs. `\include`

`\include{filename}` does the same basic job, but with two differences that matter on long documents:

- It forces a page break before and after the included content.
- Combined with `\includeonly{chapters/results}` in the preamble, it lets you recompile *just* the listed files while treating the others as already compiled — much faster than recompiling an entire 100-page thesis every time you tweak one chapter.

`\input` has neither behavior — no forced page breaks, no partial-recompilation support. For a short document this doesn't matter; for a full thesis, `\include/\includeonly` can turn a two-minute compile into a two-second one while you're actively editing a single chapter.

Worked Example 20.2

using a university/journal template

Most universities and journals distribute an official `.cls` file (often as a ready-made Overleaf template project) rather than expecting you to assemble margins, fonts, and front matter yourself:

```
\documentclass{ourthesis} % provided by your university, not a built-in class
\usepackage{...}          % any extra packages the template doesn't already load
```



```

\begin{document}
\input{frontmatter}
\input{chapters/introduction}
...
\end{document}

```

Don't fight the template's existing formatting decisions (margins, spacing, front-matter order) — if a required section looks “wrong,” the template usually already provides the correct command for it. Check the template's own documentation/comments before overriding anything with the tools from Ch. 5–8.

Worked Example 20.3

front matter, main matter, and back matter (book class)

```

\documentclass[12pt]{book}
\usepackage{...}

```

```

\begin{document}

```

```

\frontmatter
\title{A Study of Something Interesting}
\author{Your Name}
\maketitle
\thispagestyle{empty}

```

```

\tableofcontents
\listoffigures
\listoftables

```

```

\mainmatter
\input{chapters/introduction}
\input{chapters/methods}
...

```

```

\appendix
\input{chapters/derivations}

```

```

\backmatter
\bibliography{mybib}

```

```

\end{document}

```

`\frontmatter`, `\mainmatter`, and `\backmatter` are book-class-specific commands — unlike `\appendix` below, which works the same way in `article`, `report`, and `book`, these three only exist in `book` (and classes built on it). They structure a document into its three natural zones and handle two things automatically: `\frontmatter` switches to roman-numeral page numbers and unnumbered chapters, fitting for a title page, table of contents, and preface; `\mainmatter` switches back to arabic numerals restarting at 1, and chapters resume being numbered; `\backmatter` keeps arabic numbering but switches

chapters back to unnumbered — the conventional place for a bibliography or index. All three also handle the page-parity cleanup (`\cleardoublepage`, see Ch. 7) needed for a clean transition between zones in two-sided documents.

`\thispagestyle{empty}` right after `\maketitle` suppresses the header/footer/page number for just that one page (the title page) — as opposed to `\pagestyle{...}`, which would change the style for the rest of the document. `\tableofcontents`, `\listoffigures`, and `\listoftables` each generate their listing automatically from every `\caption`/`\section`/`\chapter` already in the document — same as any cross-referencing (Ch. 8), they need a second compile pass to populate correctly.

Where `\appendix` fits relative to all this: the conventional order is `\frontmatter` → `\mainmatter` → numbered chapters → `\appendix` → lettered appendix chapters → `\backmatter` → unnumbered back matter. `\appendix` doesn't touch page numbering or which “matter” you're in — it only affects chapter/section numbering, layered on top of whichever zone you're currently in.

Nuance 20.3

the same structure without book class

`article` and `report` — including many university thesis classes built on `report`, not `book` — don't define `\frontmatter`/`\mainmatter`/`\backmatter` at all; check your specific class before assuming they exist. Without them, build the same front-matter behavior by hand:

```
\documentclass{report}
```

```
...
```

```
\begin{document}
```

```
\pagenumbering{roman}
```

```
\maketitle
```

```
\thispagestyle{empty}
```

```
\tableofcontents
```

```
\listoffigures
```

```
\listoftables
```

```
\pagenumbering{arabic} % also resets the page counter to 1 - no separate \setcounter needed
```

```
\chapter{Introduction}
```

```
...
```

```
\end{document}
```

There's no manual equivalent for the numbered-vs-unnumbered chapter switching `\frontmatter`/`\mainmatter`/`\backmatter` handle automatically in `book` — in `report`/`article`, every `\chapter`/`\section` is numbered by default regardless of where it falls, `\appendix` aside.

Worked Example 20.4

```
appendices
```

```
\appendix
```

```

\chapter{Additional Derivations}    % report/book classes
\label{app:derivations}
...
\appendix

\section{Additional Derivations}    % article class (no \chapter available)
\label{app:derivations}
...

```

`\appendix` is a single command, called once, that resets the relevant counter and switches everything after it to letter-based numbering — “Appendix A,” “Appendix B,” instead of continuing the numeric sequence from the main body. This works the same way in **article**, **report**, and **book** alike; you don’t need to manually touch `\thesection` yourself in any of the standard classes, since `\appendix` already handles that internally. The only time you’d add something like `\renewcommand{\thesection}{\Alph{section}}` by hand is with a custom class (e.g. a university thesis `.cls`) that doesn’t define `\appendix` the standard way, or if you specifically want different numbering (Roman numerals instead of letters, for instance). In a **book**-class document, `\appendix` sits between `\mainmatter` and `\backmatter` — see the front/back-matter example above.

Nuance 20.4

two-column layouts and spanning floats

Some journal templates require the body — or parts of it — in two-column format:

```
\documentclass[twocolumn]{article}
```

or switch mid-document, if only one section needs it:

```
\onecolumn
```

... single-column content, e.g. a title page or abstract ...

```
\twocolumn
```

... two-column content resumes ...

Nuance: `\twocolumn` (and `\onecolumn`) force a page break when they execute — the layout doesn’t wrap mid-paragraph, it jumps to a fresh page. Use `\twocolumn`/`\onecolumn` at a natural section boundary, not mid-paragraph, or the page break will land somewhere unexpected. If the switch happens inside a float-heavy section, pair it with `\clearpage` (Ch. 7) beforehand to flush pending floats first — otherwise a float queued for one-column placement can end up trying to place itself across the layout boundary.

A regular **figure**/**table** in two-column mode is constrained to a single column’s width — too narrow for a wide diagram or data table. The starred variants, first mentioned as a forward pointer in Ch. 5’s star-convention nuance, solve this:

```

\begin{figure*}[t]
\centering
\includegraphics[width=\textwidth]{wide_diagram}
\caption{A figure spanning both columns.}
\label{fig:wide}
\end{figure*}

```

figure*/table* span the full text width instead of one column. One placement restriction worth knowing: spanning floats are reliable only with top-of-page (`[t]`) or a dedicated float page (`[p]`) — LaTeX can’t slot a full-width object “here” in the middle of a single

narrow column, so `h/b` are effectively ignored for starred floats in standard document classes.

Nuance 20.5

hiding a draft section with the `comment` package

Commenting out a large block by putting `%` at the start of every line doesn't scale. If you want to temporarily exclude a whole section that isn't ready for review, wrap it instead:

```
\usepackage{comment}
...
\begin{comment}
\section{Draft: Not Ready Yet}
This entire section is hidden from the compiled PDF until the
\texttt{comment} block around it is removed.
\end{comment}
```

Nothing between `\begin{comment}` and `\end{comment}` is processed at all — useful for parking an unfinished section without deleting it or fighting with dozens of `%` symbols.

Exercise 20.1

1. Split a short document into 2-3 chapter files using `\input`.
2. Build a front-matter block: a title page with `\thispagestyle{empty}`, a `\tableofcontents`, and at least one of `\listoffigures`/`\listoftables` — using `\frontmatter`/`\mainmatter` if you're in `book` class, or `\pagenumbering{roman}`/`\pagenumbering{arabic}` by hand otherwise.
3. Add an appendix with at least one entry, using `\appendix` followed by `\chapter{}` (report/book) or `\section{}` (article).
4. Wrap one section in `\begin{comment}... \end{comment}` and confirm it disappears from the compiled PDF, then remove the wrapper and confirm it reappears.
5. Optional, if your field's journal template uses two columns: try `\documentclass[twocolumn]{article}` and place one `figure*` that spans both columns.

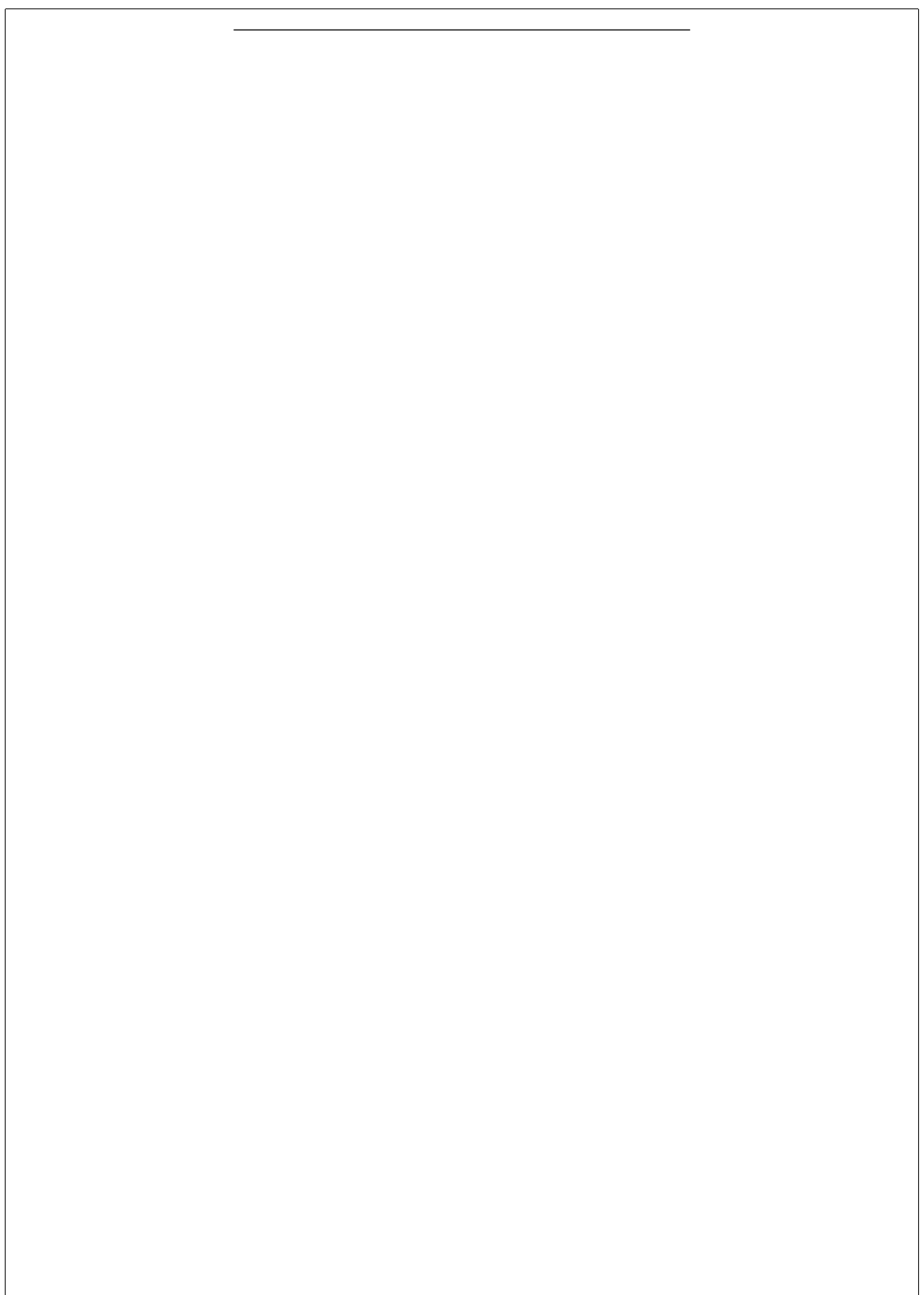
Capstone Update 20

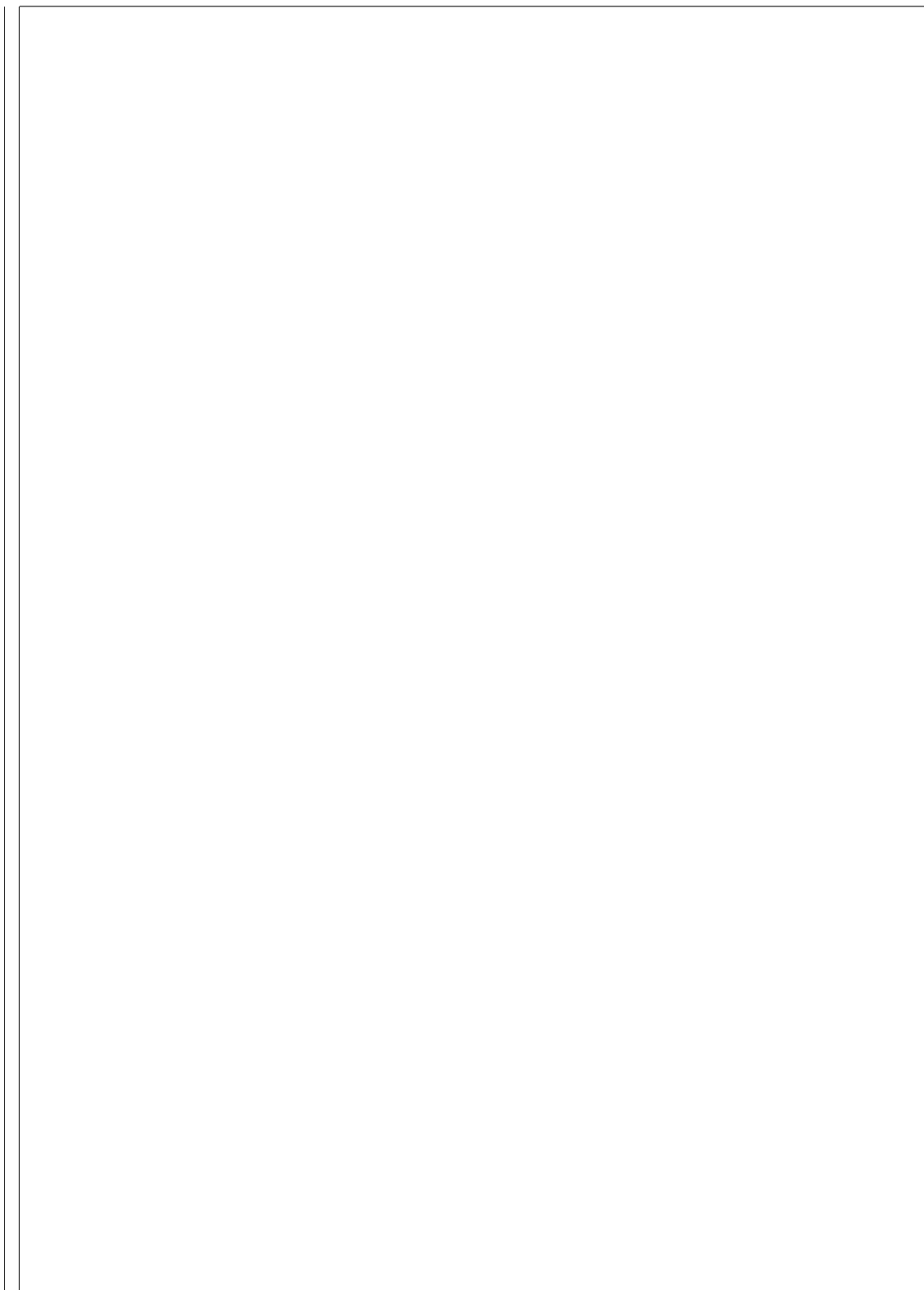
Migrate your capstone paper into a proper multi-file structure (even if it's just 2-3 `\input` files), assemble a real front-matter block (title page, table of contents, list of figures/tables — with page numbering handled correctly for your document class), add an appendix for any supplementary derivations or data, and use `comment` to park any section that isn't ready for review yet.

End of Part 7 (Ch. 19–20). Next: Part 8 — Collaboration & Diagnostics (Ch. 21–22).

Part 8: Collaboration & Diagnostics (Chapters 21–22)

--





Chapter 21 — Version Control, Collaboration & Journal Submission

Objective 21

Set up version control for a LaTeX project — via Overleaf's Git integration or its simpler History panel — collaborate with co-authors using track changes and comments, and prepare a clean, submission-ready PDF.

Worked Example 21.1

Overleaf's Git integration

```
# From Overleaf: Menu -> Git, then copy the clone URL it gives you
git clone https://git.overleaf.com/xxxxxxxxxxxx myproject
cd myproject
```

```
# edit locally in any editor, then:
git add .
git commit -m "Add methods section"
git push origin master
```

```
# to pull changes made back on Overleaf's website:
git pull origin master
```

This clones your Overleaf project as a real Git repository — edit it locally, push, and the changes appear back in your Overleaf project (and vice versa with `git pull`). (*Git access is tied to your Overleaf plan — check your account if the Git option isn't visible under Menu.*)

Nuance 21.1

not every student needs Git

Git is the gold standard, but not everyone comes in with a CS background — common in biology/chemistry labs, for instance. Overleaf's built-in **History** panel (the clock icon in the top toolbar) tracks every compile automatically, with no setup at all, and lets

you revert to any previous version directly. If the Git workflow above feels like overkill, History covers the same core need — “let me get back an earlier version” — with zero configuration.

Nuance 21.2

track changes and comments

Toggle **Review mode** (top toolbar) to turn on Word-style tracked changes — insertions and deletions appear in color, attributed to whoever made them, and can be accepted or rejected individually. Select any text and click the comment icon to leave a note without touching the text itself. Both are the natural first stop for collaborating with a co-author who doesn’t want to touch Git at all — exactly the audience the History panel above also serves.

Nuance 21.3

working offline — a TeX distribution plus an editor

Everything so far assumes Overleaf. Working locally needs two separate pieces: a **TeX distribution** (the actual compiler and package library) and an **editor**. A common combination:

- **TeX Live** — download from tug.org/texlive (or install via your OS’s package manager). The **basic** scheme installs a minimal core, with missing packages added manually via **tlmgr** (TeX Live’s own package manager) as you need them; the **full** scheme installs everything upfront instead, if you’d rather not think about it again.
- **MiKTeX** — download from miktex.org. Installs a small base by default and prompts to install any missing package on-the-fly, the first time a document actually needs it — a similar philosophy to TeX Live’s basic scheme, just automated.
- **TeXStudio** — download from texstudio.org, works on top of either distribution above. It auto-detects your installed compiler in most cases; set your default (pdfLaTeX/XeLaTeX/LuaLaTeX — the same decision as Ch. 2, just made once in Options instead of per-project) and you get the same edit/compile/preview loop Overleaf gives you, running entirely on your own machine.

Nuance 21.4

real Git, via the terminal

Once you’re working locally, Git works on a **.tex** project exactly like it would on any code project — nothing LaTeX-specific about it, since (as Ch. 1 pointed out) a **.tex** file is just plain text:

```
git init
git add .
git commit -m "Initial draft"
git remote add origin <your-repository-url>
git push -u origin main
```

This is the “real” version of what Overleaf’s Git bridge gives you in the browser — run directly from a terminal, alongside TeXStudio or whichever editor you’re using.

Nuance 21.5

PyCharm as a workaround, not a recommendation

If you'd rather have Git's visual tools — diffs, branch graphs, merge conflict resolution — built into your editor instead of typed at a terminal, PyCharm (or any JetBrains IDE) plus the community TeXify-IDEA plugin gets you LaTeX editing and compiling alongside PyCharm's polished built-in Git support. This is a genuine workaround some people use, not a purpose-built LaTeX tool — worth it specifically if you already live in a JetBrains IDE and want Git's GUI more than you want an editor designed for LaTeX first. TeXStudio remains the more natural choice if you're starting from scratch.

Worked Example 21.2

a camera-ready checklist

Before calling a PDF submission-ready:

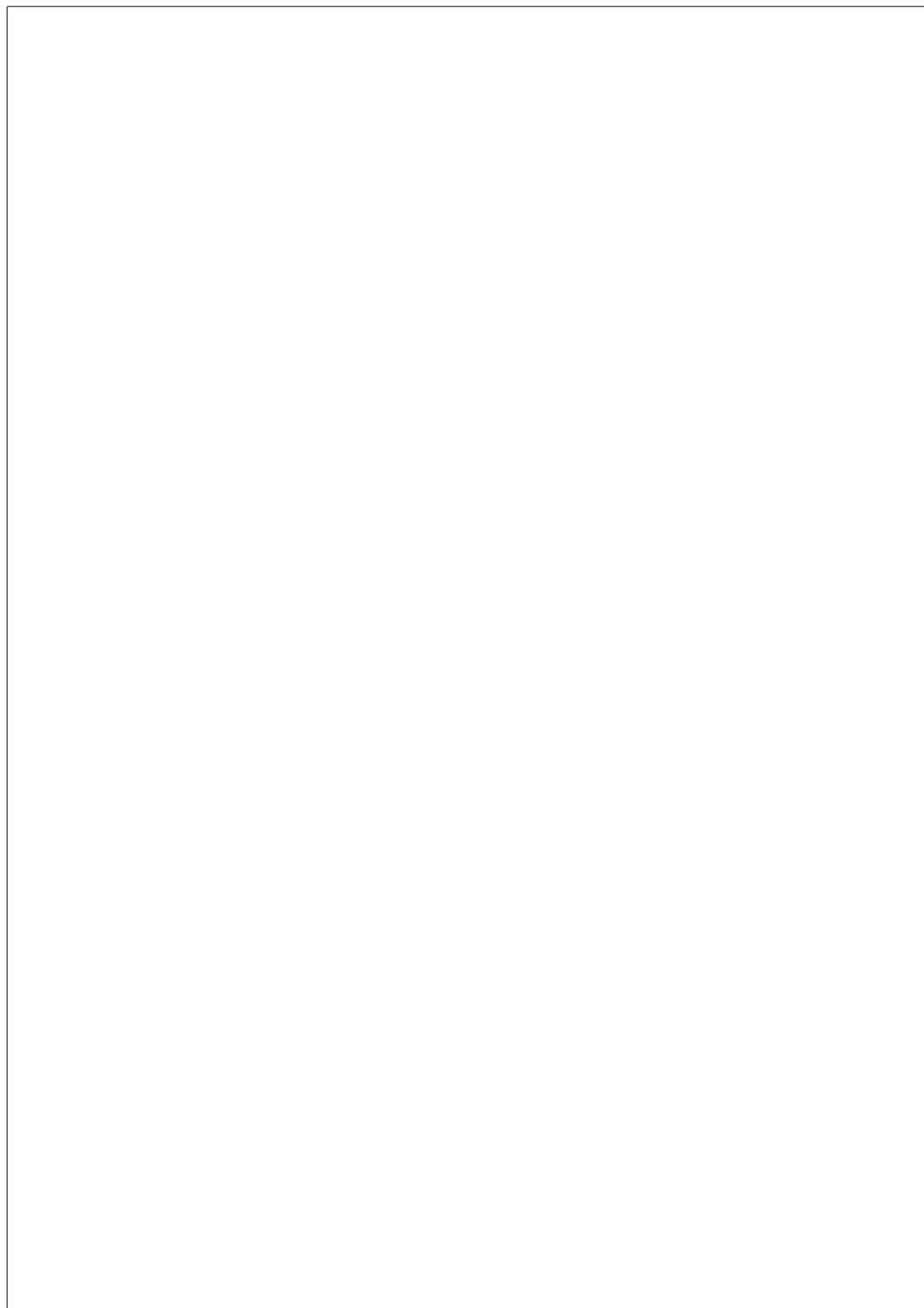
- Every draft-only section is wrapped in `\begin{comment}...\end{comment}` (Ch. 20), not just commented out with stray % signs.
- The compiler matches what the journal/thesis office actually requires (Ch. 2).
- `\hypersetup` (Ch. 8) is set appropriately — `colorlinks` for a digital submission, `hidelinks` if the venue wants no visible link styling at all.
- The document has been compiled at least twice since the last edit, so cross-references (Ch. 8), the bibliography (Ch. 13), and the table of contents (Ch. 5) are all fully settled, not showing stale ?? marks.
- No leftover placeholder text (`[Working Title...]`-style brackets from Ch. 1–4) remains anywhere in the document.

Exercise 21.1

1. Either link an Overleaf project to Git and make two commits, or practice reverting to a previous version using the History panel.
2. Turn on Review mode, make a tracked edit, and accept or reject it.
3. Optional, if you want to try the offline route: install TeXStudio plus TeX Live or MiKTeX, and compile a document you've already built in Overleaf — confirm it produces the same PDF.

Capstone Update 21

Tag or save a “submission-ready” version of your capstone paper — a Git commit/tag, an Overleaf History checkpoint, or a local Git commit if you tried the offline route — and run through the camera-ready checklist above before considering it done.



Chapter 22 — Troubleshooting Common Errors (Full Reference)

Objective 22

Read a full LaTeX log file rather than just the first red error, recognize the most common error categories on sight, and isolate a broken section methodically instead of guessing line by line.

Worked Example 22.1

what's actually in a log file

A compile log is mostly noise — package load messages, font substitution notes, and warnings that don't stop compilation. The parts that matter:

! Undefined control sequence.

```
1.12 \RRR
      {x}
```

?

- A line starting with ! is an actual **error** — this is what stopped the compile.
- The 1.12 line tells you exactly which source line LaTeX was processing when it gave up.
- Anything *not* starting with ! — package messages, font warnings, **Overfull \hbox** notices — is informational. Safe to skim past on a first read, unless the compile genuinely failed and nothing else explains why.
- LaTeX Warning: Label(s) may have changed. Rerun to get cross-references right. is a **warning**, not an error — it means exactly what Ch. 8 and Ch. 9 already told you: compile again so references/citations settle.

Nuance 22.1

error message reference table

Expands on the short version already seen in Ch. 3:

Error	Typical cause
Undefined control sequence	A typo'd command name, or the package that defines it was never loaded.
Missing \$ inserted	A math-only character ($_{}$, $^{\wedge}$) used outside math mode, or an unclosed $_{}$.
Missing } inserted / Runaway argument?	An unclosed brace $\{$ — usually reported on the line <i>after</i> the real culprit, since LaTeX doesn't notice until it hits the next command (Ch. 2).
! LaTeX Error: File 'xxx.sty' not found	The package genuinely isn't installed — rare on Overleaf (everything's preinstalled), more common on a fresh local TeX Live “basic” install (Ch. 21) that hasn't fetched it yet.
Environment X undefined	A typo in $\backslash\text{begin}\{\}$ / $\backslash\text{end}\{\}$, or the package defining that environment was never loaded.
! Package babel Error / similar package-specific errors	Read the message itself — package errors usually name the exact problem, unlike the generic errors above.

Nuance 22.2

isolating an error by bisection

When it's not obvious which of several hundred lines broke, don't read them all — cut the search in half repeatedly: 1. Wrap the back half of the document in $\backslash\text{begin}\{\text{comment}\}\dots\backslash\text{end}\{\text{comment}\}$ (Ch. 20) and recompile. 2. If the error disappears, it was in the commented-out half — move the boundary and repeat inside that half. 3. If the error remains, it's in the half still active — repeat there instead. Each round halves the amount of document left to suspect, the same binary-search logic used well outside LaTeX, converging on the exact broken line in a handful of recompiles instead of a linear scan.

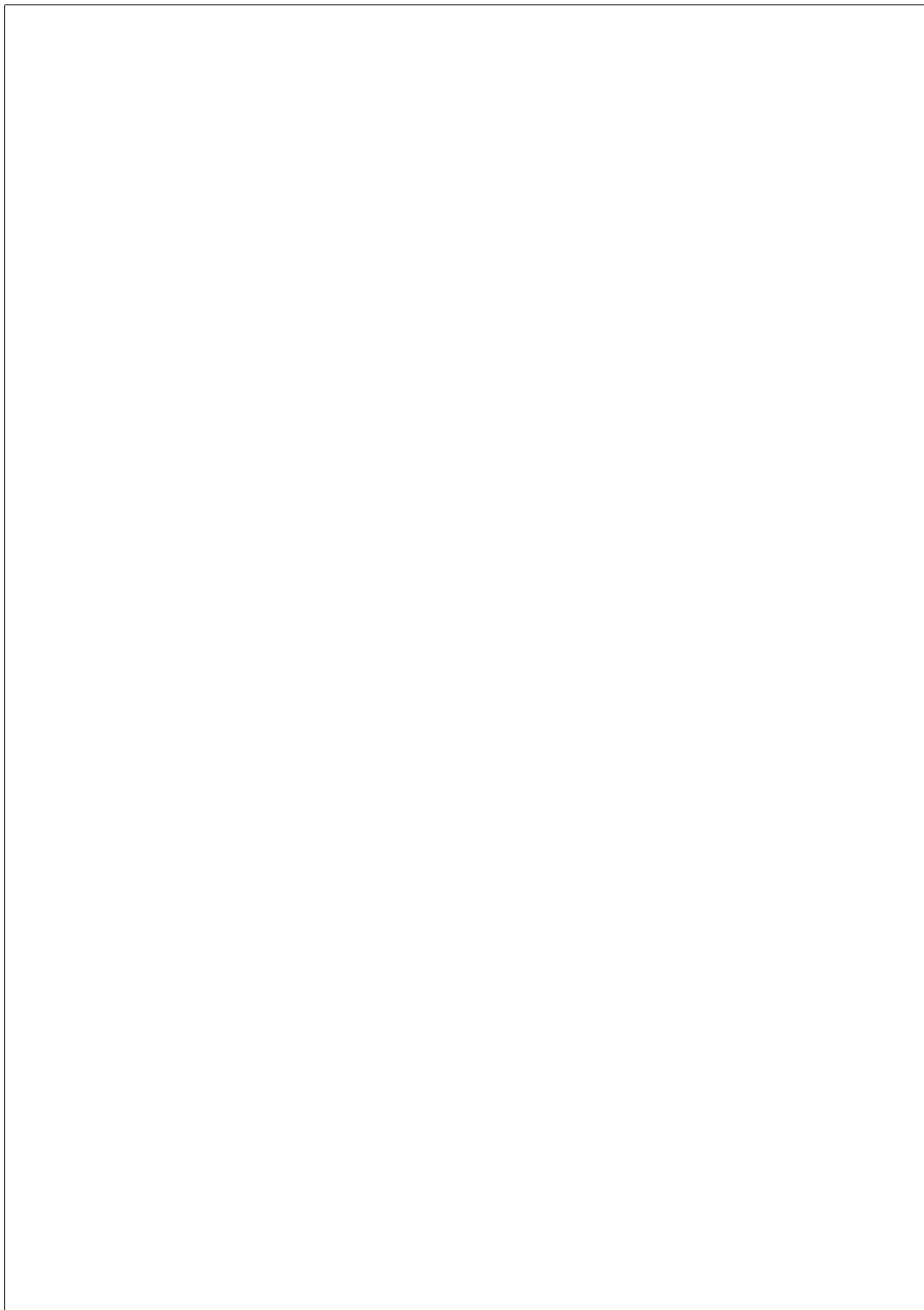
Exercise 22.1

Debug each of these independently — each has exactly one deliberate error: 1. **Easiest:** `latex \documentclass{article} \begin{document} \textbf{Hello, world.} \textit{This is a test.} \end{document}` (*Hint: one of these two formatting commands isn't a real command.*) 2. **Medium:** `latex \documentclass{article} \usepackage{amsmath} \begin{document} The result is  $x^2 + y^2 = r^2$ . \end{document}` (*Hint: count the dollar signs.*) 3. **Harder:** `latex \documentclass{article} \begin{document} \begin{itemize} \item First point \item Second point \end{enumerate} \end{document}` (*Hint: the error names an environment — but check both the $\backslash\text{begin}$ and the $\backslash\text{end}$.*)

Capstone Update 22

Intentionally break your capstone paper — remove a package it depends on, mismatch a brace, or misspell an environment name — then fix it using only the log file, without pressing undo.

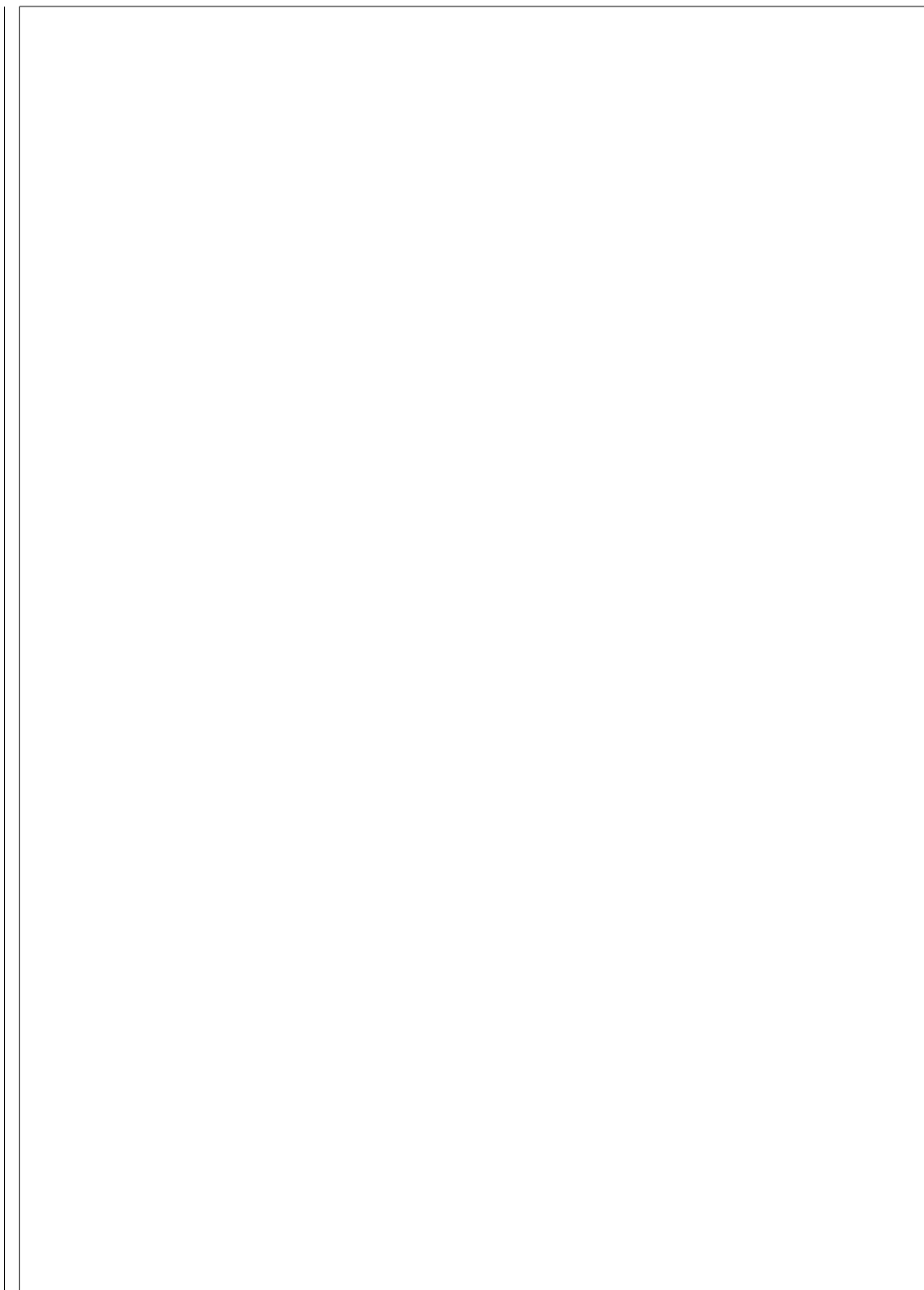
End of Part 8 (Ch. 21–22). Next: Part 9 — Capstone Assembly (Ch. 23).



Part 9: Capstone Assembly (Chapter 23)

--

This chapter introduces no new LaTeX syntax. Every command, package, and technique below was already taught in Ch. 4–22 — this chapter’s only job is to pull all of it together into one coherent, submission-ready document, and to catch the small consistency slips that creep in when a paper is built one Capstone Update at a time across 19 chapters.



Chapter 23 — Capstone Assembly

Objective 23

Assemble everything built across Ch. 4–22 into one complete, consistently formatted, submission-ready document — no new syntax, purely pulling the pieces together and finishing properly.

Worked Example 23.1

the full inventory checklist

Every Capstone Update from Ch. 4 onward added exactly one piece to your paper. Before assembling, confirm all of them are actually present, grouped here by category with their source chapter:

Project shell & front matter

- Topic, title, author, date, and abstract (Ch. 4)
- Real section headings, a settled spacing choice, and a table of contents at the right depth (Ch. 5)
- A full front-matter block — title page, table of contents, list of figures/tables, correct page numbering for your document class (Ch. 20)

Content elements

- A results table (Ch. 6)
- A figure with a proper caption and label (Ch. 7)
- Cross-references between them using `\cref`, with `colorlinks` styling applied (Ch. 8)
- The single most important equation in the paper, correctly formatted (Ch. 9)
- A derivation (2–4 steps) relevant to the topic (Ch. 10)
- One formal definition or assumption, in a `definition` environment (Ch. 11)
- A methods algorithm block, captioned and labeled (Ch. 12)

References & front-matter tooling

- A bibliography with at least 3 citations, including an online/software reference (Ch. 13)
- The citation style appropriate for your target journal (Ch. 14)
- A real reference library, via Zotero/Mendeley export (Ch. 15)
- An acronym list or nomenclature table, printed as front matter (Ch. 16)

Visuals

- A diagram — imported PDF or TikZ (Ch. 17)

- A data plot with axis labels, a legend if applicable, and error bars where relevant (Ch. 18)

Structure & hygiene

- Macros for any notation typed more than twice, with named operators via `\DeclareMathOperator` (Ch. 19)
- A proper multi-file structure with an appendix, and `comment` used to park anything not ready (Ch. 20)
- A tagged, submission-ready version (Git commit/tag or Overleaf History checkpoint) (Ch. 21)
- A clean compile with no unresolved errors (Ch. 22)

If any of these is missing, that’s the gap to close before moving on — this chapter assumes all 19 are already in your document from working through the guide in order.

Worked Example 23.2

the assembled skeleton

Rather than repeat 19 chapters’ worth of syntax here, a single compilable document threading every piece above together — in the correct structural order, using the graphene capstone topic as filler — is in `supplementary_materials.md` under “Ch. 23 — Full Assembled Skeleton.” Use it as a structural map: the order sections appear in that skeleton is the order the pieces above slot into a real paper, not an arbitrary listing order.

Nuance 23.1

a consistency pass

Nineteen chapters’ worth of Capstone Updates were written at different times, which is exactly how small inconsistencies creep in. Before considering the paper finished, check:

- **Citation style** — Ch. 14’s chosen style is used for every citation in the document, not a mix of `\cite` and `\citep` left over from Ch. 13’s first pass.
- **Number formatting** — `siunitx` (Ch. 9) is applied everywhere a number-with-unit appears, including any values added later in Ch. 10’s derivation or Ch. 18’s plot, not just the first equation it was introduced on.
- **Label prefixes** — Ch. 8’s `fig:/tab:/eq:/alg:/def:` convention is followed for every label in the document, including ones added in later chapters after the convention might have slipped your mind.
- **Spacing rules** — Ch. 5’s end-of-sentence spacing and `\\`/`\newline`/`\par` mechanics are applied consistently, not just in the sections drafted right after that chapter.

None of this is new syntax — it’s a matter of re-reading the whole document once with each of these four specifically in mind, rather than trusting that a rule followed correctly in Ch. 9 was still being followed correctly by Ch. 18.

Nuance 23.2

the final proofing pass

Separate from the consistency pass above — this one is about compile-time correctness, not stylistic drift:

- Compile **at least twice** after the last edit, so cross-references (Ch. 8), citations

(Ch. 13), and the table of contents (Ch. 5) all settle.

- Search the PDF for a literal ?? — a surviving one almost always means a reference or citation that never resolved, even after recompiling.
- Search for leftover placeholder text — the [Working Title...] style brackets from Ch. 1–4, or any “TODO” left in a `comment` block (Ch. 20) that was meant to be temporary.
- Check that floats (Ch. 6’s tables, Ch. 7’s figures) didn’t drift to an unexpected page after last-minute edits — a table that displaced to the very end of the document is easy to miss if you only reread the text, not the rendered PDF.

Worked Example 23.3

the submission checklist

Extends Ch. 21’s camera-ready checklist with the fuller picture now that every chapter’s contribution is in place:

- Every item from Ch. 21’s camera-ready checklist (draft sections wrapped in `comment`, correct compiler, `\hypersetup` set appropriately, compiled twice, no placeholder text).
- Glossary or nomenclature (Ch. 16) is present and printed, not just defined in the preamble.
- Appendix (Ch. 20) is present if the paper has any supplementary derivations, extended data, or material that doesn’t belong in the main body.
- Bibliography style (Ch. 14) matches what the target venue actually requires — not just “a style,” but the specific one named in the venue’s author guidelines.
- The consistency pass and final proofing pass above have both been run since the last substantive edit.

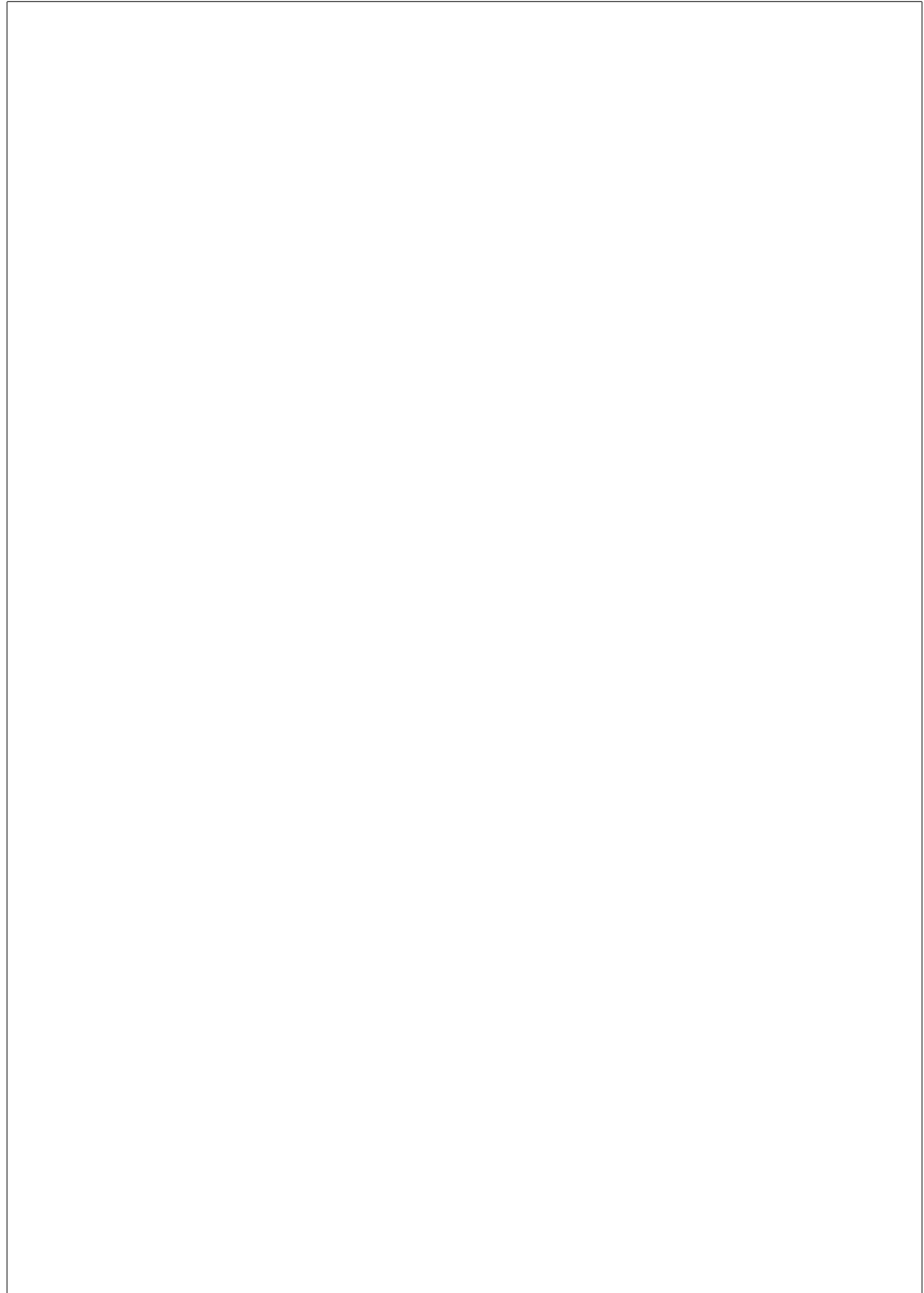
Exercise 23.1

Compile your assembled capstone paper start to finish. If anything breaks, resolve it using Ch. 22’s techniques — read the log file, identify the error category, and bisect with `comment` if the cause isn’t obvious — without pressing undo.

Capstone Update 23

Your paper is complete. Every chapter’s contribution — from the project shell in Ch. 4 to the clean compile in Ch. 22 — is now one consistently formatted, submission-ready document.

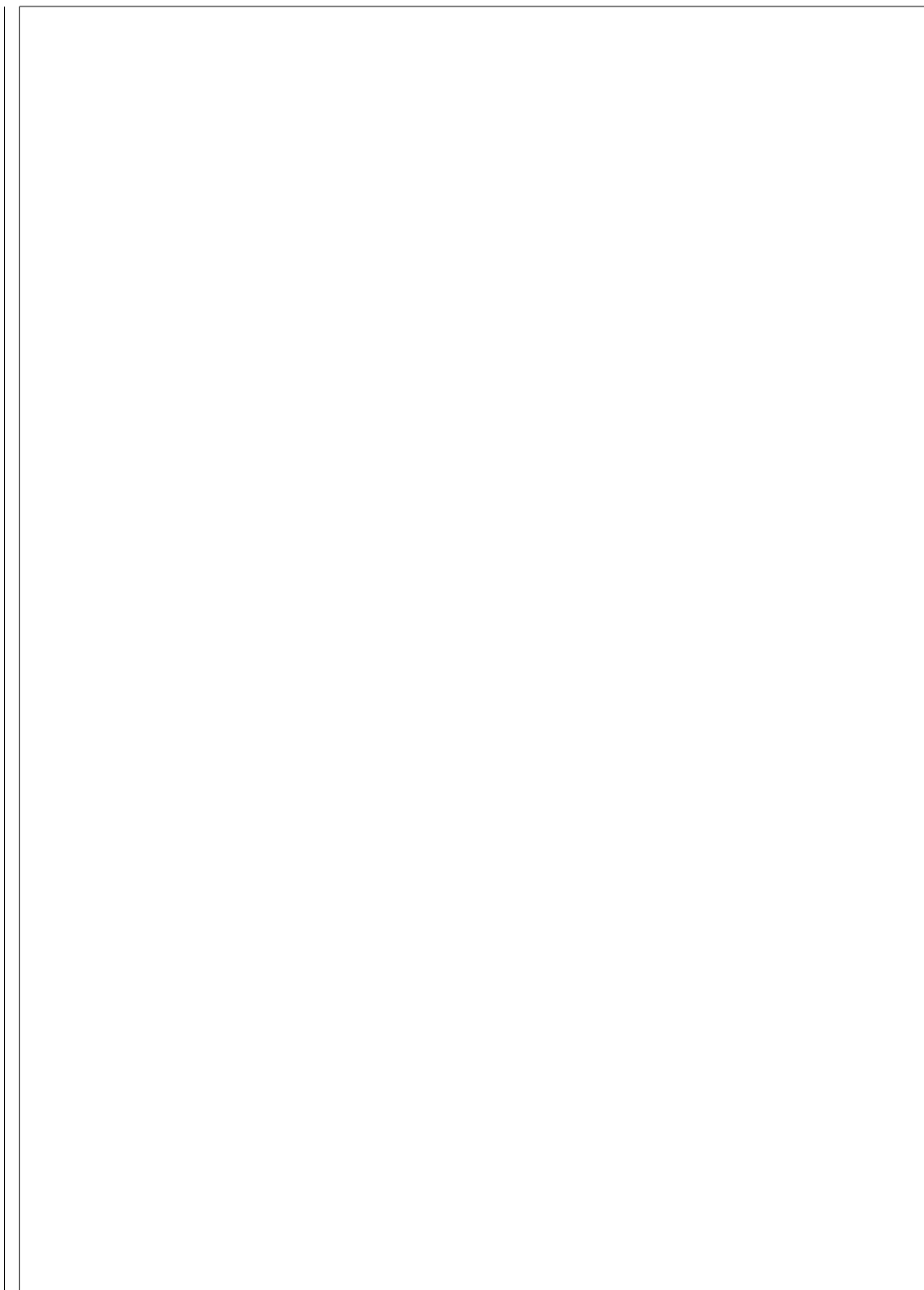
End of Part 9 (Ch. 23). This is the final chapter of the guide — see the Appendix for reference material, and `supplementary_materials.md` for the full assembled skeleton.



Appendix

--

Reference material, not narrative teaching — everything below assumes you’ve already read the chapter it points back to, and exists purely so you don’t have to go hunting for a command you’ve seen once before. Where something is fully explained elsewhere in the guide, this appendix links to it rather than re-teaching it; a few items (page geometry, dimension keywords, spacing glue) are genuinely new, since they didn’t fit naturally into any single chapter’s narrative.



A.1 — Symbol & Command Cheat Sheet

Greek letters — type the name; capitalize the command for the uppercase form where it differs from a Latin letter:

Lowercase	Command	Uppercase	Command
α	<code>\alpha</code>	—	<i>(looks like Latin A)</i>
β	<code>\beta</code>	—	<i>(looks like Latin B)</i>
γ	<code>\gamma</code>	Γ	<code>\Gamma</code>
δ	<code>\delta</code>	Δ	<code>\Delta</code>
ϵ / ε	<code>\epsilon</code> / <code>\varepsilon</code>	—	<i>(looks like Latin E)</i>
θ	<code>\theta</code>	Θ	<code>\Theta</code>
λ	<code>\lambda</code>	Λ	<code>\Lambda</code>
μ	<code>\mu</code>	—	<i>(looks like Latin M)</i>
π	<code>\pi</code>	Π	<code>\Pi</code>
σ	<code>\sigma</code>	Σ	<code>\Sigma</code>
ϕ / φ	<code>\phi</code> / <code>\varphi</code>	Φ	<code>\Phi</code>
ω	<code>\omega</code>	Ω	<code>\Omega</code>

Common math operators — built in: `\sin`, `\cos`, `\tan`, `\log`, `\ln`, `\exp`, `\max`, `\min`, `\lim`, `\sup`, `\inf`. Need one that isn't built in (`\argmin`, `\rank`, `\trace`, ...)? Declare it once with `\DeclareMathOperator` — full explanation and worked examples in Ch. 19.

Accents: `\hat{x}`, `\bar{x}`, `\dot{x}`, `\ddot{x}`, `\tilde{x}`, `\vec{x}`.

Bold and vectors: `\mathbf{x}` bolds Latin letters only — it does *not* bold Greek letters (`\mathbf{\beta}` silently fails). Use `\bm{x}` (the `bm` package) for bold Latin *and* Greek alike — full explanation in Ch. 9.

Delimiter sizing, smallest to largest: `\bigl(/ \bigr)`, `\Bigl(/ \Bigr)`, `\biggl(/ \biggr)`, `\Biggl(/ \Biggr)` — pick the size that matches your content instead of defaulting to auto-sizing `\left(/ \right)` every time. Full explanation and the one-sided-delimiter variant (`\left. ... \right|`) in Ch. 10.

Spacing in math mode: `\,` (thin space, e.g. before a differential: `\int x \, dx`), `\!` (negative thin space, pulls things together) — Ch. 10. `\quad/\qquad` (fixed 1em/2em spaces, useful for separating side conditions like “ $x > 0$ ”) — see A.3 below.

A.2 — Table Formatting Cheat Sheet

Rules — old style vs. the field standard:

% dated - hand-drawn rules on every row, vertical bars

```
\begin{tabular}{|c|c|}
\hline
A & B \\
\hline
\end{tabular}
```

% field standard - booktabs, no vertical rules

```
\begin{tabular}{cc}
\toprule
A & B \\
\bottomrule
\end{tabular}
```

Full explanation in Ch. 6.

Merging cells: `\multicolumn{cols}{align}{text}` (built into LaTeX) spans columns;
`\multirow{rows}{width}{text}` (needs `\usepackage{multirow}`) spans rows.

```
\begin{tabular}{lcc}
\toprule
\multirow{2}{*}{Sample} & \multicolumn{2}{c}{Measurement} \\
& Trial 1 & Trial 2 \\
\midrule
A & 10 & 12 \\
\bottomrule
\end{tabular}
```

Full explanation in Ch. 6.

Aligned decimals: `siunitx`'s `S` column type aligns every row's decimal point. Remember to wrap non-numeric header text in `{}`.

```
\begin{tabular}{l S[table-format=3.2]}
\toprule
{Sample} & {Value} \\
\end{tabular}
```

```

\midrule
A & 123.45 \\
B & 98.60 \\
\bottomrule
\end{tabular}

```

Full explanation in Ch. 6 and Ch. 9.

Wide or oversized tables: `sidewaystable` (from `rotating`) rotates a single float; the `landscape` environment (from `pdflscape`) rotates the whole physical page when even a rotated float isn't enough. Full explanation and examples in Ch. 6.

Scaling a table to fit the page: `\resizebox{\textwidth}{!}{...}` — use sparingly, since it shrinks the font along with everything else.

```

\resizebox{\textwidth}{!}{%
\begin{tabular}{ccccccc}
\toprule
Col1 & Col2 & Col3 & Col4 & Col5 & Col6 & Col7 & Col8 \\
\midrule
1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\
\bottomrule
\end{tabular}%
}

```

Multi-row header cell hack, paired with `\arraystretch`: nesting a small `tabular` inside a single cell wraps a long header onto two lines — reach for `multirow` first when it's a genuine data cell, and treat this as the pragmatic fallback for header labels specifically. Row height often needs loosening to match, via `\arraystretch` (full explanation in Ch. 19), scoped to just this table with an extra pair of `{ }`:

```

{\renewcommand{\arraystretch}{1.3}
\begin{tabular}{l @{} c}
\toprule
Sample & \begin{tabular}{c}{@{}c@{}}No. of\\detectors\end{tabular} \\
\midrule
A01 & 3 \\
\bottomrule
\end{tabular}
}

```

Other edge-case tools: `@{}` removes inter-column padding (used above, between `l` and the wrapped-header column); `tablefootnote` makes footnotes actually render inside a `table` float; `threeparttable` is a more formal alternative to `tablefootnote` if a table needs several notes at once. Full explanation in Ch. 6.

minipage — and how it differs from subfigure

Example 1 — two small tables side by side:

```

\begin{minipage}{0.48\textwidth}
\centering

```



```

\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
A & 12.5 \\
\bottomrule
\end{tabular}
\end{minipage}
\hfill
\begin{minipage}{0.48\textwidth}
\centering
\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
B & 7.3 \\
\bottomrule
\end{tabular}
\end{minipage}

```

Example 2 — a table next to explanatory text:

```

\begin{minipage}{0.55\textwidth}
\begin{tabular}{lc}
\toprule
Sample & Value \\
\midrule
A & 12.5 \\
\bottomrule
\end{tabular}
\end{minipage}
\hfill
\begin{minipage}{0.4\textwidth}
\small
A single measurement, included here as an example
of a table sitting beside its own explanatory note.
\end{minipage}

```

minipage vs. subfigure: both place things side by side, but they solve different problems. `subfigure` (Ch. 7, via `subcaption`) is specifically for multiple *numbered, captioned* image panels within one figure — each gets its own label (`fig:sub1`, `fig:sub2`), and the pair shares an overall caption. `minipage` is a general-purpose box with no captioning system of its own — reach for it for tables, mixed table-and-text layouts, or anything side-by-side that doesn’t need individual sub-numbering. If you’re laying out multiple images that should each get their own “Figure 1a”/“Figure 1b” style label, use `subfigure`; for everything else side-by-side, `minipage` is the simpler tool.

A.3 — Page Layout & Spacing Reference

(New material — this is genuinely reference-only content that didn't fit any single chapter's narrative, unlike the sections above.)

Page geometry — `geometry` overrides your document class's default margins:

```
\usepackage[margin=1in]{geometry}
% or, for individual sides:
\usepackage[left=1.5in, right=1in, top=1in, bottom=1in]{geometry}
% paper size, if you need something other than the class default:
\usepackage[a4paper, margin=1in]{geometry}
```

Set it once near the top of the preamble — it overrides most documentclass-level defaults. Useful the moment a university thesis format specifies exact margins (a common requirement this guide's authors ran into directly).

Dimension keywords — which one to reach for:

Keyword	Meaning	Typical use
<code>\textwidth</code>	Full width of the body text on the page	Sizing a figure/table to the whole page: <code>\includegraphics[width=\textwidth]{...}</code>
<code>\linewidth</code>	Width of the current line — adapts to context	Inside a <code>minipage</code> , <code>figure</code> , or list, where the available width is narrower than the full page
<code>\columnwidth</code>	Width of a single column	Specifically in two-column layouts (Ch. 20)

`\linewidth` is the more flexible default inside floats and lists, since it automatically adjusts; `\textwidth` stays fixed regardless of context, which occasionally causes an image to overflow a `minipage` if used there by mistake.

Spacing commands (outside math mode):

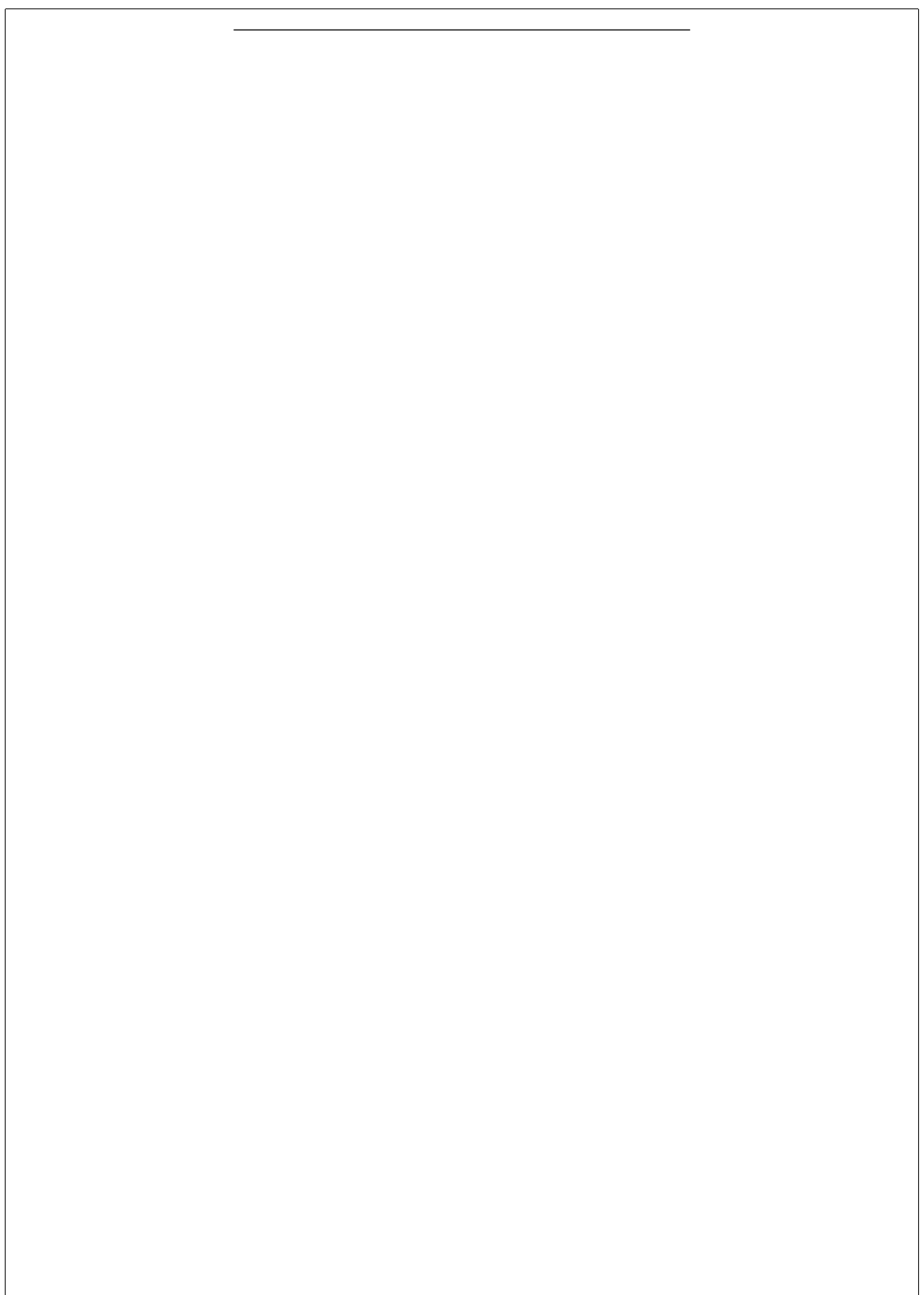
Command	Effect
<code>\hspace{1cm}</code>	Fixed horizontal space
<code>\hfill</code>	Flexible horizontal space — pushes content to opposite edges
<code>\vspace{1cm}</code>	Fixed vertical space
<code>\vspace*{1cm}</code>	Same, but survives at the top of a page (plain <code>\vspace</code> can get silently dropped there)
<code>\vfill</code>	Flexible vertical space — pushes content to the top/bottom of the page
<code>\hrule</code>	A horizontal line (rarely needed directly — use <code>booktabs</code> for tables, Ch. 6)
<code>\quad</code>	Fixed 1em space — common in math mode (Ch. 9–10), occasionally used for manual text indentation
<code>\qquad</code>	Fixed 2em space — same use cases as <code>\quad</code> , just wider

A.4 — Common Package Reference Table

Package	Purpose	First used	Load-order notes
<code>inputenc</code>	Declares UTF-8 source encoding (pdfLaTeX only)	Ch. 3	None
<code>amsmath</code>	Core math environments (<code>align</code> , <code>cases</code> , <code>split</code>), <code>\dfrac</code> / <code>\tfrac</code>	Ch. 9	Load before <code>amssymb</code>
<code>amssymb</code>	Extra math symbols, <code>\mathbb{}</code>	Ch. 19	—
<code>bm</code>	Bold Greek and Latin math symbols	Ch. 9	—
<code>siunitx</code>	<code>\SI{ }{ }</code> , aligned <code>S</code> table columns	Ch. 6, Ch. 9	—
<code>graphicx</code>	<code>\includegraphics</code> , image inclusion	Ch. 3 (exercise), Ch. 7 (taught)	—
<code>setspace</code>	<code>\doublespacing</code> / <code>\onehalfspacing</code> / <code>\singlespace</code>	Ch. 6	—
<code>enumitem</code>	Custom list labels and spacing	Ch. 6	—
<code>booktabs</code>	<code>\toprule</code> / <code>\midrule</code> / <code>\bottomrule</code>	Ch. 6	—
<code>multirow</code>	<code>\multirow{ }{ }{ }</code>	Ch. 6	—
<code>rotating</code>	<code>sidewaystable</code>	Ch. 6	—
<code>pdflscape</code>	<code>landscape</code> environment (pdfLaTeX-safe)	Ch. 6	—
<code>tablefootnote</code>	Footnotes that work inside table floats	Ch. 6	—
<code>threeparttable</code>	Formal multi-note table blocks	Ch. 6 (optional)	—
<code>epstopdf</code>	Auto-converts legacy <code>.eps</code> figures	Ch. 7 (optional)	Needs shell-escape
<code>caption</code>	Caption font/label-separator customization	Ch. 7	Load alongside <code>subcaption</code>

Package	Purpose	First used	Load-order notes
subcaption	Subfigures (subfigure package is deprecated — don't use it)	Ch. 7	—
hyperref	Clickable cross-references and links	Ch. 8	Load near the end of the preamble
cleveref	<code>\cref/\Cref</code> — auto-prefixed references	Ch. 8	Load immediately after hyperref
amsthm	<code>\newtheorem</code> , <code>\theoremstyle</code> , <code>proof</code>	Ch. 11	—
algorithm2e	Pseudocode (<code>\KwIn</code> , <code>\For</code> , <code>\If...</code>)	Ch. 12	Don't load alongside algorithm/algorithmic
algorithm+algorithmic	Alternative pseudocode package pair	Ch. 12	Don't load alongside algorithm2e
natbib	<code>\citep/\citete</code> citation commands (BibTeX backend)	Ch. 13	—
biblatex	<code>\parencite/\textcite</code> , <code>\printbibliography</code> (Biber backend)	Ch. 13	Don't load alongside natbib
url	<code>\url{}</code> for clickable plain URLs	Ch. 13	—
glossaries	Acronyms, symbols lists, <code>\gls</code>	Ch. 16	—
tikz	Vector diagrams	Ch. 17 (elective)	—
tikz-cd	Commutative diagrams	Ch. 17 (elective)	—
chemfig	Chemical structure diagrams	Ch. 17 (elective)	—
pgfplots	Data plots from CSV/inline data	Ch. 18	—
pgfplotstable	Reading external <code>.csv</code> files into pgfplots	Ch. 18	—
comment	Hide a draft block with <code>\begin{comment}...\end{comment}</code>	Ch. 20	—
geometry	Margins, paper size	A.3 (this appendix)	Set once, early in the preamble

Golden load order, restated: most packages don't care about order, but two do — **hyperref** goes near the end of the preamble, **cleveref** immediately after it, and never load both **natbib** and **biblatex**, or both **algorithm2e** and **algorithm/algorithmic**, in the same document.



--

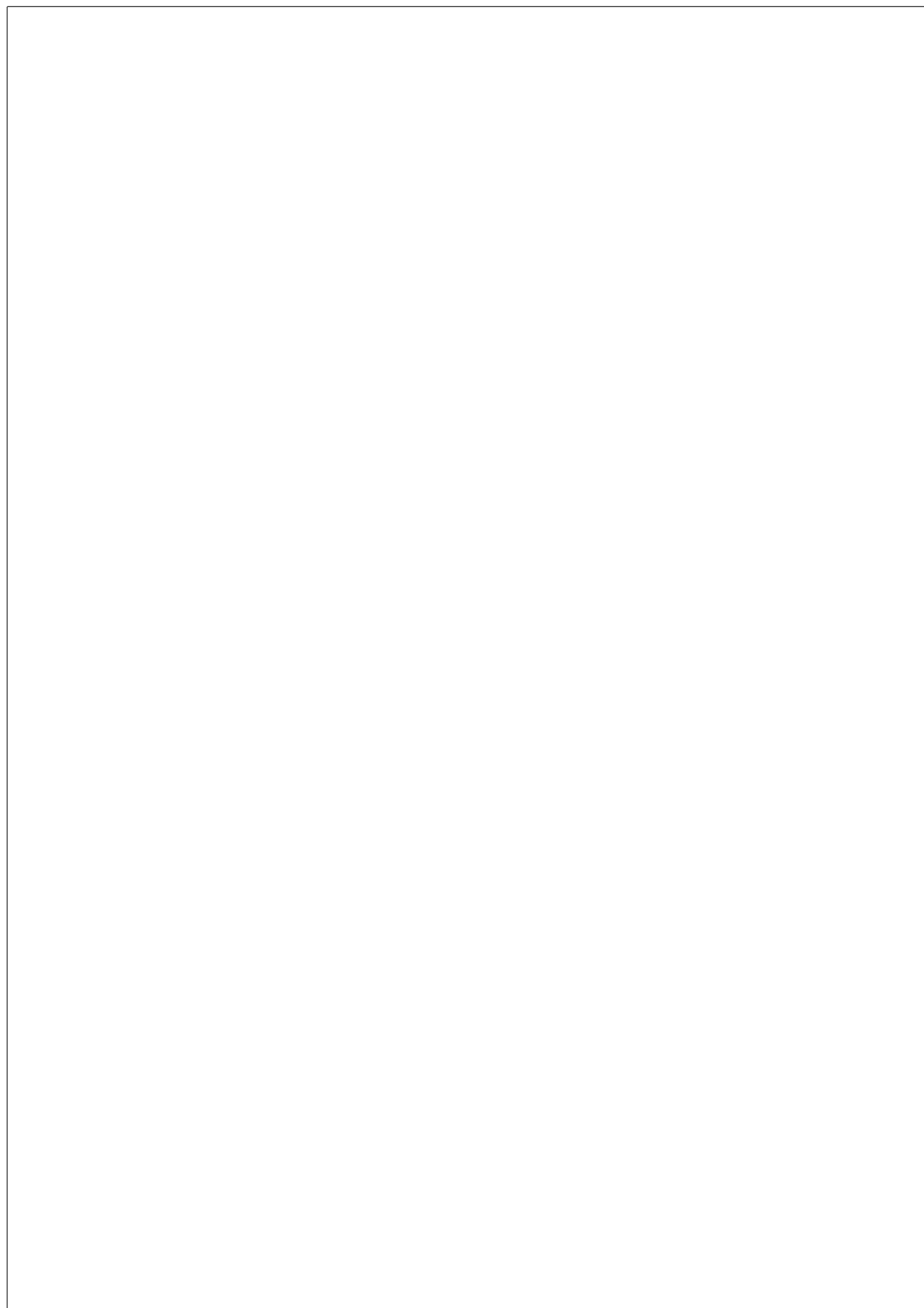
A.5 — Compiler Comparison

	pdfLaTeX	XeLaTeX	LuaLaTeX
Default in this guide	Yes	No	No
Custom fonts (fontspec)	Not supported	Supported	Supported
Unicode source files	Needs <code>inputenc</code> (Ch. 3)	Native	Native
Compile speed	Fastest	Slower	Slowest
Journal/template compatibility	Assumed by most packages and templates	Common for non-Latin scripts or custom typography	Common when Lua scripting is needed

Hard rule (first introduced in Ch. 2): if you use the `fontspec` package to change fonts, you *must* compile with XeLaTeX or LuaLaTeX — pdfLaTeX can’t process it. If you don’t need custom fonts, stick with pdfLaTeX; most journals require it for submission, and most packages assume it by default.

A.6 — `tikz-cd` and `chemfig` Starter Snippets

See Ch. 17’s “Nuance: field-specific TikZ libraries” for both.



A.7 — Quick Troubleshooting Checklist

A fast first-pass reference — Ch. 22 (once drafted) will cover diagnostics in full depth. Until then:

- **Compile fails immediately** — check the *first* red error in the log, not the last one; with “Stop on first error” enabled (Ch. 2), this is usually the very next thing Overleaf shows you.
 - **Undefined control sequence** — a typo’d command name, or a package that defines it was never loaded.
 - **Missing \$ inserted** — a math-only character ($_, ^$) used outside math mode, or an unclosed $$_.$
 - **A strange error on the line *after* where the real mistake is** — classic symptom of an unclosed brace $\{$ on the previous line; LaTeX doesn’t notice until it hits the next command (Ch. 2).
 - **?? in place of a number** — a $\backslash label/\backslash ref/\backslash cref$ mismatch; check the label spelling on both ends (Ch. 8).
 - **A figure or table lands pages away from where you placed it** — LaTeX’s float algorithm queued it; force it to flush with $\backslash clearpage$ (Ch. 7).
 - **“No citation found” / empty bibliography** — almost always a compiler/backend mismatch: BibTeX for $\backslash bibliography\{$ (`natbib`), Biber for $\backslash printbibliography$ (`biblatex`) — check Overleaf’s compiler settings (Ch. 13).
 - **Two packages fighting each other** — don’t load `natbib` and `biblatex` together, or `algorithm2e` and `algorithm/algorithmic` together (Ch. 12, Ch. 13).
-

--

A.8 — Starter .bib Files

See `supplementary_materials.md` → “Ch. 13 — Starter .bib Files for the 4 Capstone Topics.”

A.9 — `hypersetup` Snippet

Ready to paste — professional link styling instead of the default colored boxes (full explanation in Ch. 8):

```
\hypersetup{
  colorlinks=true,
  linkcolor=blue,
  citecolor=blue,
  filecolor=blue,
  urlcolor=blue,
}
```

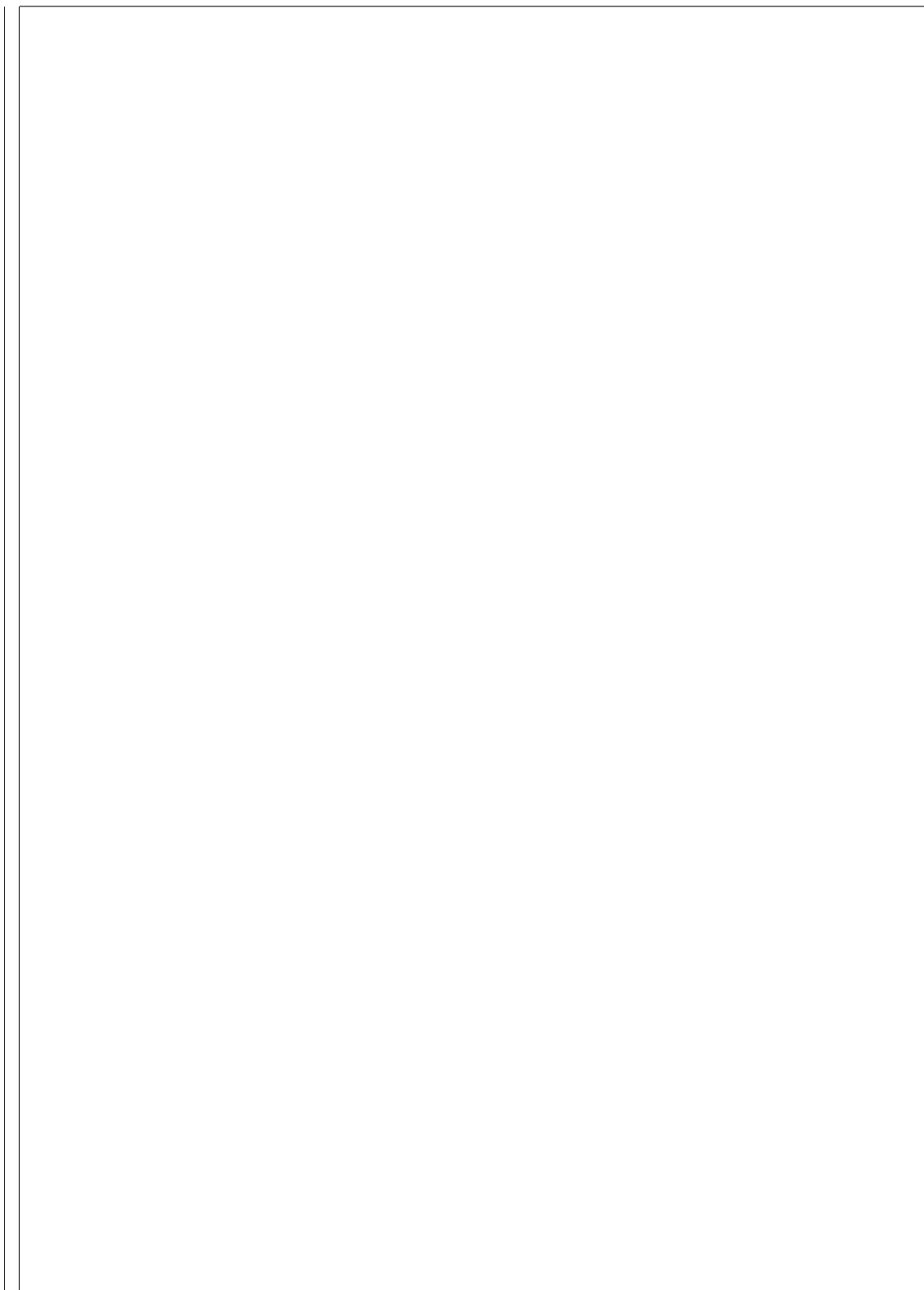
For a fully print-ready PDF with no visible link styling at all, use `hidelinks` instead of the block above.

End of Appendix.

Supplementary Materials

--

Companion assets referenced from the main chapter files — sample data, larger snippets, and anything too bulky to sit inline in a chapter without breaking its flow. Each entry is tagged with the chapter it supports.



Ch. 5 — Dense Table for Testing setspace

Ch. 5 explains `\doublespacing`, `\onehalfspacing`, `\singlespacing`, and wrapping a section in `singlespace` — but there’s nothing dense enough in that chapter to actually *see* the effect on. Paste the snippet below into your capstone project (or a scratch file) and toggle the `singlespace` wrapper on/off around the table to compare.

```
\documentclass{article}
\usepackage{setspace}

\begin{document}
\doublespacing

\section{Testing Line Spacing}
This paragraph is double-spaced by default, as set by \verb|\doublespacing|
above. Notice the extra vertical space between these lines compared to
normal single-spaced text.

\begin{singlespace}
\begin{table}[htbp]
\centering
\caption{Sample dense dataset for spacing comparison}
\label{tab:dense-demo}
\begin{tabular}{lccccc}
\hline
Sample & Trial 1 & Trial 2 & Trial 3 & Trial 4 & Mean \\
\hline
A01 & 12.3 & 11.8 & 12.5 & 12.1 & 12.18 \\
A02 & 15.6 & 15.2 & 15.9 & 15.4 & 15.53 \\
A03 & 9.8 & 10.1 & 9.6 & 9.9 & 9.85 \\
A04 & 22.1 & 21.8 & 22.4 & 22.0 & 22.08 \\
A05 & 18.4 & 18.0 & 18.7 & 18.3 & 18.35 \\
A06 & 7.2 & 7.5 & 7.1 & 7.3 & 7.28 \\
A07 & 30.5 & 30.1 & 30.8 & 30.3 & 30.43 \\
A08 & 5.9 & 6.1 & 5.8 & 6.0 & 5.95 \\
A09 & 27.3 & 27.0 & 27.6 & 27.2 & 27.28 \\
A10 & 14.0 & 13.7 & 14.3 & 13.9 & 13.98 \\
\end{tabular}
\end{table}
\end{singlespace}
```

```

\hline
\end{tabular}
\end{table}
\end{singlespace}

```

This paragraph, after the table, returns to being double-spaced again, since `\verb|\doublespacing|` is still in effect outside the `\verb|singlespace|` environment.

```

\end{document}

```

What to look for: the paragraphs before and after the table have visibly more line spacing than the ten rows inside it. Try deleting the `\begin{singlespace}/\end{singlespace}` pair and recompiling — the table rows spread out to match the double spacing and become noticeably harder to scan. That contrast is the whole point of the nuance in Ch. 5.

(Note: this table uses plain `\hline` rules rather than `booktabs`, since `booktabs` isn't introduced until Ch. 6 — feel free to swap in `\toprule/\midrule/\bottomrule` once you get there.)

Ch. 6 — Full Working Example: Table “Dark Arts” Combined

A single compilable table pulling together several of Ch. 6’s edge-case tools: `@{}`, `\resizebox`, `tablefootnote`, `minipage` (placed alongside the table), and the multi-row header hack. Realistic scenario: a table with a wrapped header, a couple of footnoted values, and a note underneath.

```
\documentclass{article}
\usepackage{booktabs}
\usepackage{graphicx}
\usepackage{tablefootnote}

\begin{document}

\begin{table}[htbp]
\centering
\caption{Detector readings by sample, with wrapped header and footnoted values}
\label{tab:dark-arts}
\resizebox{\textwidth}{!}{%
\begin{tabular}{l @{} c c c}
\toprule
Sample & \begin{tabular}{c}{@{}c@{}}No. of\detectors\end{tabular} & Reading (mV) & Uncertainty \\
\midrule
A01 & 3 & 452.1\tablefootnote{Measured at 300K.} &  $\pm 2.4$  \\
A02 & 4 & 398.7 &  $\pm 1.9$  \\
A03 & 2 & 511.0\tablefootnote{Repeated measurement; see notes.} &  $\pm 3.1$  \\
\bottomrule
\end{tabular}%
}
\end{table}

\begin{minipage}{0.9\textwidth}
\small
\textit{Note:} readings above were collected over three trials each;
see the supplementary dataset for raw values.
\end{minipage}

\end{document}
```

What each tool is doing: `\resizebox{\textwidth}{!}{...}` wraps the whole `tabular` so it scales to fit the page width — the `!` tells it to scale the height automatically to match, preserving proportions. `@{}` right after the first `l` column removes the padding between “Sample” and the wrapped-header column, since they should sit close together. The wrapped header itself is a small `tabular` nested inside a single cell — the pragmatic hack for a two-line header, versus `multirow`’s proper multi-row *data* cells. `tablefootnote` lets the two footnotes actually render at the bottom of the page instead of silently failing, which is what a plain `\footnote` would do inside this `table` float. The `minipage` below places a narrower note block directly under the table, independent of the table’s own width.

(If you need several distinct notes tied specifically to the table’s own width rather than the page’s, `threeparttable` is the more formal alternative to the `tablefootnote` calls above — it wraps the whole table plus a dedicated notes block as one unit. Worth reaching for once a table needs more than one or two footnotes.)

Ch. 7 — Full Working Example: A Stuck Float, Before and After `\clearpage`

Demonstrates the exact problem `\clearpage` solves: figures queued up with no body text to “anchor” them can drift several pages past where they were placed in the source.

```
\documentclass{article}
\usepackage{graphicx}
\usepackage{lipsum}    % dummy body text only

\begin{document}

\section{Methods}
\lipsum[1]

\begin{figure}[htbp]
\centering
\rule{0.5\textwidth}{3cm}
\caption{Experimental setup.}
\label{fig:setup}
\end{figure}

\begin{figure}[htbp]
\centering
\rule{0.5\textwidth}{3cm}
\caption{Calibration curve.}
\label{fig:calibration}
\end{figure}

% No \clearpage here yet -- both figures above are still "pending"
% and may not render before this point in the document.

\section{Results}
\lipsum[2]
```

```
\end{document}
```

Compile this version first and notice where the two figures actually land — likely not right after the Methods text, and possibly drifting into or past the Results section too, since LaTeX is still looking for the best place to slot them in.

Now add `\clearpage` right before `\section{Results}`:

```
\begin{document}
```

```
\begin{figure}
```

```
\end{figure}
```

```
\clearpage
```

```
\section{Results}
```

```
\lipsum[2]
```

```
\end{document}
```

Recompile. Both figures are now forced to place before `\clearpage` takes effect — guaranteed to appear before the Results section starts, instead of potentially drifting into it.

Ch. 9 — Reverse-Engineering Sample Targets

Ch. 9’s exercise 2 asks you to reverse-engineer a professionally typeset equation into LaTeX code. If you don’t have a paper handy, work from the five targets below instead. Each renders in the compiled PDF exactly as it should look — your job is to write the LaTeX that reproduces it, then compile and compare. They rise in difficulty and together exercise most of Ch. 9’s mechanics: subscripts and superscripts, `\frac`, roots, Greek letters, operators with limits, and spacing.

Try each one before looking at the source. Because these are typeset from LaTeX, the markup is available if you get stuck — but you’ll learn far more by getting your own version to match first.

Target 1 — the quadratic formula. Exercises `\frac`, `\sqrt`, `\pm`, and a superscript.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Target 2 — the derivative as a limit. Exercises `\lim` with an underset, a nested `\frac`, and function-argument parentheses.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Target 3 — the normal distribution’s density. Exercises Greek letters (`\mu`, `\sigma`, `\pi`), an exponential with a stacked fraction in the exponent, and `\sqrt`.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Target 4 — a finite sum with bounds. Exercises `\sum` with sub/superscript limits and a squared term.

$$S_n = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Target 5 — a physical quantity with units. Exercises subscripts, a superscript on a unit, and the thin-space `\,` between value and unit (Ch. 9’s units nuance).

$$E_k = \frac{1}{2}mv^2 = 3.2 \times 10^{-19} \text{ J}$$

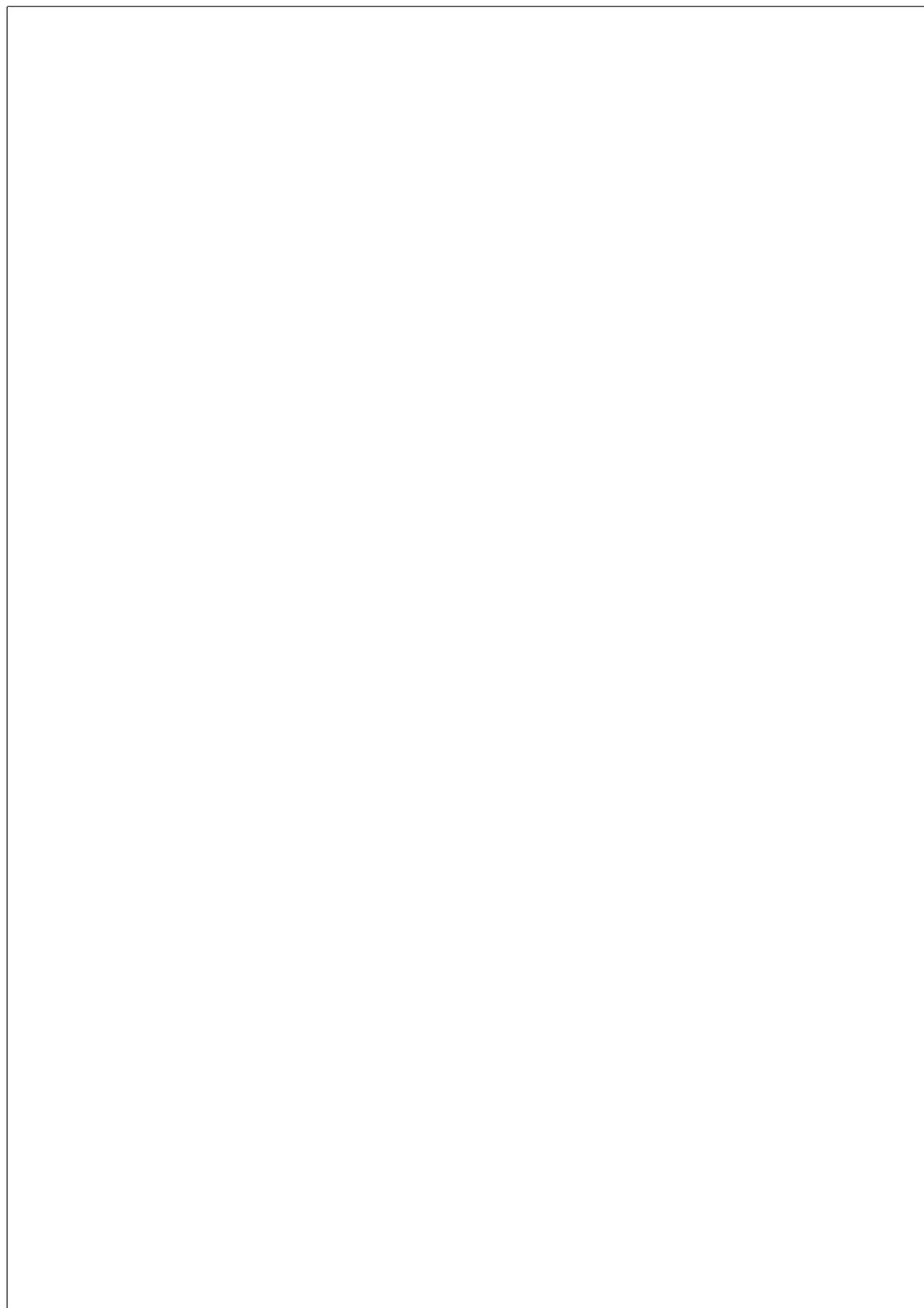
(No answer key is provided on purpose — the point is to build the markup yourself. The \$\$\dots\$\$ source above is your check.)

Ch. 11 — Simple Theorem Bank for Beginners

Ch. 11's exercise asks you to state and prove your own lemma. If you'd rather work from existing, uncontroversial material than invent something from scratch, here are a few classic, easy statements to pick from — pick one and typeset it as a **theorem**, **lemma**, or **definition**:

- **Pythagorean theorem:** for a right triangle with legs a , b and hypotenuse c , $a^2 + b^2 = c^2$. *(Already used as the worked example in Ch. 11 — pick a different one if you want practice beyond copying it.)*
- **Triangle inequality:** for any real numbers a , b , $|a + b| \leq |a| + |b|$.
- **Sum of two even integers is even:** if $m = 2j$ and $n = 2k$ for integers j, k , then $m + n = 2(j + k)$ is also even.
- **Product of two odd integers is odd:** if $m = 2j + 1$ and $n = 2k + 1$, then $mn = 2(2jk + j + k) + 1$ is odd.
- **Density of rationals:** between any two distinct rational numbers $p < q$, there exists another rational number r with $p < r < q$ (e.g. $r = \frac{p+q}{2}$).
- **Euclid's theorem (informal statement):** there is no largest prime number — the list of primes never ends.

These are deliberately simple — the point of the exercise is practicing the LaTeX mechanics (`\newtheorem`, `\theoremstyle`, `proof`, `\qedhere`, cross-referencing), not the mathematical difficulty.



Ch. 11 — Exercise 2, Solved (Full Working Example)

Ch. 11’s exercise 2 asks you to “state and prove a small lemma of your choosing (e.g. ‘the sum of two even integers is even’).” Below is that exact example, fully solved, as a complete, compilable document — also folding in theorem styles, a shared counter, and `\qedhere` from the rest of the chapter. Between this and the Pythagorean theorem worked example already in the main chapter, that’s two fully solved references to learn from; exercises 1 and 3 (sharing the counter yourself, and cross-referencing) are left for you to do.

```
\documentclass{article}
\usepackage{amsthm}
\usepackage{amsmath}
\usepackage{hyperref}
\usepackage{cleveref}

\theoremstyle{plain}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}[theorem]{Lemma}

\theoremstyle{definition}
\newtheorem{definition}{Definition}

\theoremstyle{remark}
\newtheorem{remark}{Remark}

\begin{document}

\begin{definition}
\label{def:even}
An integer  $n$  is even if  $n = 2k$  for some integer  $k$ .
\end{definition}

\begin{lemma}
\label{lem:sum-even}
The sum of two even integers is even.
\end{lemma}
```

Ch. 12 — Full Working Examples (Covering Most Reference-Table Commands)

Ch. 12's reference table lists nine `algorithm2e` commands, but the chapter's single worked example only exercises three of them (`\KwIn`, `\KwOut`, `\While`). The two examples below, between them, put almost every remaining command from the table into actual use — `\For`, `\If`/`\ElseIf`/`\Else`, `\Return`, and `\;` throughout. Anything not covered here (rarer commands like `\ForEach` or `\Repeat`/`\Until`) is worth looking up directly in the `algorithm2e` package documentation at this point in the guide.

A single, complete, compilable document with two algorithms — Example 1 exercises branching plus a `\For` loop; Example 2 exercises a `\While` loop with a `\Return` inside it. Between them, nearly every command in the reference table is used at least once.

```
\documentclass{article}
\usepackage{amsmath}
\usepackage[ruled,vlined]{algorithm2e}

\begin{document}

\begin{algorithm}[htbp]
\caption{Classify Samples by Threshold}
\label{alg:classify}
\KwIn{samples  $S = \{s_1, \dots, s_n\}$ , thresholds  $t_1 < t_2$ }
\KwOut{list of labels  $L$ }
 $L \leftarrow \emptyset$ ;
\For{$i \leftarrow 1$ \KwTo $n$}{
  \If{$s_i < t_1$}{
    append "Low" to  $L$ ;
  }
  \ElseIf{$s_i < t_2$}{
    append "Medium" to  $L$ ;
  }
  \Else{
    append "High" to  $L$ ;
  }
}
```

```

}
\Return{$L$}
\end{algorithm}

\begin{algorithm}[htbp]
\caption{Halving Search}
\label{alg:halving}
\KwIn{sorted list  $A$  of length  $n$ , target value  $x$ }
\KwOut{index of  $x$  in  $A$ , or  $-1$  if not found}
 $\text{lo} \leftarrow 1$ ;  $\text{hi} \leftarrow n$ ;
\While{ $\text{lo} \leq \text{hi}$ }{
   $\text{mid} \leftarrow \lfloor (\text{lo} + \text{hi}) / 2 \rfloor$ ;
  \If{ $A[\text{mid}] = x$ }{
    \Return{ $\text{mid}$ }
  }
  \ElseIf{ $A[\text{mid}] < x$ }{
     $\text{lo} \leftarrow \text{mid} + 1$ ;
  }
  \Else{
     $\text{hi} \leftarrow \text{mid} - 1$ ;
  }
}
\Return{-1}
\end{algorithm}

\end{document}

```

Between the chapter's own worked example and these two, every command in Ch. 12's reference table has now been used in a real algorithm at least once.

Ch. 13 — Starter .bib Files for the 4 Capstone Topics

Ch. 13's exercise and Capstone Update both assume you have references to work with. Rather than send you hunting for real papers before you've even seen what a .bib file looks like, here's a ready-made starter file for each of the 4 capstone topics from Ch. 4 — 5 entries each, with at least one @online (biblatex) and one @misc (software) entry per topic, matching Ch. 13's nuance on citing web resources and software.

A note on accuracy: several entries below (Balandin et al. 2008, Robbins & Monro 1951, Boyd & Vandenberghe 2004, Michaelis & Menten 1913, Lineweaver & Burk 1934, Cornish-Bowden, Timoshenko & Goodier) are real, well-known works in their respective fields, included so the starter files feel like genuine literature rather than obvious placeholders. A few others (e.g. the Hibbeler "article" entry) are fabricated for illustration. Treat this whole file as dummy practice data for learning .bib syntax — if you later switch your capstone to your own real research topic, replace these with your actual sources.

Topic 1 — Physics: Thermal conductivity of graphene composites

```
@article{balandin2008superior,
  author = {Balandin, A. A. and Ghosh, S. and Bao, W. and Calizo, I. and Teweldebrhan, D. and Miao,
  title   = {Superior Thermal Conductivity of Single-Layer Graphene},
  journal = {Nano Letters},
  year    = {2008},
  volume  = {8},
  number  = {3},
  pages   = {902--907}
}

@article{ghosh2010dimensional,
  author = {Ghosh, S. and Bao, W. and Nika, D. L. and Subrina, S. and Pokatilov, E. P. and Lau, C.
  title   = {Dimensional Crossover of Thermal Transport in Few-Layer Graphene},
  journal = {Nature Materials},
  year    = {2010},
  volume  = {9},
  pages   = {555--558}
}

@inproceedings{shahil2012graphene,
```

```

author    = {Shahil, K. M. F. and Balandin, A. A.},
title     = {Graphene--Multilayer Graphene Nanocomposites as Highly Efficient Thermal Inter},
booktitle = {Proceedings of the IEEE Nanotechnology Materials and Devices Conference},
year      = {2012},
pages     = {45--48}
}

@online{comsol2023thermal,
  author = {{COMSOL AB}},
  title  = {Heat Transfer Module User's Guide},
  year   = {2023},
  url    = {https://www.comsol.com/heat-transfer-module},
  note   = {Accessed: 2026-07-10}
}

@misc{matlab2023,
  author    = {{MathWorks}},
  title     = {MATLAB, version 9.14 (R2023a)},
  year      = {2023},
  howpublished = {Software},
  note      = {The MathWorks Inc., Natick, MA}
}

Topic 2 — CS/Math: Convergence analysis of a gradient descent algorithm

@article{robbins1951stochastic,
  author = {Robbins, H. and Monro, S.},
  title  = {A Stochastic Approximation Method},
  journal = {The Annals of Mathematical Statistics},
  year   = {1951},
  volume = {22},
  number = {3},
  pages  = {400--407}
}

@book{boyd2004convex,
  author    = {Boyd, S. and Vandenberghe, L.},
  title     = {Convex Optimization},
  publisher = {Cambridge University Press},
  year      = {2004}
}

@inproceedings{bottou2010large,
  author    = {Bottou, L.},
  title     = {Large-Scale Machine Learning with Stochastic Gradient Descent},
  booktitle = {Proceedings of COMPSTAT'2010},
  year      = {2010},
  pages     = {177--186}
}

```

```
@online{ruder2016overview,
  author = {Ruder, S.},
  title  = {An Overview of Gradient Descent Optimization Algorithms},
  year   = {2016},
  url    = {https://ruder.io/optimizing-gradient-descent/},
  note   = {Accessed: 2026-07-10}
}
```

```
@misc{python2023,
  author      = {{Python Software Foundation}},
  title       = {Python Language Reference, version 3.11},
  year        = {2023},
  howpublished = {Software}
}
```

Topic 3 — Engineering: Stress-strain analysis of a cantilever beam

```
@book{timoshenko1970theory,
  author   = {Timoshenko, S. P. and Goodier, J. N.},
  title    = {Theory of Elasticity},
  publisher = {McGraw-Hill},
  year     = {1970},
  edition  = {3rd}
}
```

```
@book{gere2012mechanics,
  author   = {Gere, J. M. and Goodno, B. J.},
  title    = {Mechanics of Materials},
  publisher = {Cengage Learning},
  year     = {2012},
  edition  = {8th}
}
```

```
@article{hibbeler2016cantilever,
  author   = {Hibbeler, R. C.},
  title    = {Deflection Analysis of Cantilever Beams under Distributed Loading},
  journal  = {Journal of Structural Engineering},
  year     = {2016},
  volume   = {142},
  number   = {4},
  pages    = {112--120}
}
```

```
@online{ansys2023beam,
  author = {{ANSYS Inc.}},
  title  = {Structural Analysis Guide: Beam Elements},
  year   = {2023},
  url    = {https://www.ansys.com/products/structures},
  note   = {Accessed: 2026-07-10}
}
```

```
@misc{solidworks2023,  
  author      = {{Dassault Syst`emes}},  
  title       = {SolidWorks Simulation, 2023 Edition},  
  year        = {2023},  
  howpublished = {Software}  
}
```

Topic 4 — Biology/Chem: Reaction rate of enzyme kinetics

```
@article{michaelis1913kinetik,  
  author = {Michaelis, L. and Menten, M. L.},  
  title  = {Die Kinetik der Invertinwirkung},  
  journal = {Biochemische Zeitschrift},  
  year   = {1913},  
  volume = {49},  
  pages  = {333--369}  
}
```

```
@article{lineweaver1934determination,  
  author = {Lineweaver, H. and Burk, D.},  
  title  = {The Determination of Enzyme Dissociation Constants},  
  journal = {Journal of the American Chemical Society},  
  year   = {1934},  
  volume = {56},  
  number = {3},  
  pages  = {658--666}  
}
```

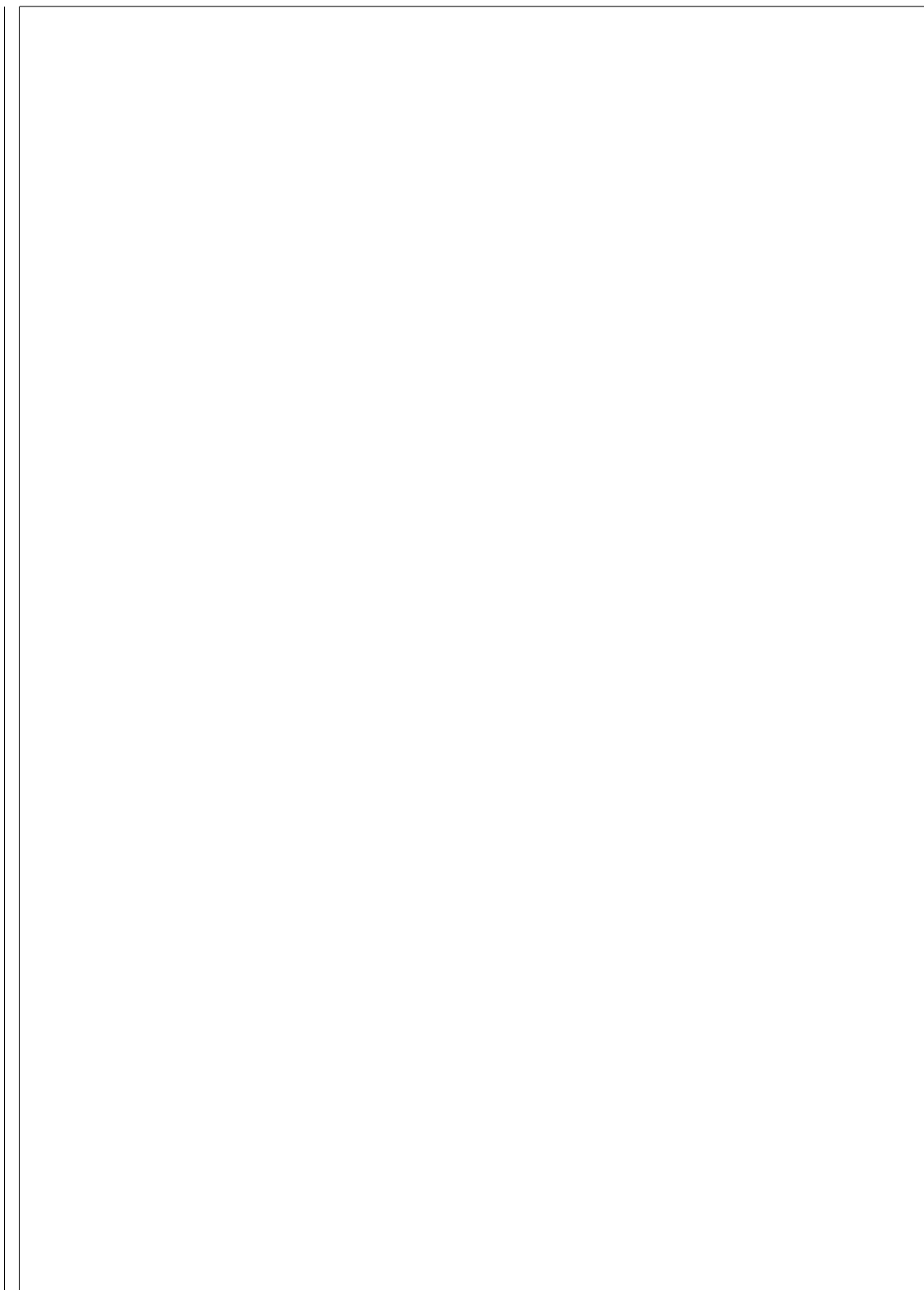
```
@book{cornishbowden2012fundamentals,  
  author   = {Cornish-Bowden, A.},  
  title    = {Fundamentals of Enzyme Kinetics},  
  publisher = {Wiley-Blackwell},  
  year     = {2012},  
  edition  = {4th}  
}
```

```
@online{nist2023enzyme,  
  author = {{National Institute of Standards and Technology}},  
  title  = {Enzyme Kinetics Reference Database},  
  year   = {2023},  
  url    = {https://www.nist.gov/enzyme-kinetics-reference},  
  note   = {Accessed: 2026-07-10}  
}
```

```
@misc{graphpad2023,  
  author      = {{GraphPad Software}},  
  title       = {GraphPad Prism, version 10},  
  year        = {2023},  
  howpublished = {Software}
```

}

Using these with plain BibTeX instead of biblatex: rename every @online entry above to @misc, and move its url field into howpublished = {\url{...}} — per Ch. 13’s nuance on the @online/@misc distinction.



Ch. 16 — Glossary Printing Command Reference

Quick reference for the most common `\printglossary` scenarios, assuming the setup from Ch. 16 (`\usepackage[acronym]{glossaries}`, plus `\newglossary[slg]{symbols}{sym}{sbl}{List of Symbols}` if you're also using a symbols list):

% Acronyms only:

```
\printglossary[type=\acronymtype]
```

% A symbols list, if you declared one with \newglossary and used type=symbols:

```
\printglossary[type=symbols]
```

% Everything defined without an explicit type= (the default glossary):

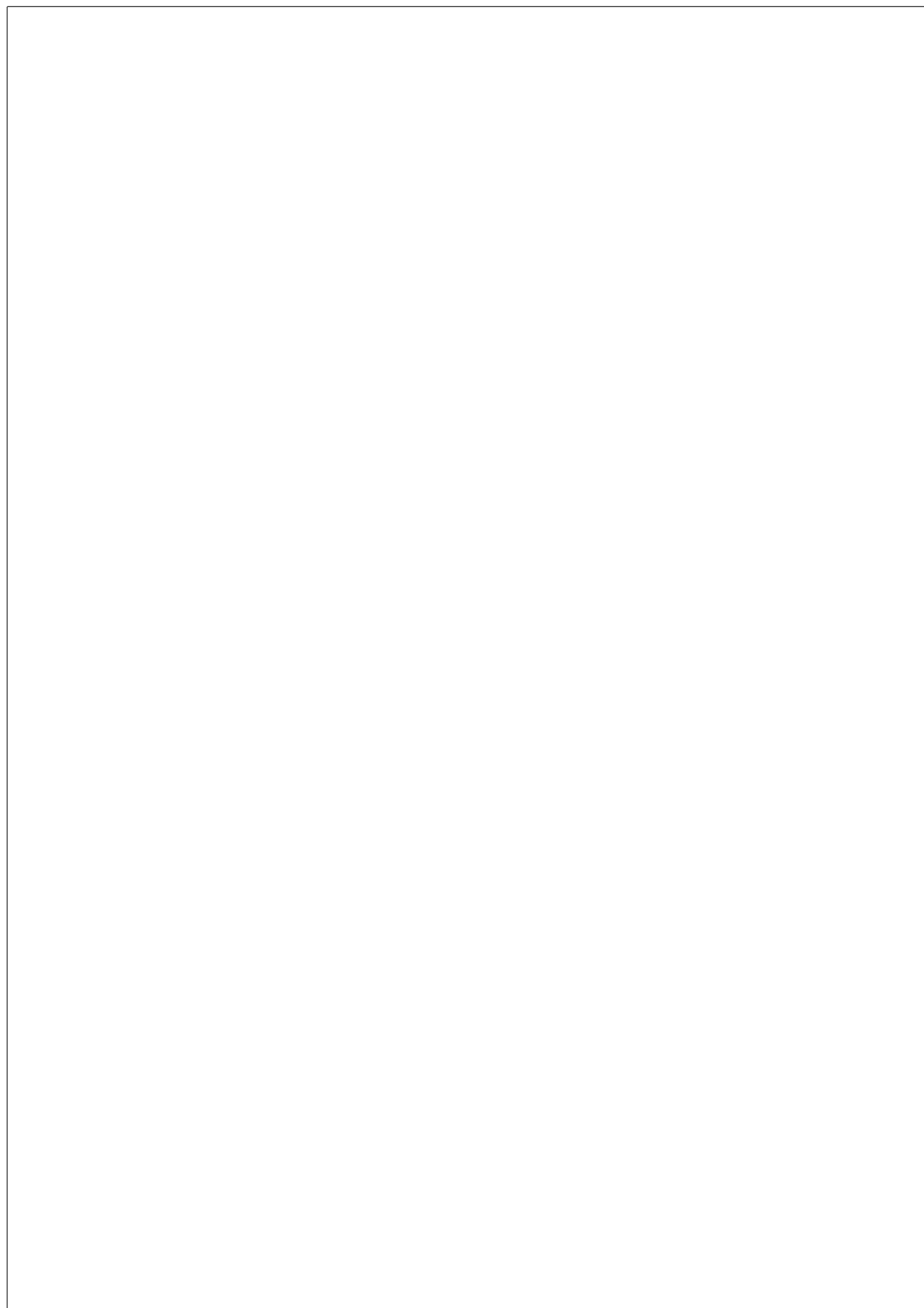
```
\printglossary
```

% Separate, titled lists for both acronyms and symbols in the same document:

```
\printglossary[type=\acronymtype, title={List of Acronyms}]
```

```
\printglossary[type=symbols, title={List of Symbols}]
```

Remember: `\printglossary` with no `type=` only prints entries that were *also* defined with no `type=`. If every entry you defined has an explicit `type=`, a bare `\printglossary` call will print nothing — always match the `type=` on the way in with the `type=` on the way out.



Ch. 17 — Full Working Example: Neural Network Architecture Diagram

Ch. 17's own worked example is a deliberately minimal 3-box pipeline, since TikZ is flagged as elective and the chapter doesn't want to scare anyone off. If you want to see how far TikZ scales when a diagram actually needs it, here's a fully-commented, more complex example: a fully-connected feedforward network with 4 inputs, two hidden layers of 3 neurons each, and 1 output.

```
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{positioning}

\begin{document}

\begin{figure}[htbp]
\centering
% \layersep controls the horizontal spacing between layers. Defined as a
% plain macro (not a TikZ style option) so it can be reused in coordinate
% arithmetic below.
\def\layersep{2.5cm}

\begin{tikzpicture}[
  neuron/.style={draw, circle, minimum size=0.8cm},
  every node/.style={font=\small}
]

% --- Input layer: 4 neurons, stacked vertically and labeled x_1..x_4 ---
\foreach \i in {1,...,4} {
  \node[neuron] (I-\i) at (0, -\i*1.2) {$x_{\i}$};
}

% --- Hidden layer 1: 3 neurons, vertically offset to sit roughly centered
% relative to the 4 inputs above ---
\foreach \i in {1,...,3} {
```

```

\documentclass{standalone}
\usepackage{tikz}
\usetikzlibrary{positioning}

\begin{tikzpicture}
  \node[neuron] (H1-\i) at (\layersep, -\i*1.2 - 0.6) {};
}

% --- Hidden layer 2: 3 neurons, same vertical layout as hidden layer 1 ---
\foreach \i in {1,...,3} {
  \node[neuron] (H2-\i) at (2*\layersep, -\i*1.2 - 0.6) {};
}

% --- Output layer: a single neuron ---
\node[neuron] (0) at (3*\layersep, -3.0) {$y$};

% --- Connections: input -> hidden layer 1, fully connected (4x3 = 12 edges) ---
\foreach \i in {1,...,4} {
  \foreach \j in {1,...,3} {
    \draw[->] (I-\i) -- (H1-\j);
  }
}

% --- Connections: hidden layer 1 -> hidden layer 2, fully connected (3x3 = 9 edges) ---
\foreach \i in {1,...,3} {
  \foreach \j in {1,...,3} {
    \draw[->] (H1-\i) -- (H2-\j);
  }
}

% --- Connections: hidden layer 2 -> output (3 edges) ---
\foreach \i in {1,...,3} {
  \draw[->] (H2-\i) -- (0);
}

% --- Layer labels, placed above the first node in each column.
% Requires the "positioning" library loaded above for the "above=...of" syntax. ---
\node[above=0.6cm of I-1, align=center] {Input\\Layer};
\node[above=0.6cm of H1-1, align=center] {Hidden\\Layer 1};
\node[above=0.6cm of H2-1, align=center] {Hidden\\Layer 2};
\node[above=0.8cm of 0, align=center] {Output};

\end{tikzpicture}
\caption{A fully connected feedforward network: 4 inputs, two hidden layers of 3 neurons each, and a single output.}
\label{fig:nn-architecture}
\end{figure}

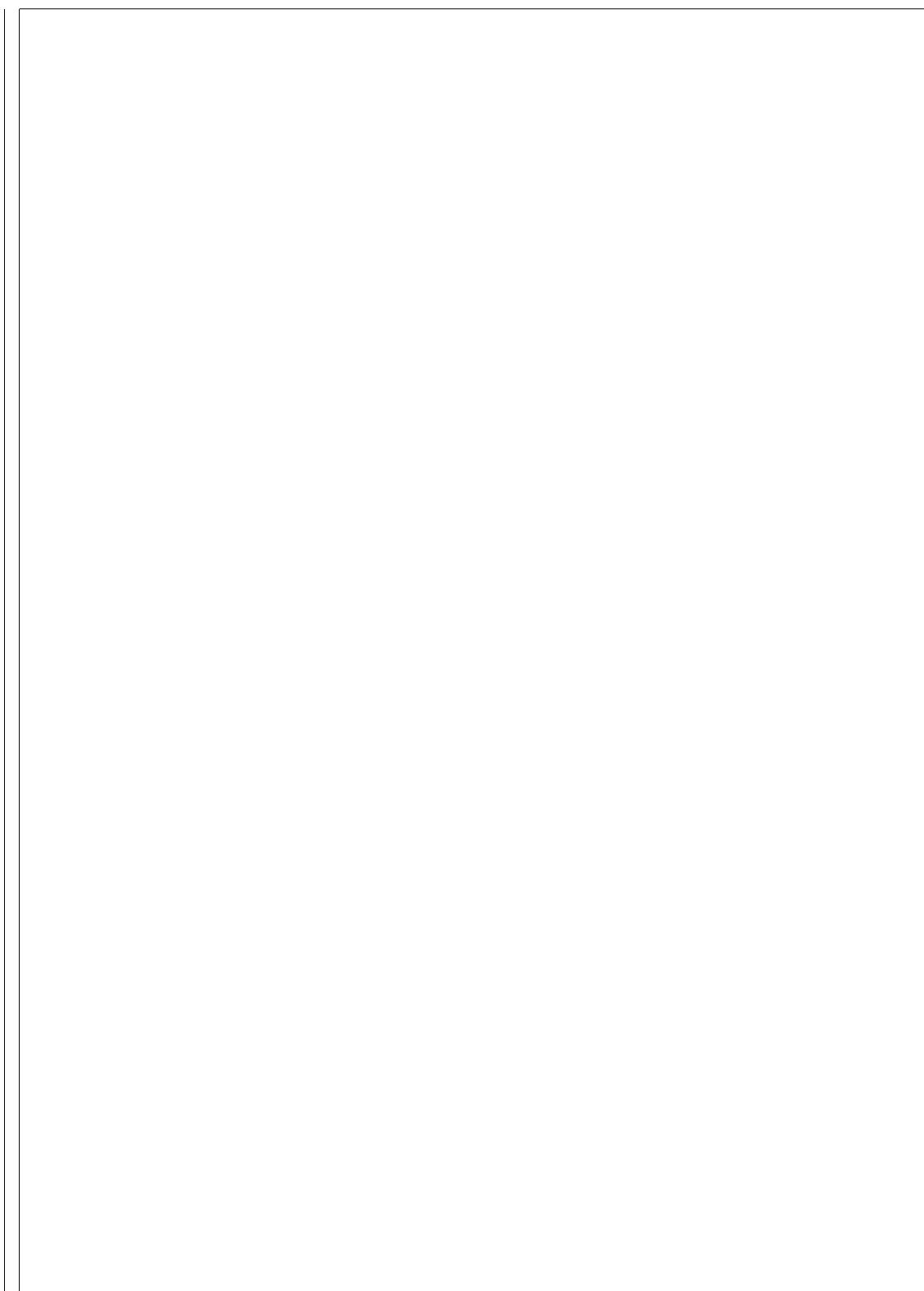
\end{document}

```

What each section is doing, in plain terms:

- The `\foreach` loops (from `pgffor`, loaded automatically with `tikz`) place each layer's neurons in one pass instead of writing out every `\node` individually — 4 inputs, 3+3 hidden neurons, and 1 output, all from four short loops.

- Each neuron is named systematically (I-1...I-4, H1-1...H1-3, H2-1...H2-3, 0) so the connection loops can refer to them by name rather than by coordinate.
 - The three connection blocks are nested `\foreach` loops — “for every neuron in this layer, draw an arrow to every neuron in the next layer” — which is what makes the network “fully connected” without hand-writing all 24 individual `\draw` commands.
 - `\layersep` is a plain TeX macro (`\def`), not a TikZ style key, which is why it can be used directly inside coordinate arithmetic like `(\layersep, ...)` and `(2*\layersep, ...)`.
 - The `positioning` library (`\usetikzlibrary{positioning}`) is what enables the `above=0.6cm of I-1` syntax used for the layer labels — without loading it, that syntax would fail.
-



Ch. 18 — Sample Dataset (ch18_sample_data.csv)

A real, standalone .csv file (not just a table to copy-paste) is included alongside this guide: `ch18_sample_data.csv`. It has 8 rows, an uncertainty column, and is shaped like enzyme-kinetics-style saturation data — usable directly with Ch. 18's `table[x=..., y=..., col sep=comma]` syntax and its error-bar syntax.

```
concentration,rate,error
1,2.1,0.15
2,3.8,0.22
3,5.2,0.18
4,6.9,0.25
5,8.1,0.20
6,8.9,0.30
7,9.3,0.28
8,9.5,0.32
```

Ch. 18 — Full Working Example (Complete Document)

A single, complete, compilable document that reads `ch18_sample_data.csv` directly (rather than typing data inline) and plots it with error bars — put this `.tex` file in the same Overleaf project folder as `ch18_sample_data.csv` for it to compile.

```
\documentclass{article}
\usepackage{pgfplots}
\usepackage{pgfplotstable}
\pgfplotsset{compat=1.18}

\begin{document}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}
\begin{axis}[
    xlabel={Concentration (mM)},
    ylabel={Reaction Rate},
]
\addplot+[
    error bars/.cd,
    y dir=both,
    y explicit,
] table[
    x=concentration,
    y=rate,
    y error=error,
    col sep=comma
] {ch18_sample_data.csv};
\end{axis}
\end{tikzpicture}
\caption{Reaction rate vs. \ substrate concentration, with error bars
read directly from \texttt{ch18\_sample\_data.csv}.}
\label{fig:enzyme-kinetics}
\end{figure}
```

```
\end{document}
```

y **error=error** tells pgfplots to pull the error-bar size for each point from the CSV's **error** column, instead of typing +- (x,y) values by hand as in the chapter's inline example — the natural approach once your uncertainties are already sitting in a data file alongside the measurements themselves.

Ch. 20 — Full Working Example: Two-Column Layout with a Spanning Figure

Ch. 20's two-column nuance covers `\onecolumn/\twocolumn` and `figure*/table*` as isolated snippets — this shows them working together in a compilable two-column article, including the placement restriction the chapter mentions (starred floats reliably support only `[t]/[p]`).

```
\documentclass[twocolumn]{article}
\usepackage{graphicx}
\usepackage{lipsum}    % dummy body text only, to make column wrapping visible - remove in a real doc

\begin{document}

\title{A Two-Column Layout Example}
\author{Your Name}
\maketitle

\section{Introduction}
\lipsum[1]

\begin{figure*}[t]
\centering
\rule{0.9\textwidth}{4cm}    % placeholder box standing in for a wide diagram
\caption{A wide figure spanning both columns - for instance, a full
experimental setup diagram or a multi-panel results figure.}
\label{fig:wide-example}
\end{figure*}

\section{Methods}
\lipsum[2]

\begin{figure}[htbp]
\centering
\rule{0.4\textwidth}{3cm}    % placeholder box standing in for a single-column figure
\caption{A regular figure, confined to one column.}
```

```
\label{fig:narrow-example}  
\end{figure}  
  
\lipsum[3]  
  
\end{document}
```

What to notice: the `figure*` sits above both columns at the top of the page, breaking the two-column flow, while the regular `figure` right after it stays confined to a single column and lets text flow around it as usual. Swap the `\rule{...}{...}` placeholders for a real `\includegraphics{...}` call once you have an actual image — the rules are here only so the example compiles without one.

(Uses `lipsum` purely to generate enough filler text for column-wrapping to actually be visible in this standalone example — not something to keep in your own document.)

Ch. 20 — Full Working Example: Front/Back-Matter Skeleton (book Class)

A minimal but complete book-class document walking through the full `\frontmatter` → `\mainmatter` → `\appendix` → `\backmatter` structure from Ch. 20, using the graphene capstone topic as filler content.

```
\documentclass[12pt]{book}
\usepackage{amsmath}

\begin{document}

\frontmatter

\title{Thermal Conductivity of Graphene Composites: A Capstone Study}
\author{Your Name}
\date{\today}
\maketitle
\thispagestyle{empty}

\tableofcontents
\listoffigures
\listoftables

\mainmatter

\chapter{Introduction}
\label{ch:intro}
This chapter introduces the capstone problem: characterizing thermal
conductivity in graphene-based composite materials.

\chapter{Methods}
\label{ch:methods}
Measurement and simulation methods used throughout this study are
described here.
```

```

\chapter{Results and Discussion}
\label{ch:results}
Results referenced from \cref{ch:methods} are presented and interpreted.

\appendix

\chapter{Supplementary Derivations}
\label{app:derivations}
Additional derivations too lengthy for the main chapters are collected
here.

\backmatter

\chapter{Bibliography}
\label{ch:bibliography}
% In a real document, replace this placeholder with \bibliography{}
% (Ch. 13) or \printbibliography (Ch. 13–16), not manually-typed content.

\end{document}

```

Walking through the structure:

- `\frontmatter` sets roman-numeral page numbers for everything that follows, up until `\mainmatter`. `\thispagestyle{empty}` right after `\maketitle` keeps the title page free of a visible page number, even though the page is still counted internally.
- `\mainmatter` resets to arabic page 1 and switches chapter numbering back on, so the three `\chapter` calls that follow become “Chapter 1,” “Chapter 2,” “Chapter 3” as expected.
- `\appendix` switches to lettered chapters (“Appendix A”) without leaving `\mainmatter`’s arabic page numbering — it doesn’t need `\mainmatter` to be called again.
- `\backmatter` switches chapters back to unnumbered automatically — a plain `\chapter{Bibliography}` here already behaves like `\chapter*{Bibliography}` would elsewhere, so there’s no need to add a `*` just because you’re in back matter.

Ch. 23 — Full Assembled Skeleton (Every Chapter, One Document)

The capstone paper itself was set up in Ch. 4 as `article` class, not `book` — so unlike the `book-class` skeleton above (which exists purely to demonstrate the `\frontmatter/\mainmatter/\backmatter` mechanism), this one builds front matter the manual way Ch. 20 covers for `article/report`. It threads every technique from Ch. 4–22 into one compilable document, using the graphene capstone topic. Kept as a single file for readability here; a real project this size would split into `\input` files per Ch. 20 (preamble, sections, bibliography) — commented pointers below mark where those splits would go.

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}                                % Ch. 3

% ----- Preamble: would live in its own \input file per Ch. 20 -----
\usepackage{amsmath, amssymb}                             % Ch. 9
\usepackage{siunitx}                                     % Ch. 9
\usepackage{bm}                                           % Ch. 9
\usepackage{graphicx}                                     % Ch. 7
\usepackage{booktabs}                                    % Ch. 6
\usepackage{enumitem}                                    % Ch. 6
\usepackage{caption}                                     % Ch. 7
\usepackage[ruled,vlined]{algorithm2e}                   % Ch. 12
\usepackage[acronym]{glossaries}                         % Ch. 16
\usepackage[style=numeric,backend=biber]{biblatex}       % Ch. 13-14
\addbibresource{graphene.bib}                           % Ch. 13 starter .bib, Topic 1
\usepackage{comment}                                     % Ch. 20
\usepackage{tikz}                                         % Ch. 17
\usepackage{pgfplots}                                    % Ch. 18
\pgfplotsset{compat=1.18}
\usepackage[colorlinks=true, linkcolor=blue, citecolor=blue]{hyperref} % Ch. 8
\usepackage{cleveref}                                    % Ch. 8

\newtheorem{definition}{Definition}                     % Ch. 11
\newcommand{\R}{\mathbb{R}}                             % Ch. 19
\DeclareMathOperator{\argmin}{arg\,min}                 % Ch. 19

\makeglossaries                                           % Ch. 16
```

```

\newacronym{tc}{TC}{Thermal Conductivity}

\title{Thermal Conductivity of Graphene-Based Composite Materials}
\author{Iqra Siddique \and Syed Ali Mohsin Bukhari}
\date{\today}

\begin{document}

% ----- Front matter: manual equivalent, article has no \frontmatter (Ch. 20) -----
\pagenumbering{roman}
\maketitle
\thispagestyle{empty}

\begin{abstract}
This capstone paper characterizes thermal conductivity (\gls{tc}) in
graphene-based composite materials, combining a steady-state
conduction model with measured results.
\end{abstract}

\tableofcontents
\listoffigures
\listoftables
\printglossary[type=\acronymtype]

\clearpage
\pagenumbering{arabic}

% ----- Body: would split into \input{sections/...} files per Ch. 20 -----
\section{Introduction}
\label{sec:intro}
Graphene composites exhibit exceptional \gls{tc}, motivating the
steady-state model developed in Section~\ref{sec:methods} and tested
against the results in Section~\ref{sec:results}.

\begin{comment}
\section{Old literature-review draft -- not ready for review}
Parked here per Ch. 20 until the related-work section is finalized.
\end{comment}

\section{Methods}
\label{sec:methods}

\begin{definition}
\label{def:steady-state}
A system is in \emph{steady-state thermal conduction} if the
temperature at every point is independent of time.
\end{definition}

\begin{algorithm}[htbp]

```

```

\caption{Finite-difference thermal conductivity estimation}
\label{alg:fd}
\KwIn{temperature grid  $T$ , thermal diffusivity  $\alpha$ , grid spacing  $\Delta x$ }
\KwOut{steady-state temperature field}
\Repeat{convergence}{
  \For{each interior grid point}{
    update  $T$  using the discretized heat equation\;
  }
}
\end{algorithm}

Heat flow (\cref{fig:setup}) follows Fourier's law:
\begin{equation}
\label{eq:fourier}
q = -k \nabla T
\end{equation}
where  $k$  is reported in  $\text{si}\{\text{watt}\text{per}\text{meter}\text{per}\text{kelvin}\}$ . Expanding
\cref{eq:fourier} in one dimension gives a short derivation:
\begin{align}
q_x &= -k \frac{dT}{dx} \\
\int q_x \, dx &= -k \int dT
\end{align}

\section{Results}
\label{sec:results}

\begin{table}[htbp]
\centering
\caption{Measured thermal conductivity by composite fraction.}
\label{tab:results}
\begin{tabular}{@{}l S[table-format=2.1] @{}}
\toprule
{Sample} & { $k$  ( $\text{si}\{\text{watt}\text{per}\text{meter}\text{per}\text{kelvin}\}$ )} \\
\midrule
Pure graphene & 25.0 \\
10\% composite & 18.4 \\
20\% composite & 14.2 \\
\bottomrule
\end{tabular}
\end{table}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}
\node[draw, rectangle] (a) at (0,0) {Sample};
\node[draw, rectangle] (b) at (3,0) {Heat Source};
\draw[->] (b) -- (a);
\end{tikzpicture}
\caption{Experimental heat-flow setup.}

```

```

\label{fig:setup}
\end{figure}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}
\begin{axis}[
  xlabel={Composite fraction (\%)},
  ylabel={\k$ (\si{\watt\per\meter\per\kelvin})}
]
\addplot coordinates {(0,25.0) (10,18.4) (20,14.2)};
\end{axis}
\end{tikzpicture}
\caption{Thermal conductivity versus composite fraction (\cref{tab:results}).}
\label{fig:plot}
\end{figure}

Results agree with \citet{balandin2008superior} and confirm the trend
reported in \citep{shahil2012graphene}.

% ----- Back matter -----
\printbibliography

\appendix
\section{Extended Derivation}
\label{app:derivation}
The full multi-step derivation behind \cref{eq:fourier}, too long for
the main methods section, is collected here.

\end{document}

```

Walking through where each chapter's piece landed:

- **Preamble** (Ch. 3, 9, 19): `inputenc`, `siunitx/bm` for math conventions, and the `\R/\argmin` macros from Ch. 19 — all defined once, used throughout.
- **Front matter** (Ch. 4, 16, 20): `title/author/date/abstract` from Ch. 4, wrapped in the `manual roman-numeral/\clearpage/arabic-numeral` pattern Ch. 20 gives for non-book classes, with the acronym list (Ch. 16) printed alongside the TOC/LOF/LOT.
- **Ch. 11's definition** and **Ch. 12's algorithm2e block** open the Methods section as structured content, ahead of the prose.
- **Ch. 9's key equation** and **Ch. 10's derivation** sit together, cross-referenced with `\cref` (Ch. 8) rather than a bare `Section~\ref`.
- **Ch. 6's table** and **Ch. 7/17's diagram** and **Ch. 18's pgfplots chart** all carry labels in Ch. 8's prefix convention (`tab:`, `fig:`) and are cross-referenced from the text, not just captioned in isolation.
- **Ch. 13's citations** appear in Ch. 14's chosen style (`biblatex/numeric` here — swap for whatever your target journal requires) via `\citet/\citep`, pulling from the same starter `graphene.bib` introduced in Ch. 13.
- **Ch. 20's comment** parks an unfinished section without deleting it, and the `\appendix` block holds overflow material exactly as Ch. 20 describes.

- Not shown inline, since they're processes rather than markup: Ch. 15's reference-manager sync, Ch. 21's Git tag/History checkpoint, and Ch. 22's log-reading discipline — all still required before this skeleton is actually submission-ready, per Ch. 23's checklist.

*(Compiling this for real needs Biber, not BibTeX, per the **biblatex** choice above — Ch. 13's compiler decision tree — plus the extra **makeglossaries** run Ch. 16 flags for the glossary to populate.)*

